Reference textbook (topic outline): Fundamentals of Data Structures in C++, 2nd Edition
Ellis Horowitz, Sartaj Sahni, Dinesh Mehta

Data Structures

Full course handout

A4 · Two slides per page · Lin Cing

Contents

1. ADTs, correctness, and cost

Chapter 1 · ADTs, correctness, and cost 2

2. Linear structures and pointer operations

Chapter 2 · Arrays, sparse matrices, and strings 12

Chapter 3 · Stacks and queues 23

Chapter 4 · Linked lists 51

3. Trees, graphs, and connectivity

Chapter 5 · Trees, binary trees, and heaps 65

Chapter 6 · Graph and network algorithms 87

4. Sorting and hashing

Chapter 7 · Sorting 112

Chapter 8 · Hashing 125

5. Mergeable heaps and amortized analysis

Chapter 9 · Advanced priority queues 141

6. Balanced search trees and external indexes

Chapter 10 · Efficient binary search trees 176

Chapter 11 · Multiway search trees and external indexes 202

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Data structures: contracts, representations, and cost

A data structure is more than a storage shape. Each operation needs a contract, a state transition, and a cost model. This chapter develops all three using traceable examples.

Algorithms and programs

An algorithm gives unambiguous steps and terminates on legal inputs. A service program may run indefinitely. Here termination is part of each operation’ s contract; empty input is legal only when its behavior is specified.

Abstract values before storage

An abstract data type (ADT) specifies observable values and operations; a representation describes storage. Implementations may arrange memory differently while satisfying the same observable contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A bounded stack contract

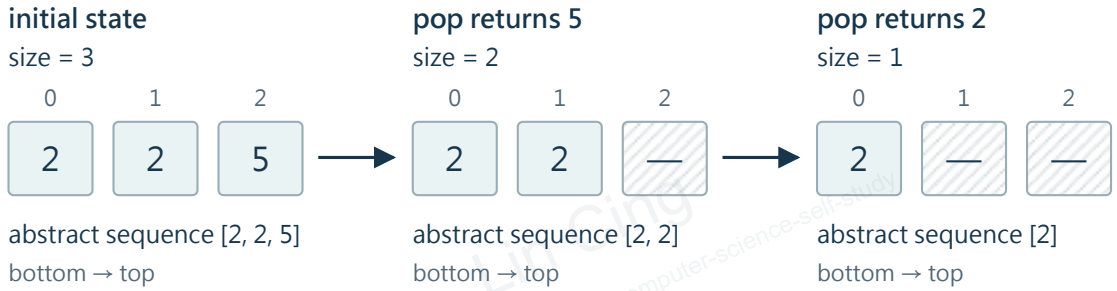
For fixed capacity $C \geq 0$, the abstract value is a finite sequence S of length at most C, with the top at the right. create(C) returns empty; isEmpty and isFull test lengths 0 and C. A successful push(x) returns success and produces $S \parallel [x]$; a successful pop returns and removes the last element. Push when full returns failure; pop when empty reports failure. Both leave S unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicates matter

A stack is an ordered sequence, not a membership set: one pop from $[2, 2]$ leaves $[2]$. The figure connects array slots to the ordered abstract sequence.

Capacity 3: an array representation maps to a sequence



α(a, size) = [a[0], ..., a[size − 1]]

Hatched slots are outside the abstract value; equal elements occupy distinct positions.

Representation invariant and abstraction function

For array a, capacity C and integer s, a legal state satisfies $0 \le s \le C$, and $a[0], \ldots, a[s-1]$ hold valid elements. The abstraction function is $\alpha(a, s) = [a[0], \ldots, a[s-1]]$. Stale values in $a[s..C)$ are not stack elements; do not read $a[C]$.

Proving push preserves the contract

Assume the invariant and $s < C$. First successfully write $a[s] = x$, then set $s \leftarrow s + 1$. The length stays within capacity and the old prefix is unchanged, so the new abstract value is the old value followed by x. If full, report failure before modifying the state.

Pop and invalid states

When $s > 0$, obtain $a[s-1]$ then decrement s; the abstract sequence loses precisely its old last element. When $s = 0$, do not read $a[-1]$. A stale old slot is outside the abstract value; $s > C$ is an invalid representation to which the abstraction function does not apply.

C++ values and exceptions

An empty pop can return `std::optional<T>` or report a specified exception. Copying or moving `T` and allocating storage may throw; do not change `size` first and assume success. The unchanged-on-failure guarantee requires suitable non-throwing operations or an explicit rollback strategy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Different search contracts

Here lower_bound uses a 0-based nondecreasing array and returns the first position $i \in [0, n]$ whose element is not less than x, or n when none exists. The Algorithms course FIRST-MATCH uses 1-based indices and returns NIL when absent. An insertion position of n is not the same contract as NIL.

The complete lower_bound invariant

For natural nondecreasing order, maintain $0 \le l \le r \le n$; every $0 \le j < l$ satisfies $A[j] < x$, and every $r \le j < n$ satisfies $A[j] \ge x$. Unclassified elements are in $[l, r)$; candidate insertion positions lie in the closed interval $[l, r]$. Return the first not-less position even with duplicates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

lower_bound: an executable update rule

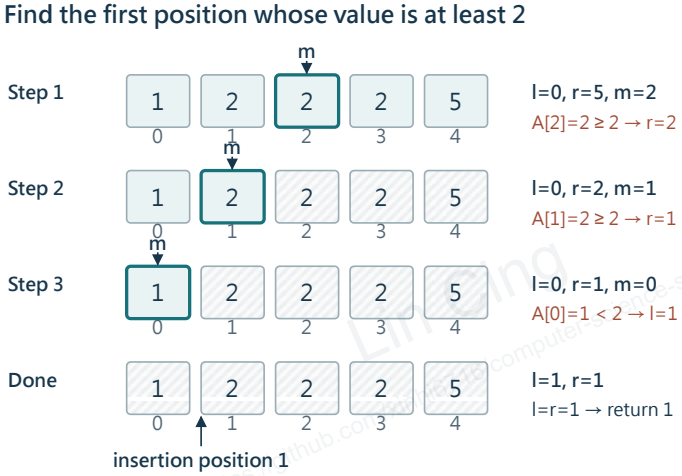
Input $A[0..n)$ is sorted in nondecreasing order. Return the first position whose value is not less than x. The right endpoint is exclusive and starts at n.

```
l ← 0;  r ← n
while l < r:
    m ← l + floor((r - l) / 2)
    if A[m] < x:  l ← m + 1
    else:         r ← m
return l
```

Initially both classified regions are empty. Each update classifies one side while retaining possible insertion positions. If every element is less than x, return n without reading $A[n]$.

Three search steps

For $A = [1, 2, 2, 2, 5]$ and $x = 2$, every update preserves the invariant. Stop at $l = r = 1$. An empty array returns 0; $x = 6$ returns 5 without reading $A[5]$.



Solid: unclassified elements [l,r); candidate positions [l,r].
Hatching marks elements, not candidate positions.

Search invariant: initialization and both updates

Let k be the first position whose value is not less than x, or n if none exists. We prove $k \in [l, r]$ at the start of every iteration.

- **Initialization:** $l = 0, r = n$, so $k \in [0, n]$. Both classified regions are empty.
- **If $A[m] < x$:** sortedness gives $A[0..m] < x$, so $k \geq m + 1$. Setting $l = m + 1$ retains k.
- **If $A[m] \geq x$:** sortedness gives $A[m..n) \geq x$, so $k \leq m$. Setting $r = m$ still retains k.
- **Termination:** $l = r$ leaves the single candidate $k = l$. The proof also covers $k = n$ and duplicate keys.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why search terminates

When $l < r$, $m = l + \lfloor (r - l)/2 \rfloor$ satisfies $l \leq m < r \leq n$, so $A[m]$ is valid. If $A[m] < x$, classify elements $[l, m]$ on the left; otherwise classify elements $[m, r)$ on the right. Position m can still be the answer. The nonnegative integer $r - l$ decreases each iteration; at $l = r$, the classified intervals imply the postcondition.

Comparators and executable semantics

A C++ comparator comp must be a strict weak ordering, and the array must be sorted by it. The left region satisfies comp(A[j],x), the right region !comp(A[j],x); the result is the first position where comp(A[i],x) is false. Check $i < n$ before reading A[i]; equivalence then requires both !comp(A[i],x) and !comp(x,A[i]), not necessarily ==. The index type must represent n and the midpoint arithmetic without overflow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Comparisons are not the whole cost

With random access and $O(1)$ comparisons, lower_bound takes $\Theta(\log(n+1))$ worst-case comparisons. If strings have at most L characters, each lexicographic comparison uses $O(L)$ character operations, giving $O(L\log(n+1))$ character operations. Including indexing and control costs, total time is $O((L+1)(1+\log(n+1)))$. An empty array makes no comparisons but still incurs constant control cost.

State the cost model

Separate fixed storage, input, output, and auxiliary space. Time may count comparisons, element moves, or bit operations. C++ `vector` allocation and construction costs depend on the element type and allocator; a unit-cost model must say so.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Additions versus bit complexity

Add n integers to an accumulator in sequence. If **both operands and the result of every addition** fit in at most L bits, grade-school bitwise addition costs $O(L)$ per step and $O(nL)$ total; $n=0$ has only initialization cost.

A short final result is not enough. For example, $2^N+(-2^N)=0$ has a constant-length result, but both inputs have $\Theta(N)$ bits; reading the complete input can already require $\Omega(N)$ time.

With fixed-width machine words and a guarantee of no overflow, a word-RAM model may instead charge $O(1)$ per addition, giving $O(n)$ total time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursion and live frames

For nonnegative n, $fac(0) = 1$ and $fac(n) = n\,fac(n-1)$. Evaluating $fac(3)$ calls 3, 2, 1, 0: four calls, three edges, and four simultaneously live frames at the deepest point. Returns are 1, 1, 2, 6. We count each call once and do not treat arbitrary-precision multiplication as free.

Two Fibonacci conventions

With $F(0) = 0, F(1) = 1, F(5) = 5$; with $G(0) = G(1) = 1$ and the same recurrence, $G(5) = 8$. Counting the initial direct-recursion call gives $C(0) = C(1) = 1$ and $C(n) = 1 + C(n-1) + C(n-2)$, so $C(5) = 15$ and $C(8) = 67$. Total calls and peak frame depth are different quantities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiplying by two is not two calls

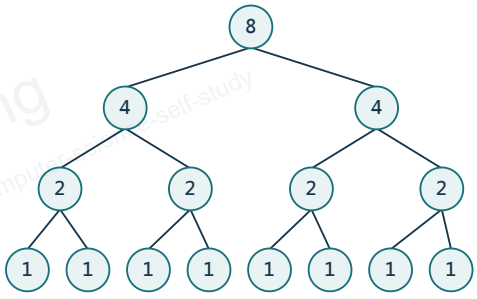
In a source-level unit-cost model, for $n = 2^k \geq 1$ and $T(1) = \Theta(1)$, `2*Test(n/2)+8` makes one recursive call and has $T(n/2) + \Theta(1) = \Theta(\log n + 1)$ time. Two separately evaluated calls yield $2T(n/2) + \Theta(1) = \Theta(n)$. No compiler common-subexpression optimization is assumed.

Multiplication by 2 does not create a second recursive call

2 × Test(n/2) + 8
one recursive evaluation per call

4 calls including base; time Θ(log n)

Test(n/2) + Test(n/2) + 8
two recursive evaluations per call



15 calls including base; time Θ(n)

Model: source-level call count without sharing repeated calls; start n=8, base n ≤ 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Count the final failed test

For fixed $n \geq 1$ and $i = 1, \ldots, n$, start $x = n$ and repeatedly set $x \leftarrow x - i$ while $x \geq 0$. The body runs exactly $\lfloor n/i \rfloor + 1$ times, plus one failed test per i. At $n = 6$, body counts are 7, 4, 3, 2, 2, 2: total 20, or 26 condition tests.

From exact counts to growth

The body count is $S(n) = \sum_{i=1}^{n} (\lfloor n/i \rfloor + 1)$. Since $n/i - 1 < \lfloor n/i \rfloor \leq n/i$, $nH_n < S(n) \leq nH_n + n$. Integral bounds give $\log(n+1) \leq H_n \leq 1 + \log n$, hence $S(n) = \Theta(n \log n)$ for $n \geq 2$; handle the empty range separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Big O, lower bounds, and cost functions

$f \in O(g)$ means there exist constants $c > 0, n_0$ such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$. Ω gives the corresponding lower bound and Θ requires both. Worst-case cost is $W(n) = \max_{|x|=n} T(x)$; the symbol O alone does not mean worst case.

Worst case, expectation, amortization

Worst-case cost maximizes over legal inputs. Expected cost needs a specified input or algorithm-randomness distribution. Amortized cost bounds a sequence of operations and allocates its total among them. These are distinct statements and may coexist for one algorithm.

Duplicate keys change quicksort

With a two-way Lomuto partition that puts equal keys on one side, an all-equal input yields $n - 1$ and 0 every time; random pivot selection still costs $\Theta(n^2)$. The standard expected $\Theta(n \log n)$ claim here assumes distinct keys and random pivots. Three-way partition separates less, equal, and greater keys and does not recurse on the equal region.

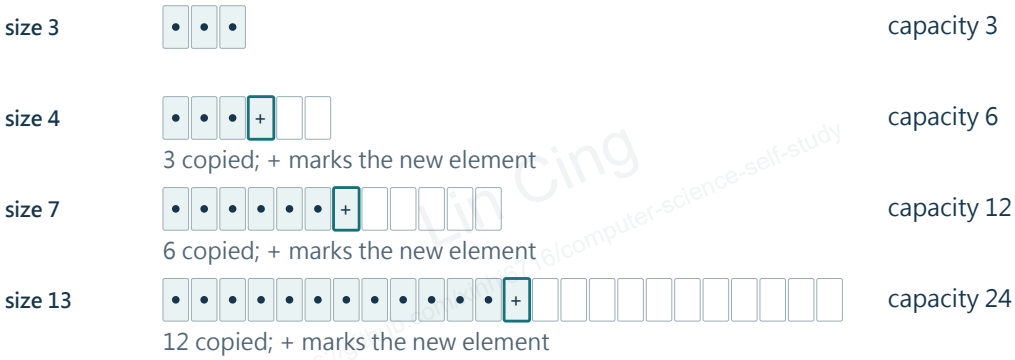
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Actual dynamic-array copies

Start with capacity $c_0 = 3$, doubling when full. Appending 13 elements copies 3, 6, and 12 elements at appends 4, 7, and 13, totaling 21. Substituting $1 + 2 + 4 + 8 = 15$ for arbitrary starting capacity undercounts this example.

Initial capacity 3; insertions 4, 7 and 13 trigger doubling

● existing + new blank = unused



Total moves = 3 + 6 + 12 = 21; amortized moves per insertion = 21/13.
Moves only; construction and allocation are separate.

Why a sequence of appends is linear

Let initial capacity $c_0 \geq 1$. At $n = 0$, both copies and writes are zero; assume $n \geq 1$ below. With no resize, the copy count is zero. Otherwise copied capacities are $c_0, 2c_0, \ldots, 2^k c_0$ and the last old capacity $c = 2^k c_0 < n$. Their geometric sum $2c - c_0 < 2n$. Including n new-element writes gives fewer than $3n$ writes. Initial allocation is excluded and must be charged separately under the allocator and construction model if counted.

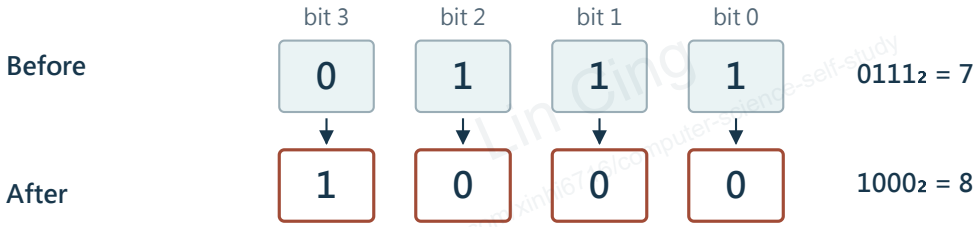
Potential method: store credit

For append-only doubling use $\Phi(s, C) = \max(0, 2s - C)$, initially zero. A non-resizing append increases potential by at most 2, giving amortized cost at most 3. At a full capacity $C > 0$, resizing costs $C + 1$ and changes potential from C to 2, giving amortized cost 3. Nonnegative potential bounds every prefix' s total actual cost by its total amortized cost.

A bit-counter potential example

Use a fixed four-bit counter modulo 16, charge one write per flipped bit, and let potential be the number of ones. $0111 \to 1000$ writes four bits and changes potential from 3 to 1, for amortized cost $4 - 2 = 2$. In general, flipping t trailing ones and setting one zero changes potential by $1 - t$, again giving 2. Wrapping all ones to zeros writes four bits and decreases potential by four.

Incrementing a binary counter: actual and amortized cost



4 bit flips: actual cost c = 4; potential Φ = number of 1 bits.

Φ before = 3, Φ after = 1; amortized cost ĉ = 4 + (1 − 3) = 2.

Assume sufficient bit width; memory allocation is excluded.

Four questions before finishing

Are empty input, duplicates and failure specified? Is there a verifiable representation invariant and abstraction function? Does the correctness argument cover preservation and termination? Does the cost claim specify the operation, input size, and assumptions for expectation or amortization? Next, apply these tools to linear structures.

References and attribution

Lin Cing, handwritten notes on data structures.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are Chapter 1's ADTs, invariants, and cost models, together with C++ object lifetimes.
- Define the Array ADT by its index domain. Derive multidimensional address formulas and explain the limits of contiguous storage.
- Trace sparse transpose and string matching. Distinguish characters from encoding units.

The Array ADT index contract

- A one-dimensional array maps a fixed index domain `{0,…,n-1}` to values. Both `get(i)` and `set(i,x)` require `0≤i<n`.
- Resizing is an extra capability of a dynamic sequence. It is not required by a fixed-length Array ADT.
- The postcondition of `set(i,x)` changes the value at i to x and preserves every other position.
- Contiguous memory is a representation, not the mathematical ADT definition. Segmented storage can offer indexed semantics with different costs.

C++ representation and lifetime

- `std::array<T,N>` has a compile-time fixed length. `std::vector<T>` can change length at runtime.
- Except for the `vector<bool>` specialization, vector elements occupy contiguous storage. `size()` counts existing elements. `capacity()` counts allocated element slots.
- `operator[]` is not a general bounds-checking mechanism. Examples check external indices before access.
- Reallocation invalidates existing element pointers, references, and iterators. `reserve` does not increase `size` to the requested capacity.

Row-major two-dimensional addresses

- For an `R×C` array of fixed-width elements of w bytes, the element offset of zero-based `(i,j)` is `iC+j`.
- With base address b, the byte address is `b+w(iC+j)`. Apply this formula only to valid elements within the same contiguous allocation.
- For the original example `R=3,C=4,w=8`, `(2,1)` has element offset 9 and byte offset 72.
- Rows of `vector<vector<T>>` may be allocated separately. The entire matrix need not be one contiguous block, so the whole-matrix formula does not generally apply.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coordinates and addresses in row-major order

Count complete rows, then columns within the row; element and byte offsets have different units.

Row-major: from coordinates to contiguous storage

Matrix 3×4; select (2,1)

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	**(2,1)**	(2,2)	(2,3)

First skip 2 full rows: 2×4
Then advance 1 column: +1

offset = 2×4 + 1 = 9

Contiguous element slots (numbers are element offsets)

0	1	2	3	4	5	6	7	8	**9**	10	11
									↑		

Element width 8 bytes: byte offset 9×8=72; address b+72.

Heavy borders and arrow mark the target; the formula assumes one contiguous storage block.

Multidimensional strides: row-major and column-major

For dimensions $D_0 \times \cdots \times D_{d-1}$ and valid indices $0 \le i_t < D_t$, row-major changes the last index fastest. Advancing index t skips the product of the later dimensions:

$$\text{offset}_{row} = \sum_{t=0}^{d-1} i_t \prod_{u=t+1}^{d-1} D_u.$$

Column-major changes the first index fastest; its stride is the product of earlier dimensions:

$$\text{offset}_{col} = \sum_{t=0}^{d-1} i_t \prod_{u=0}^{t-1} D_u.$$

An empty product is 1. Multiply the element offset by width w to get the byte offset; check dimension products and index ranges before allocation or address arithmetic.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Traversal order and cache behavior

- In row-major storage, increasing column index j in the inner loop reads adjacent elements.
- Increasing row index i in the inner loop instead gives a stride of C elements. It may reduce reuse of each cache line.
- Both traversals take `Θ(RC)` RAM operations. Actual times may differ. No cache speed ratio is claimed without measurement.
- Check multiplication bounds before allocating if `R×C` can overflow the index type. A valid logical shape does not guarantee a feasible allocation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

COO and CSR sparse representations

- `nnz=k` counts nonzero entries. COO stores triples `(row,column,value)` and separately retains the matrix dimensions.
- Our COO invariant requires row-major order, valid indices, unique coordinates, and no explicit zeros. Normalize other input by sorting, combining duplicate coordinates, and removing zeros.
- CSR stores `row_ptr`, `col_idx`, and `values`. Row i occupies `[row_ptr[i],row_ptr[i+1])`.
- CSR space is `Θ(R+k)`. The R term matters when many rows are empty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked sparse example

The matrix is 3×4. Entry IDs connect the nonzeros across representations. Row 1 has CSR interval $[1, 3)$, holding -2 and 7. For $x = [1, 2, 3, 4]^T$, $Ax = [10, 26, 8]^T$.

Sparse matrix: use entry IDs to connect COO and CSR

Matrix A

	0	1	2	3
0	0	#0 5	0	0
1	#1 -2	0	0	#2 7
2	0	#3 4	0	0

COO (row, col, value)

- #0 (0, 1, 5)
- #1 (1, 0, -2)
- #2 (1, 3, 7)
- #3 (2, 1, 4)

CSR: entry IDs match, even without color

	#0	#1	#2	#3
col_idx	1	0	3	1
values	5	-2	7	4

row_ptr = [0, 1, 3, 4]
row 1: [1,3) → #1, #2
#1: (1,0,−2) #2: (1,3,7)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fast transpose counts columns

1. Count original entries in each column as `count[j]`.
2. Prefix-sum the counts to obtain starting positions `start[j]` for the transposed rows.
3. Scan original COO order. Write `(j,i,v)` at `next[j]` for entry `(i,j,v)`, then increment `next[j]`.
4. Preserve `start` as immutable interval boundaries. Do not overwrite it with the moving cursors.

Time is `O(C+k)`, auxiliary space is `O(C)`, and output space is `O(k)`. A complete output CSR also requires `C+1` row-pointer positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why fast transpose does not overwrite entries

- Column counts are `[1,2,0,1]`. Prefix starting positions are `[0,1,3,3]`.
- Column output intervals are disjoint, and each length equals its number of entries. Every input entry therefore has one unique output slot.
- Stable distribution in input row order keeps column indices increasing within each transposed row.
- The result is `(0,1,-2)`, `(1,0,5)`, `(1,2,4)`, `(3,1,7)`. An empty column naturally becomes an empty interval.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fast transpose: each output write

Follow the prefix boundaries and cursor for each write; an empty column occupies no output slot.

Fast transpose: count, partition, and stable scatter

count = [1, 2, 0, 1] start = [0, 1, 3, 3, 4]

next = [0, 1, 3, 3] (before scatter)

Input COO	Column	Output slot	Transposed item	next after write
(0,1,5)	1	1	(1,0,5)	[0,2,3,3]
(1,0,−2)	0	0	(0,1,−2)	[1,2,3,3]
(1,3,7)	3	3	(3,1,7)	[1,2,3,4]
(2,1,4)	1	2	(1,2,4)	[1,3,3,4]

Output in slot order 0 → 3:
(0,1,−2) (1,0,5) (1,2,4) (3,1,7)

Column 2 has count 0, so [3,3) is empty; original row order is preserved within each block. start keeps the boundaries; next points to the next writable slot.

Complete COO transpose allocation and cursors

```
transpose(R,C,entries):
    require sorted unique valid nonzero entries
    count=[0]*C
    for (r,c,v) in entries: count[c]+=1
    start=[0]*(C+1)
    for c=0..C-1: start[c+1]=start[c]+count[c]
    next=copy(start[0:C]); out=array(len(entries))
    for (r,c,v) in entries:
        out[next[c]]=(c,r,v); next[c]+=1
    return (C,R,out)
```

For the same 3×4 example, (0,1,5) writes out[1], (1,0,−2) writes out[0], (1,3,7) writes out[3], and (2,1,4) writes out[2]. Start is [0,1,3,3,4]; final next=[1,3,3,4] equals the interval ends. Disjoint intervals with exactly matching write counts prevent overwrites and omissions. Stable distribution by original row preserves sorted columns within each transposed row.

Empty column 2 has interval [3,3). With R=0, C=4, retain result dimensions 4×0. Time is Θ(C+k), output space Θ(k), and auxiliary space Θ(C). Normalizing unsorted or duplicate input adds separate sorting/combination work. Sparse addition also requires equal dimensions, not merely equal nnz.

Sparse arithmetic costs

- Add two sorted COO inputs by advancing as in merge sort. Combine equal coordinates and omit a sum of zero.
- For input nonzero counts k_1,k_2, the merge costs $O(k_1+k_2)$, excluding input normalization.
- CSR matrix-vector multiplication $y_i=\Sigma_j A_{ij} x_j$ costs $O(R+k)$, including output initialization.
- Sparse matrix multiplication is not generally $O(k_1+k_2)$. The output may become dense, and accumulating equal coordinates requires additional organization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete sparse addition and matrix-vector multiplication

```
add_COO(A,B):
    require same dimensions and canonical row-major entries
    i=0; j=0; out=[]
    while i<len(A) and j<len(B):
        if coord(A[i])<coord(B[j]): out.append(A[i]); i+=1
        else if coord(B[j])<coord(A[i]): out.append(B[j]); j+=1
        else:
            value=A[i].value+B[j].value
            if value!=0: out.append((A[i].row,A[i].col,value))
            i+=1; j+=1
    append remaining A[i:] and B[j:]; return out
CSR_matvec(R,C,row_ptr,col_idx,values,x):
    require len(x)==C
    y=[0]*R
    for r=0..R-1:
        for p=row_ptr[r]..row_ptr[r+1]-1:
            y[r]+=values[p]*x[col_idx[p]]
    return y
```

Complete sparse addition and matrix-vector multiplication (continued)

Let B contain(0,1,−5),(2,0,3). Adding it to A cancels(0,1), emits A's(1,0,−2),(1,3,7), then B's(2,0,3), then A's(2,1,4). Four entries remain, not kA+kB=6. For x=[1,2,3,4], row0 gives 5×2=10; row1 accumulates−2 then26; row2 gives 4×2=8.

CSR requires 0≤row_ptr[r]≤row_ptr[r+1]≤k, row_ptr[0]=0, row_ptr[R]=k, and valid columns in[0, C). Equal array lengths alone do not establish these invariants. Arithmetic costs assume fixed-range values and do not cover overflow or arbitrary-precision accumulation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String ADT and encoding levels

- A string is a finite sequence over alphabet Σ. Basic operations include length, concatenation, substring, and comparison.
- `std::string` stores char code units and can contain embedded `\0`. `size()` determines its logical length. C-string termination rules do not always apply.
- A UTF-8 code point can occupy multiple bytes. A user-perceived character can contain multiple code points.
- Matching here compares fixed-size symbols. If symbols are UTF-8 bytes, returned positions are byte positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Copying and viewing strings

- With contiguous code units and a stored length, indexed access can cost O(1). Copying a substring of length k takes Θ(k) time and output space.
- Concatenating lengths N and M into an independent result copies Θ(N+M) units. Repeatedly copying the whole accumulated prefix can give quadratic total work.
- A nonowning string view stores a start and length. Slicing can take constant time, but the view does not extend the source lifetime. A released or reallocated source invalidates the view.
- Lexicographic comparison searches for the first differing symbol. The comparison costs $O(\ell + 1)$, where ℓ is the common-prefix length and the additional term covers the end or mismatch check. Equal hashes, pointers, or lengths do not generally replace content equality.

Naive matching and the KMP prefix function

- Let text length be N and pattern length be M. Comparing at every candidate start uses at most `O(NM)` symbol comparisons; including initialization and empty inputs, time is `O(1+NM)`, for example repeated symbols with a mismatch at the last pattern position.
- Define `π[i]` as the length of the longest proper prefix of `P[0..i]` that is also a suffix.
- After q matched symbols, a mismatch sets `q=π[q-1]` to try a shorter known border. The text position never moves backward.
- Prefix construction and search each have a linear total number of fallback steps. The overall bounds are `O(N+M)` time and `O(M)` auxiliary space.

The fallback chain enumerates possible shorter borders. Any border surviving the mismatch must be a border of the already matched prefix, so skipping other lengths cannot discard a match. Each successful extension increases q by at most one. Each fallback strictly decreases q, so the total number of fallbacks is bounded by prior increases, separately during construction and search.

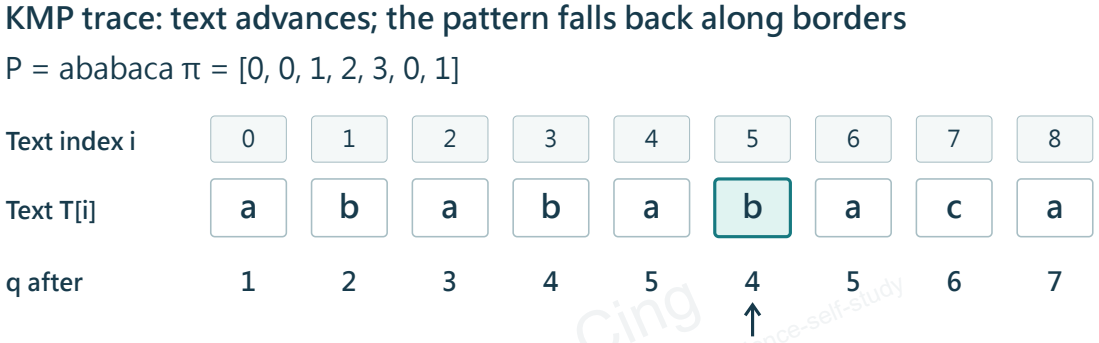
Worked KMP trace

- For `P=ababaca`, the prefix function is `[0,0,1,2,3,0,1]`.
- After matching `ababa`, the next text symbol is b while the pattern expects c. Fall back from `q=5` to `π[4]=3`.
- Now `P[3]=b` matches that same text symbol, giving `q=4` without rescanning earlier text.
- In `abababaca`, the complete match starts at zero-based position 2. Our interface defines an empty pattern to match at position 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP: reuse the same text character after fallback

In the figure, q is the length of the matched pattern prefix.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete KMP construction and search

```
prefix(P):
    pi=[0]*len(P)
    for i=1..len(P)-1:
        q=pi[i-1]
        while q>0 and P[q]!=P[i]: q=pi[q-1]
        if P[q]==P[i]: q+=1
        pi[i]=q
    return pi
first_match(T,P):
    if len(P)==0: return 0
    pi=prefix(P); q=0
    for i=0..len(T)-1:
        while q>0 and P[q]!=T[i]: q=pi[q-1]
        if P[q]==T[i]: q+=1
        if q==len(P): return i-len(P)+1
    return not_found
```

For P=ababaca, construction steps are i=1:b, q=0→0; i=2:a,0→1; i=3:b,1→2; i=4:a,2→3; i=5:c,3→1→0 with no extension; i=6:a,0→1. Thus pi=[0,0,1,2,3,0,1]. Reading T=abababaca gives q=1,2,3,4,5,4,5,6,7. At text index 5, b makes q fall from 5 to 3 before extending to 4. The match ends at i=8 and starts at 2.

Complete KMP construction and search (continued)

Construction accesses P[q] only for nonempty P and i≥1. Search handles an empty pattern first and immediately returns when q=M, avoiding P[M]. To enumerate overlapping matches, set q=pi[M−1] after each success instead of clearing it. Fixed-symbol comparison gives O(N+M) total work. Byte-oriented UTF-8 matching returns byte offsets, not grapheme indices.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft vector.overview and vector.capacity; Knuth, Morris, Pratt, Fast Pattern Matching in Strings (1977).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Start with removal order, then choose a representation

Stacks and queues both store a sequence. Their central difference is **which value may leave next**. We build contracts first, trace concrete storage next, then apply the same state-based reasoning to search and expressions.

Progression	Question to reconstruct
Array stack	Where is top? Why can an expensive growth still amortize?
Circular queue	How does FIFO survive physical wraparound?
Container simulation	How can two LIFO stacks implement FIFO?
Search	What must survive backtracking? Why does BFS minimize edges?
Expressions	When is an operator emitted? Which operand is left?

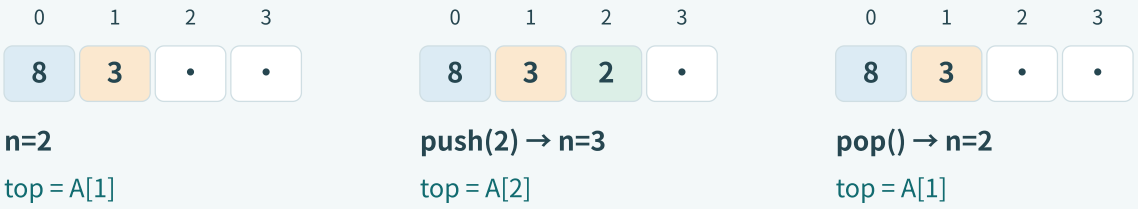
Prerequisites: arrays, half-open intervals and loop invariants. Failure policies and operation-cost assumptions are stated explicitly.

3.1 The stack contract: one end for both operations

- The abstract state is $[a_0,\ldots,a_{n-1}]$, with the top at the end. `push(x)` appends x.
- `top()` returns the last element without changing the state. `pop()` removes it and requires a nonempty stack.
- The interface must define empty-stack failure, for example an optional result, exception, or error code. It must not silently access invalid memory.
- With a dynamic array, top and pop take constant time and push takes amortized constant time under constant-time element operations. A fixed-capacity version also defines a full-stack result.

Push and pop as visible state changes

Capacity 4; zero-based indices; colored slots are live



Sequences are written **bottom to top**. `top()` only observes 3. After `pop()` removes 3, 8 becomes the new top. Never read `A[n-1]` when n is zero.

Use a count n to avoid a negative empty-stack index

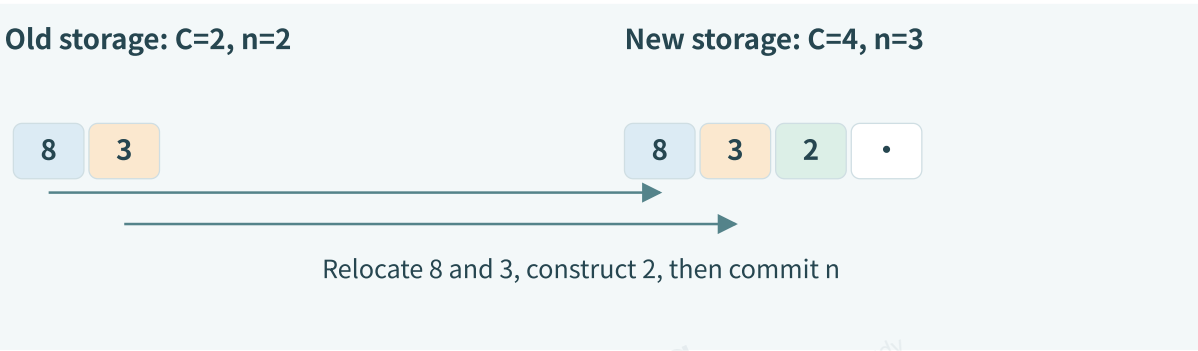
- Maintain `0≤n≤capacity`, with valid elements in `[0,n)`. The top is `A[n-1]`, accessible only when `n>0`.
- For push, ensure capacity, write `A[n]`, then increment n. Do not increment n before allocation succeeds.
- For pop, check `n>0` before removing the last live object. Generic C++ must handle destruction as well as the index.
- Representing empty as n=0 avoids encoding top=−1 in an unsigned index.

Allocated storage is not yet a set of live objects

Let storage be uninitialized capacity C and n the number of live objects. The abstract sequence is storage[0..n). Allocating C slots does not construct C objects of type T.

```
top(): require n>0; return const T& to storage[n-1]
push(T value):           // snapshot constructed before entering the body
    if n==C: grow_to(max(1,2*C))
    construct_at(storage+n, value)   // commit n only after construction succeeds
    n=n+1
pop():
    require n>0
    destroy_at(storage+n-1)
    n=n-1
```


How much relocation does doubling actually do?



The third push takes linear time. **Amortized constant does not mean worst-case constant per call.** This version does not shrink on pop.

push(8), push(3), push(2), pop() give [8], [8,3], [8,3,2], [8,3].

Doubling: add the cost across the whole sequence

Start with zero capacity and grow on full to 1, 2, 4, 8, ... For m≥1 pushes, final capacity C is the least power of two at least m, so m≤C<2m.

$$\text{relocations} = 1 + 2 + 4 + \cdots + C/2 = C - 1 < 2m.$$

There are also m constructions of newly appended values. Total work is Θ(m), hence Θ(1) amortized per push, **assuming bounded-cost element operations**. Long strings and large matrices require their own value-cost accounting.

For mixed pops, this version never shrinks, so add constant work per pop. Growing on full and immediately halving when half empty can thrash; a shrinking policy needs a separate threshold and proof.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

push(top()): save the value before relocating

With capacity 1 and contents [7], a `const T&` parameter can alias the old top. Releasing old storage during growth invalidates it. `noexcept` does not repair a dangling reference.

This chapter's by-value push(T value)	State and reason
Copy parameter before entering the body	value=7 is already independent
Grow successfully and relocate old value	Capacity 2; existing sequence still [7]
Construct new element from value	Increment n only after construction succeeds
Finish	[7,7]; references into old storage are invalid

This copying interface assumes copy-constructible T and charges snapshot construction to the call. Noncopyable T cannot use this copying form of `push(top())`.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Exception guarantees depend on the failure stage

Failure stage	What the pseudocode can promise
Copying the by-value parameter	Body not entered; n, capacity and existing values unchanged
Relocating during growth	Rollback needs a suitable copy or noexcept-move strategy and cleanup; arbitrary T does not guarantee it
New-element construction after successful growth	n unchanged and existing values valid; capacity and prior references may have changed

An unchanged size does not establish a strong guarantee for the entire push. The strong guarantee requires unchanged observable state on failure. This discussion assumes nonthrowing destruction; an actual implementation must handle allocation limits, capacity-doubling overflow and cleanup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 The queue contract: remove from the other end

- The abstract state is again $[a_0,\ldots,a_{n-1}]$. `enqueue(x)` appends, while `dequeue()` removes the first element.
- Shifting all remaining array elements after each front removal makes a dequeue cost $\Theta(n)$.
- A circular array preserves logical order without moving every remaining element.
- FIFO specifies return order. It guarantees neither thread safety nor equal execution time for jobs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular storage separates logical order from slots

- For capacity `C>0`, store head and size. Logical element k occupies `(head+k) mod C`.
- Empty means size=0 and full means size=C. No unused slot is needed to distinguish ambiguous equal head and tail indices.
- Enqueue writes at `(head+size) mod C`, then increments size. Dequeue reads head, sets `head=(head+1) mod C`, then decrements size.
- Each operation takes worst-case Θ(1) with fixed capacity and constant-time element movement. Resizing copies in logical order and can take Θ(n) for one operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Wrap the index without changing FIFO order

With capacity 4, enqueue A,B,C, dequeue twice, then enqueue D,E,F.

State	head	size	Logical sequence
Enqueue ABC	0	3	A,B,C
Dequeue twice	2	1	C
Enqueue D,E	2	3	C,D,E
Enqueue F	2	4	C,D,E,F

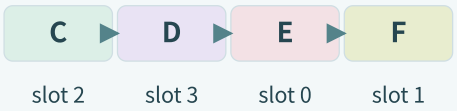
Physical slots 0,1,2,3 finally contain E,F,C,D.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C is at physical slot 2 but logical position 0

Physical storage: head=2, size=4, C=4

FIFO order: start reading at head



slot(k) = (head+k) mod C

Value C: physical slot 2 ↔ logical position 0; no value movement is needed

head=2, size=4; read slots 2→3→0→1 for C→D→E→F. A full queue rejects enqueue instead of overwriting its oldest value.

Fixed capacity: validate before changing state

Use head and size with C>0. This fixed-capacity version returns full instead of overwriting the oldest value. Value movement costs and exception behavior belong to the element contract.

```
slot(k):
    require 0<=k<C
    return head+k if k<C-head else k-(C-head)
try_enqueue(x):
    if size==C: return full
    construct_at(A+slot(size),x)
    size=size+1; return success
```

Dequeue: obtain the value, destroy, then advance

```
front(): require size>0; return reference(A[head])
dequeue():
    require size>0
    value=read_or_move(A[head])
    destroy_at(A+head)
    head=0 if head==C-1 else head+1
    size=size-1
    return value
```

Assume successful value operations and nonthrowing destruction. If returning a value can throw on move, specify that exception contract separately; rollback is not automatic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check enqueue and dequeue with the abstraction map

The branched slot formula avoids overflowing head+k. Enqueue first checks nonfull, so k=size<C. After the last removal, head may be any valid slot. The abstraction map `(head+k) mod C` proves correctness: enqueue appends exactly the new last value, and dequeue maps old logical k+1 to new logical k. Both have worst-case Θ(1) fixed-capacity work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Growth must relocate in logical order

Grow physical `[E,F,C,D]`, head=2,size=4,C=4 to C′=8. Copying the old slots unchanged and retaining head=2 is wrong: the new modulus sends the expected E position to unconstructed slot 4.

Logical k	Old slot (2+k) mod 4	Value	New slot
0	2	C	0
1	3	D	1
2	0	E	2
3	1	F	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

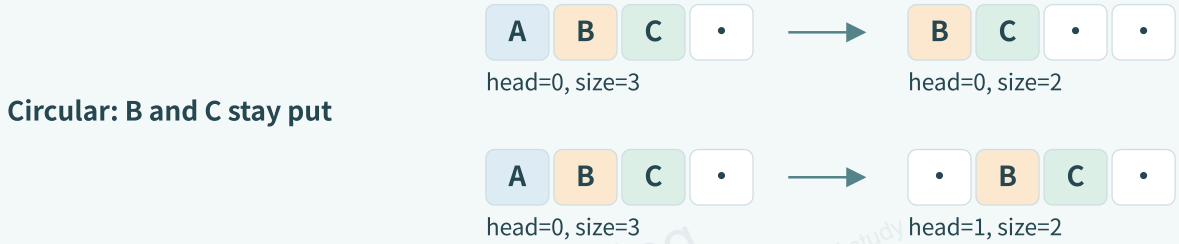
Commit the new storage after successful relocation

```
allocate new capacity C'
for k=0..size-1: construct new[k] from old[slot(k)]
commit new storage; head=0; C=C'; destroy/release old storage
```

The logical result is C,D,E,F. Append G at new slot 4. One growth costs Θ(size). Doubling only on full gives amortized Θ(1) enqueue from empty by the geometric movement sum. This does not satisfy fixed-capacity worst-case constant latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Distinguish empty, full and movement cost



C=4; empty and full may share head; distinguish size=0 from 4

With capacity 4, empty and full can share the same head. Size or another explicit convention distinguishes them. A shifting array moves every remaining value; the circular version advances head and destroys the old front object. Both preserve FIFO values but have different costs.

Three circular-queue conventions

Let the array length be $C \geq 2$. This chapter's `head,size` representation uses all C slots: empty is `size=0` and full is `size=C`. Another convention puts front just before the first element and reserves a slot: `front=rear` is empty and `(rear+1) mod C=front` is full, giving capacity $C - 1$.

Keeping front before the first element, a tag may distinguish the two cases when `front=rear`: tag=false means empty, tag=true means full, so all C slots can be used. The tag decides empty versus full only when the indices coincide; when nonempty `(front+1) mod C` identifies the first element, corresponding to `head` in the head/size representation. Each representation needs its own complete update and failure rules; their tests are not interchangeable.

Tag representation: when does the state change?

Start with `front=rear=0, tag=false`. Assume element construction, reading and destruction succeed:

```
enqueue(x):
    if front==rear and tag: return full
    rear=(rear+1) mod C; store x at rear
    if rear==front: tag=true
dequeue():
    if front==rear and not tag: return empty
    front=(front+1) mod C; take and remove A[front]
    if front==rear: tag=false
    return value
```

When `front!=rear`, tag may retain its earlier value and is not used to decide empty or full. These are fixed-capacity abstract steps; generic operations that may throw require a separate commit order and cannot automatically promise unchanged state on failure.

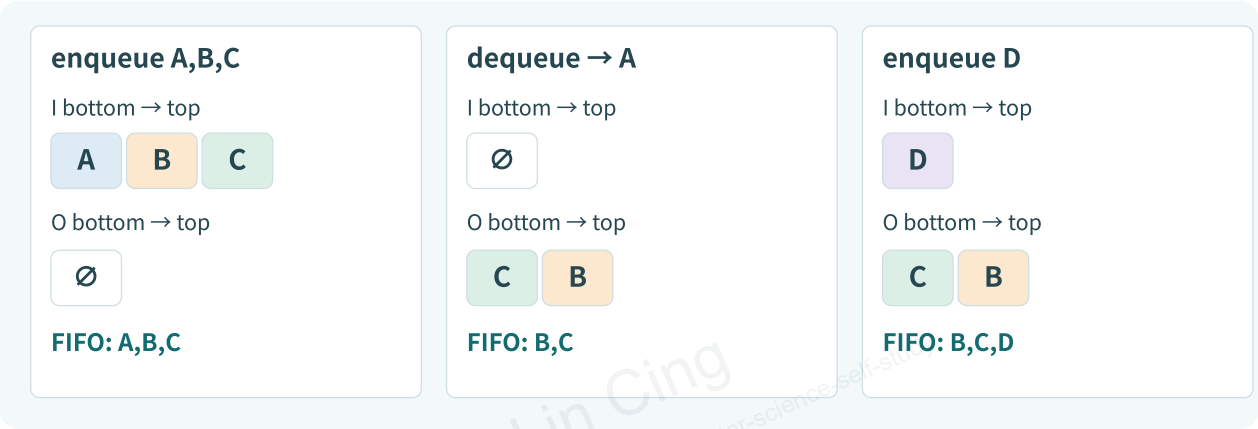
3.3 Two stacks can implement one FIFO queue

Let I hold incoming values and O hold values ready for removal. Both stack sequences are written bottom to top.

```
enqueue(x): I.push(x)
dequeue():
    if O.empty():
        while not I.empty(): O.push(I.pop())
    require not O.empty()
    return O.pop()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Old values in O stay ahead of new values in I



The FIFO sequence is `reverse(O) + I`. **Transfer only when O is empty**; new values in I must not overtake older values still in O.

A large transfer can still have constant amortized cost

Charge one unit per stack push/pop and use worst-case constant node-stack operations. This abstract model excludes allocator latency and exceptions. Let $\Phi = 2|I|$, initially zero.

Operation	Actual cost c	ΔΦ	Amortized c+ΔΦ
enqueue	1	+2	3
dequeue with nonempty O	1	0	1
Transfer k≥1 values, then dequeue	2k+1	−2k	1

Since $\sum c = \sum \hat{c} + \Phi_0 - \Phi_{final} \leq \sum \hat{c}$, m successful operations cost O(m) in total. An empty-queue failure check costs constant work separately.

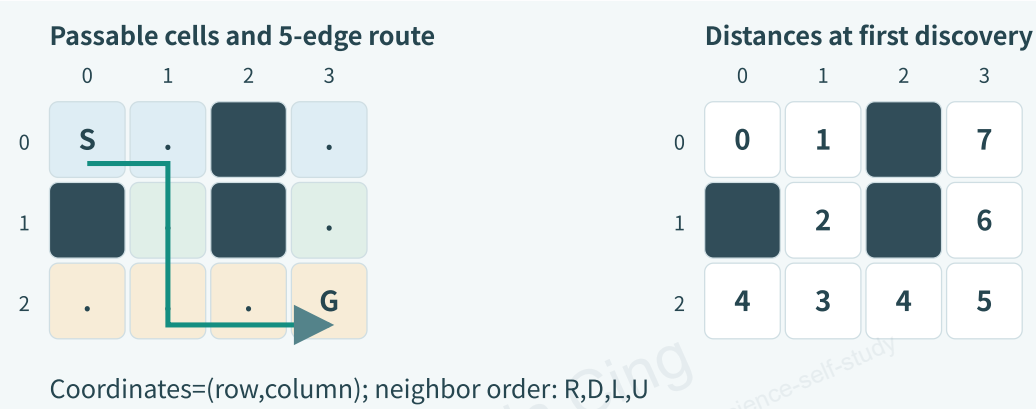
One dequeue can still take Θ(n). If the underlying stacks resize, include their amortized costs as well; neither layer's work is free.

3.4 A maze is a graph: fix the legal moves

- Passable cells are vertices, with edges between adjacent passable cells. This example permits four directions and excludes diagonals.
- Search state includes the current cell, visited information, and predecessors for path reconstruction.
- DFS can use a stack. It finds a path but does not generally minimize the number of steps.
- If all moves cost 1, BFS uses a queue. The first discovery level of the exit gives the shortest number of steps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One maze, both the route and the distances



Coordinates are zero-based `(row,column)` from the upper left. Moves are right, down, left, up, each costing 1. Dark cells are walls; the polyline is a five-edge S–G route.

This maze has only one simple S–G path, so DFS and BFS happen to find equal-length routes. **That does not establish shortest paths for DFS in general.** A later five-vertex counterexample separates the two.

Lin Cing

DFS frames retain the point at which to resume

A frame `(u,next_direction)` stores where recursive neighbor scanning will resume. Mark visited on first push.

```
visited[start]=true; parent[start]=none; push((start,0))
while stack is not empty:
    (u,i)=top()
    if u==goal: return parent path
    if i==4: pop(); continue
    top().next_direction=i+1
    v=neighbor(u,i)                  // right, down, left, up
    if v is in bounds, passable, and not visited[v]:
        visited[v]=true; parent[v]=u; push((v,0))
return unreachable
```

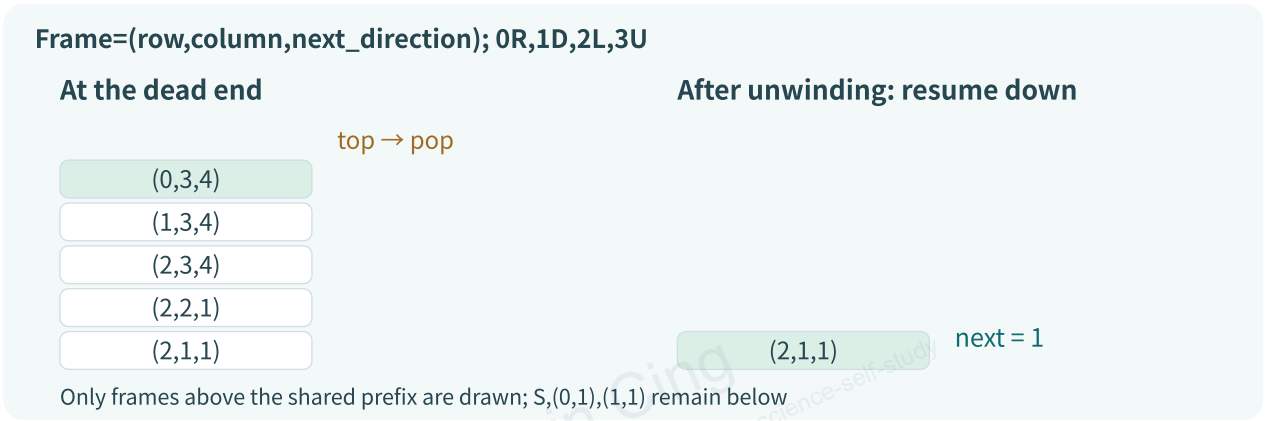
https://github.com/xinhi

Why DFS terminates and reconstructs a path

At (2,1), right neighbor (2,2) is pushed first, then (2,3)=G is found. The path contains five edges. Removing the goal test while retaining passability gives complete discovery order `(0,0),(0,1),(1,1),(2,1),(2,2),(2,3),(1,3),(0,3),(2,0)`. Each frame tries at most four directions. Finite cells and one first push per cell prove termination. Parent edges strictly reverse discovery time, so they are acyclic and reconstruct a legal path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Backtracking discards one finished frame, not all progress



To demonstrate real popping, this page continues past G and reaches the upper-right dead end (0,3). After all four directions, pop `(0,3,4)`. The next top `(1,3,4)` is also exhausted.

Unwinding eventually restores `(2,1,1)`: right has already been tried, so resume with down, then discover left neighbor (2,0). Keep visited marks on pop. This algorithm tests reachability; enumerating all simple paths would require a different marking policy.

BFS assigns distance and parent on discovery

Each cell is enqueued at most once. Mark before enqueueing so that two predecessors cannot enqueue the same undiscovered cell twice.

```
dist[start]=0; parent[start]=none; enqueue(start)
while queue is not empty:
    u=dequeue()
    for v in legal_four_neighbors(u):
        if v is undiscovered:
            dist[v]=dist[u]+1; parent[v]=u; enqueue(v)
```

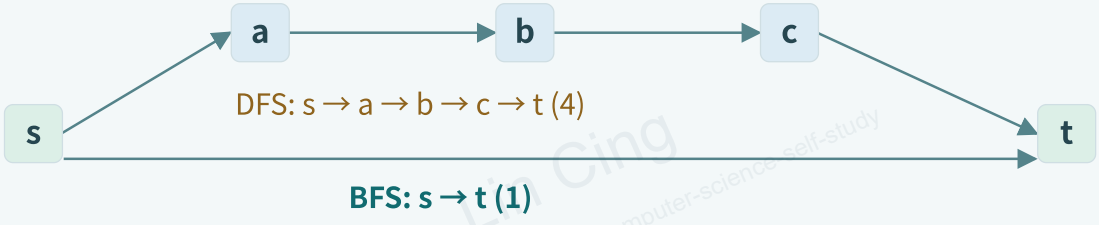
Use undefined dist as the undiscovered marker. Fix parent at first discovery. If only G is needed, reconstruct its path at first discovery instead of traversing the whole maze.

BFS dequeue trace: finish a layer before the next

Dequeued cell	New cells and distances	Queue afterward
(0,0)	(0,1):1	[(0,1)]
(0,1)	(1,1):2	[(1,1)]
(1,1)	(2,1):3	[(2,1)]
(2,1)	(2,2):4, (2,0):4	[(2,2),(2,0)]
(2,2)	(2,3):5	[(2,0),(2,3)]
(2,0)	None	[(2,3)]
(2,3)	(1,3):6	[(1,3)]
(1,3)	(0,3):7	[(0,3)]
(0,3)	None	[]

Reconstruct from parent, then prove optimality

Follow G's parents through (2,2), (2,1), (1,1), (0,1), (0,0) and reverse for five edges. When a distance-d vertex is removed, the queue contains only distances d or d+1. A newly discovered neighbor has distance d+1. Any shorter route would have an earlier-layer predecessor already processed, contradicting first discovery.



Fix s neighbor order as a,t; arrows are directed unit-cost edges

With neighbors a,t at s, DFS first takes four edges. BFS discovers t at distance 1 while processing s.

Prove reachability, optimality and cost separately

- Every explored cell is reachable from the start, and each predecessor edge leads to an earlier discovery.
- If DFS tries every legal neighbor and has no pending frames, it has covered every reachable cell. An undiscovered exit is unreachable.
- With V passable cells and E adjacency edges, adjacency-list DFS and BFS take Θ(V+E) for a complete traversal. Marks and predecessors use Θ(V) space.
- A four-neighbor grid has E=O(V), giving linear work. Copying a complete path at every step can violate that implementation bound.

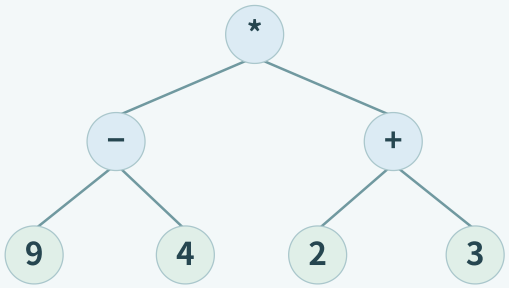
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 The tree fixes operands; notation fixes scan order

- Infix `a+b*c`, prefix `+ a * b c`, and postfix `a b c * +` describe the same expression tree.
- Initially restrict the grammar to binary operators, integer tokens, parentheses, and explicit precedence. A negative numeric token and a unary minus operator are different syntax choices.
- Infix requires precedence and associativity. Default left-associative `a-b-c` means `(a-b)-c`.
- Whitespace cannot reliably tokenize every programming language. A complete compiler performs lexical analysis before parsing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One expression tree, three reading orders



Prefix: root → left → right

* − 9 4 + 2 3

Postfix: left → right → root

9 4 − 2 3 + *

(9−4) × (2+3) = 5 × 5 = 25

Postfix evaluation: pop the right value before the left value

The root * has left subtree `9-4` and right subtree `2+3`. Prefix visits root,left,right; postfix visits left,right,root. Fully preserving this tree in infix requires `(9-4)*(2+3)`.

Postfix evaluation pops the right operand first

1. Scan tokens left to right. Push numeric values.
2. For a binary operator, pop the right operand b first, then the left operand a. Push the result of `a op b`.
3. Reject if fewer than two values are available. At the end exactly one value must remain.

For `8 3 2 * -`, the stacks are `[8]`, `[8,3]`, `[8,3,2]`, `[8,6]`, `[2]`. Division additionally requires zero-divisor and arithmetic-range checks.

For prefix, scan backward and preserve operand roles

- Scan prefix right to left. At an operator, the first popped value is the left operand and the second is the right operand.
- For `- 8 * 3 2`, first compute $3 \times 2 = 6$, then $8 - 6 = 2$.
- Each token is pushed and popped at most a constant number of times. Infix-to-postfix conversion and prefix or postfix evaluation therefore take O(t), for t tokens.
- Auxiliary stack space is O(t) in the worst case. Fixed-range arithmetic is treated as constant time. Arbitrary-precision arithmetic needs a separate cost model.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Make the pop condition explicit

The input grammar contains nonnegative integer tokens, binary `+ - * / ^`, and parentheses. Here ^ means **right-associative exponentiation**, not C++ XOR. Other operators associate left. Unary minus and negative numeric tokens are outside this input grammar.

Category	Precedence	Associativity
+, −	1	left
*, /	2	left
^	3	right

```
must_pop(top, incoming):
    return top != '(' and
       (prec(top) > prec(incoming) or
        (prec(top) == prec(incoming) and incoming is left-associative))
```

Test **incoming's associativity**. Parentheses use a separate matching rule rather than ordinary binary precedence.

Infix to postfix: process one token at a time

output holds determined postfix fragments; ops holds pending operators. need_operand rejects adjacent values, empty parentheses and missing right operands.

```
output = []; ops = []; need_operand = true
for token in tokens:
    if token is a number:
        require need_operand; output.append(token); need_operand=false
    else if token == '(':
        require need_operand; ops.push('(')
    else if token == ')': close_group()
    else if token is binary operator op:
        require not need_operand
        while ops is not empty and must_pop(ops.top,op):
            output.append(ops.pop())
        ops.push(op); need_operand=true
    else: reject
```


A right parenthesis drains only its local group

```
close_group():
    require not need_operand
    while ops is not empty and ops.top != '(': output.append(ops.pop())
    require ops is not empty; ops.pop()   // discard matching left parenthesis
    need_operand=false
```

The helper shares output, ops and need_operand. It sets need_operand=false on success. Empty groups, missing operands and unmatched right parentheses all reject. Parentheses are never emitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

End of input is part of the algorithm

Continue with the same output, ops and need_operand from the preceding page:

```
require not need_operand                    // EOF: reject empty or incomplete
input
while ops is not empty:
    require ops.top != '('                  // reject unmatched left parenthesis
    output.append(ops.pop())
return output
```

Every failed requirement rejects the complete input. A partial output is not a successful conversion. Repeated popping is essential: one incoming operator may require several earlier operators to be emitted.

Even 9−4 needs an EOF drain

`9-4`

Token	Operator stack, bottom to top	Output
`9`	∅	`9`
`-`	`-`	`9`
`4`	`-`	`9 4`
`EOF`	∅	`9 4 -`

Before EOF the output is only `9 4`. Draining gives `9 4 -`, with value 5. Prefix is `- 9 4`.

Parentheses finish their local subexpression

`(9-4)*(2+3)`

Token	Operator stack, bottom to top	Output
(	(	∅
9	(	9
-	(-	9
4	(-	9 4
)	∅	9 4 -

The right parenthesis finishes 9−4, emitting 9 4 −. ops is empty; multiplication and the second group follow.

Finish the second group, then drain multiplication

`(9-4)*(2+3)`

Token	Operator stack, bottom to top	Output
*	*	9 4 -
(	* (	9 4 -
2	* (	9 4 - 2
+	* (+	9 4 - 2
3	* (+	9 4 - 2 3
)	*	9 4 - 2 3 +
EOF	∅	9 4 - 2 3 + *

Postfix stack states are `[9]→[9,4]→[5]→[5,2]→[5,2,3]→[5,5]→[25]`. Prefix `* - 9 4 + 2 3` also evaluates to 25.

Trace repeated popping and right association

`9-4*2/1+3`

Token	Operator stack, bottom to top	Output
9	∅	9
-	-	9
4	-	9 4
*	- *	9 4
2	- *	9 4 2
/	- /	9 4 2 *
1	- /	9 4 2 * 1
+	+	9 4 2 * 1 / -
3	+	9 4 2 * 1 / - 3
EOF	∅	9 4 2 * 1 / - 3 +

Equal precedence: inspect incoming associativity

At +, pop / and then the equal-precedence left-associative -. Popping once is insufficient. The value is 4. For right-associative `2^3^2`, do not pop on the second ^. Postfix is `2 3 2 ^ ^`, giving 512 rather than 64.

Right association gives 2^(3^2)=2^9=512. Left association gives (2^3)^2=8^2=64. These are different expression trees, not interchangeable evaluation orders.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reject invalid syntax and unsafe arithmetic

Input	Rejection point	Reason
`(9-4`	EOF	A left parenthesis remains after emitting -
`9-4)`	At)	No matching left parenthesis
`9-`	EOF	need_operand remains true
`9 4`	Second number	Missing binary operator
`()`	At)	Missing operand inside parentheses

Do not infer unary versus binary syntax from stack size. Evaluation separately rejects division by zero and fixed-integer overflow.

Why the invariant and linear bound hold

An unmatched left parenthesis separates stack segments. Within each segment, precedence is nondecreasing from bottom to top. Equal-precedence operators can remain together when waiting for right association. Output contains the postfix order of completed subexpressions; stacked operators await their right operand or a higher-precedence subexpression.

Repeated popping emits exactly the operators that must finish before the new operator. A right parenthesis finishes its entire local segment while preserving outside operators. At EOF no further operand can arrive, so every pending operator must be emitted. A remaining left parenthesis has no valid mate and must cause rejection, not appear as an output operator.

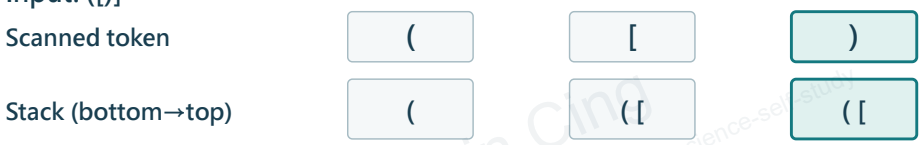
Each operator and parenthesis is pushed and popped at most once. A single token can trigger multiple pops, but the complete t-token conversion uses O(t) time and O(t) space. This is an aggregate proof, not constant worst-case work per token. Unary operators, functions, negative numeric tokens, and special floating-point values require an expanded grammar and proof.

Bracket matching: retain the kinds still open

The input alphabet is `()[]{}`. Push each opening bracket. On a closing bracket, reject if the stack is empty or its top is not the matching opener; otherwise pop. Accept only if the stack is empty at the end. If other characters are allowed, explicitly specify whether to ignore or reject them.

Bracket matching: the closer must match the stack top

Input: ([)]



On), the top is [, which expects].

Reject immediately; equal bracket counts do not guarantee nesting.

For ([]), the stack is empty after the scan, so accept.

The heavy border marks the mismatch independently of color.

Equal counts do not ensure correct nesting

`([)]` has equal counts of each opening and closing kind, but at the third character `)` the stack, bottom to top, is `(`, `[`. The top expects `]`, so reject. For `([])`, the successive states are `[ ( ]`, `[ ( , [ ]`, `[ ( ]`, `[]`, and the input is accepted.

Invariant: after every accepted prefix, the stack contains exactly its unmatched opening brackets in order, with the next one to close on top. Each symbol causes at most one push or pop: $O(n)$ time and $O(n)$ auxiliary space. The empty string is accepted under this contract.

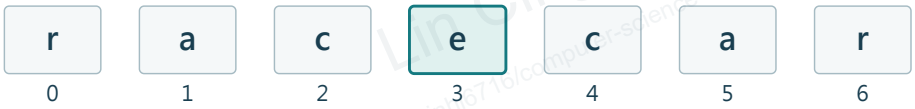
Palindrome with known length: reverse the first half

The input is a sequence S with known length n. Push its first $\lfloor n/2 \rfloor$ symbols in order. If n is odd, skip the middle symbol. Scan the second half left to right, comparing and popping the top each time. Reject on the first mismatch and accept only if all pairs match.

For `racecar`, push `r,a,c`, skip `e`, then compare `c,a,r` against successive tops. The empty and one-symbol sequences are accepted. Specify the symbol unit: bytes, Unicode code points, and grapheme clusters may give different results.

Palindrome: push the first half and compare the second

racecar: length 7; skip the middle e



Push r,a,c: the stack bottom→top is [r,a,c].

Skip e; compare c, a, r in order. All three match.

Pop after each comparison; accept after all symbols match.

Empty and one-symbol inputs are accepted; length is known.

Palindrome correctness and cost

After the first half is pushed, the top is immediately left of the middle. Pop number k compares the two positions symmetric about the middle. Every pair matches exactly when the sequence equals its reversal. Time is $O(n)$ and the stack holds at most $\lfloor n/2 \rfloor$ symbols.

This version knows the length and can scan the corresponding positions. A one-way stream of unknown length cannot identify its midpoint before reaching it without a different input assumption or algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recognizing $a^n b^n$: count and order both matter

Define $L = \{a^n b^n \mid n \geq 1\}$ over the alphabet `a`,`b`. Read at least one `a`, pushing one marker per `a`. On the first `b`, enter the second phase and pop a marker per `b`. Reject an `a` in phase two, any other symbol, or a `b` read with an empty stack. At the end, accept only if both `a` and `b` have appeared and the stack is empty.

`aabb` is accepted; `aab` leaves one marker; the second `b` in `abb` has no marker; and `abab` has an `a` in phase two. The empty string is excluded here because $n \geq 1$; defining $n \geq 0$ would include it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6 Stack permutations do not allow every output order

Input arrives in fixed order 1,2,3. Only push the next input value or pop the current top to output. No bypass is allowed.

Target	Forced operations	Result
2,1,3	push 1,push 2,pop 2,pop 1,push 3,pop 3	Feasible; maximum depth 2
3,1,2	To output 3 first: push 1,push 2,push 3,pop 3	Impossible: top 2 blocks the required 1

For each target x, push unread inputs until top=x. If input is exhausted and top still differs, reject; otherwise pop x. **The choices are forced**: when top is not x, popping would output a wrong value, so pushing the next input is the only possible progress. Each distinct value enters and leaves at most once: O(n) time and O(n) space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why are there five outputs for n=3, rather than six?

Write U for push and D for pop. A legal sequence has n of each, with at least as many U as D in every prefix. This is exactly a balanced-parenthesis sequence, with U as an opening parenthesis.

Match the first push with the pop that removes it, obtaining `U A D B`. If A has k pushes, B has n−1−k. Both are legal, and the decomposition is unique:

$$C_0 = 1, \qquad C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}.$$

Thus C_1=1, C_2=2, C_3=2+1+2=5 and C_4=5+2+2+5=14. For a fixed distinct input, each output forces its operation sequence, as proved earlier. We therefore count distinct feasible outputs as well as operations.

Catalan's formula subtracts the illegal paths

Ignoring the prefix restriction gives $\binom{2n}{n}$ U/D sequences. For an illegal path, swap U and D through its first visit to height−1. The original prefix had one more D, so the transformed complete path has n+1 U and n−1 D.

Invert this transformation by swapping the prefix through the first visit to height+1. The inverse recovers exactly one illegal path; hence the correspondence is bijective. There are $\binom{2n}{n-1}$ illegal paths, giving

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}.$$

For n=0 there is one empty operation sequence. Distinct values, fixed input order and no bypass are essential to this output count. Repeated values need a separate count of distinct outputs.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft stack and queue; MIT 6.006, Lecture Notes 9–10.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are pointers, object lifetimes, Stack and Queue ADTs, and Chapter 1 invariants.
- Distinguish a node's position, value, and ownership. Identify operations that actually need a search.
- Trace singly linked, circular, and doubly linked lists, linked stacks and queues, and polynomial addition.

Valid singly linked states

- Each node contains a value and next. Head points to the first node. Null represents an empty list.
- Following next from head forms a finite acyclic chain ending in null. Exactly one container is responsible for releasing each node.
- If tail is stored, it points to the last node in a nonempty list and `tail.next=null`. Both head and tail are null for an empty list.
- External pointers may be nonowning references. Deleting the referenced node invalidates them. An address does not imply a live object at that address.

Insertion with a known predecessor

To insert new node x after p:

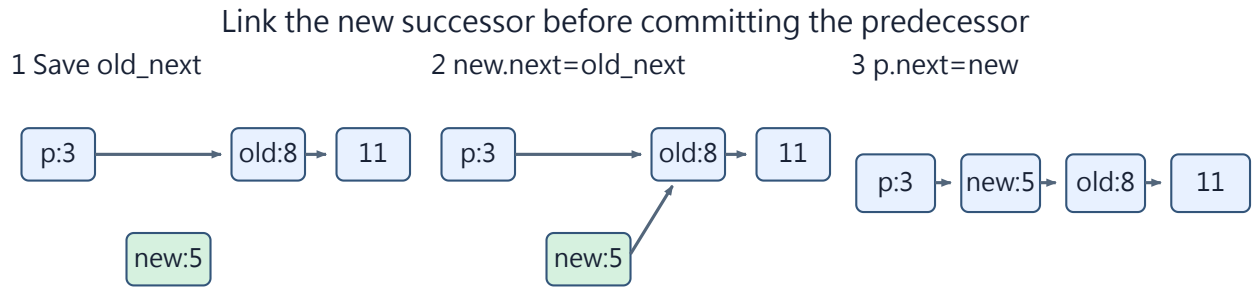
1. Allocate and construct x, setting `x.next=p.next`.
2. Set `p.next=x`. If p was the tail, set `tail=x`.
3. Update the element count. If construction fails, preserve the original list.

With a valid known predecessor and constant-time allocation, the update costs O(1). Finding the ith predecessor first costs O(i).

Complete singly linked insertion and erasure

```
insert_after(p,value):
    require p is a live node of this container
    old_next=p.next
    x=allocate_and_construct(value)
    x.next=old_next
    p.next=x
    if tail==p: tail=x
    size+=1
erase_after(p):
    require p is live and p.next!=null
    victim=p.next; after=victim.next
    p.next=after
    if tail==victim: tail=p
    destroy_and_release(victim); size-=1
```


Complete singly linked insertion and erasure (continued)



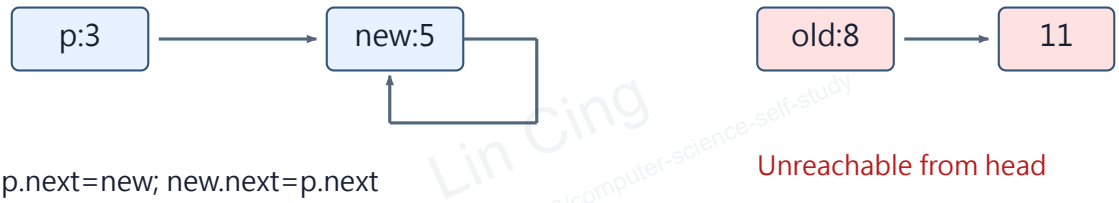
Insert 5 after p=3 with successor chain 8→11. After allocation x is not reachable from head. Setting x.next=8 preserves the old 3→8→11 path and links x to 8. Committing p.next=x produces 3→5→8→11. Reversing the statements gives x.next=x, losing head reachability of 8 and 11. This is a structural reachability failure.

Head updates need a separate case or a nondatum dummy predecessor. Expose an empty head/tail state consistently rather than treating the dummy as user data. With a known valid predecessor the link work is O(1); finding that predecessor by position remains O(n). Arbitrary external pointers do not satisfy the ownership precondition.

Reachability counterexample for reversed insertion assignments

```
old_next=p.next
p.next=x              // wrong first update
x.next=p.next         // reads x, not old_next
```

Wrong order creates a self-loop and loses the old suffix



p.next=new; new.next=p.next

Unreachable from head

Starting at head=p in3→8→11, the wrong updates produce3→5→5→…. Nodes8,11 remain allocated but lose the owning path from the container. An external pointer retaining their address does not restore the representation. Correct insertion links x to saved old_next before committing p.next=x. The two writes are not automatically atomic for concurrent observers; this chapter assumes single-threaded structural updates.

Save the successor before deletion

- To delete after p, save `victim=p.next` and `after=victim.next`, set `p.next=after`, then release victim.
- If victim was the tail, set tail=p. Removing the only node requires both head and tail to become null.
- Dereference victim before release. Reading `victim.next` afterward is use-after-free.
- A pointer only to the node being removed does not generally permit O(1) deletion of a singly linked tail. Copying a successor's value also changes node-identity semantics.

Head operations, queue endpoints, and iterative clearing

```
push_front(v):
    x=Node(v,next=head); head=x
    if tail==null: tail=x
    size+=1
pop_front():
    require head!=null
    x=head; result=copy(x.value); head=x.next
    if head==null: tail=null
    release(x); size-=1; return result
enqueue(v):
    x=Node(v,next=null)
    if tail==null: head=x
    else: tail.next=x
    tail=x; size+=1
clear():
    while head!=null:
        x=head; head=x.next; release(x)
    tail=null; size=0
```


Head operations, queue endpoints, and iterative clearing (continued)

These operations share one owning singly linked representation. A linked stack uses push_front/pop_front; a linked queue uses enqueue/pop_front. Assume single-threaded updates and successful allocation/value copying. Copy a possibly throwing result before changing links, so failure preserves the original list. Save the next pointer before releasing a node.

Starting at (head,tail,size)=(null,null,0), enqueue 4 makes both endpoints point to 4. Enqueue 9 gives 4→9 with size 2. Removing 4 leaves head=tail=9 and size 1; removing 9 clears both endpoints and size. Enqueue 7 then creates a valid new chain. Clearing head while retaining the released tail would make that last insertion write through a dangling pointer.

Inductively, insertion adds a fresh node through exactly one owning link, preserves acyclicity, and leaves tail.next=null. Removing the head preserves all remaining nodes and their order. Each update changes size with the reachable-node count. Endpoint operations take worst-case O(1) under fixed-size value and constant allocation costs. Clearing releases each reachable node once in Θ(n) time and O(1) extra stack space, avoiding a linear recursive-destructor call chain.

Three-pointer reversal invariant

Initialize `prev=null,curr=head`. Repeatedly save `next=curr.next`, set `curr.next=prev`, then advance prev and curr.

- Prev points to the reversed prefix and curr to the unprocessed suffix. Their node sets are disjoint and their union is the original list.
- After one iteration on `3→8→11`, the reversed part is `3→null` and the unprocessed part is `8→11`.
- At `curr=null`, set head=prev. If tail is maintained, the old head becomes the new tail.
- Every node is handled once. Time is Θ(n) and auxiliary pointer space is Θ(1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete reversal and the disjoint-segment invariant

```
reverse(head):
    old_head=head; prev=null; curr=head
    while curr!=null:
        after=curr.next; curr.next=prev
        prev=curr; curr=after
    head=prev; tail=old_head
```

Reversing 3→8→11 gives (prev,curr) states (empty,3→8→11),(3,8→11),(8→3,11), (11→8→3,empty). Disjoint segments covering exactly the original node set prove preservation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reversal, one pointer update at a time

Reverse a singly linked list: disjoint parts retain all nodes

Original: 3 → 8 → 11 → null

Step	prev: reversed prefix	curr: unprocessed suffix
0	null	3 → 8 → 11 → null
1	3 → null	8 → 11 → null
2	8 → 3 → null	11 → null
3	11 → 8 → 3 → null	null

Save curr.next before reversing; at the end head=prev and tail=old head.

Empty and single-node chains also work; numbers identify nodes.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

C++ ownership and copying

- `unique_ptr` represents single ownership and transfers the release responsibility with ownership. A nonowning raw tail pointer still needs a validity invariant.
- Default shallow copying of owning raw pointers can make two containers release the same node. Define deep copying or explicitly prohibit copying.
- `shared_ptr` is not an automatic answer for every linked structure. Owning links in both directions or around a cycle create reference-count cycles.
- Recursive destruction of a long chain may create a deep call stack. A teaching implementation can clear iteratively and explain the lifetime behavior.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Linked stacks and queues

Structure	Pointer arrangement	Update cost
Stack	Head is the top	O(1) insertion and deletion at head
Queue	Remove at head, insert at tail	O(1) each when tail is stored

- Enqueue into an empty linked queue makes both head and tail point to the new node.
- Dequeue of the last node must also set tail to null. Otherwise the next enqueue can write through a dangling pointer.
- These bounds exclude copying arbitrarily large values. An ADT does not guarantee actual allocator latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Canonical polynomial representation

- A node stores `(exponent,coefficient)`. Exponents are nonnegative integers in strictly decreasing order, and coefficients are nonzero.
- $P=4x^5-2x^2+7$ becomes `(5,4)→(2,-2)→(0,7)`.
- Duplicate exponents indicate unnormalized input. Combine their coefficients and remove a node when its coefficient cancels to zero.
- The empty list represents the zero polynomial. Do not mix it with a separate `(0,0)` representation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial addition as sorted merging

- Let $Q=-4x^5+3x^3+2x^2$. Start two cursors at the largest exponents and compare current exponents.
- Output the term with the larger exponent and advance that cursor. For equal exponents, add coefficients, output only a nonzero sum, and advance both.
- The degree-five and degree-two terms cancel, giving $P+Q=3x^3+7$.
- The output remains canonical and equals the sum of consumed input terms. Each node is advanced past once, giving $\Theta(p+q)$ time and $O(p+q)$ output space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial merging with consumed-term states

```
add_polynomials(p,q):
    require both normalized, descending exponents
    out=[]
    while p!=end and q!=end:
        if p.exp>q.exp: append_copy(out,p); advance(p)
        else if q.exp>p.exp: append_copy(out,q); advance(q)
        else:
            c=p.coeff+q.coeff
            if c!=0: append(out,(p.exp,c))
            advance(p); advance(q)
    append_copy(out,remaining(p)); append_copy(out,remaining(q))
    return out
```

For $P=4x^5-2x^2+7$ and $Q=-4x^5+3x^3+2x^2$, equal exponents 5 cancel to zero. Comparing 2 and 3 emits $3x^3$. Equal exponents 2 cancel. Q ends, so append P's remaining 7. The result is $3x^3+7$. At least one cursor advances every iteration, and output remains descending with no zero coefficients.

Fixed integers require an overflow policy. A floating near-zero tolerance is not exact algebraic equality and changes the abstract value. Choose exact integers, rationals, or an explicitly approximate model before claiming polynomial identities.

Circular-list termination

- A nonempty circular singly linked list satisfies `tail.next=head`. It does not end at null. Stop traversal on returning to the starting node.
- With tail stored, head is `tail.next`. Inserting at either end changes a constant number of links.
- Starting at B in `A→B→C→A`, one traversal gives B, C, A. Waiting for null would never terminate.
- With a sentinel, an empty list may have the sentinel point to itself. A sentinel is a structural node, not user data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular tail insertion and endpoint costs

```
circular_push_back(value):
    x=new_node(value)
    if tail==null: x.next=x; tail=x
    else: x.next=tail.next; tail.next=x; tail=x
```

Circular insertion into empty gives A.next=A. Adding B gives tail=B,B.next=A,A.next=B. Traverse with a do/while stopping on return to head, checking tail=null first. With only tail stored, deleting the head is O(1), but deleting the tail needs predecessor search O(n).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular queue: clear tail when its only node leaves

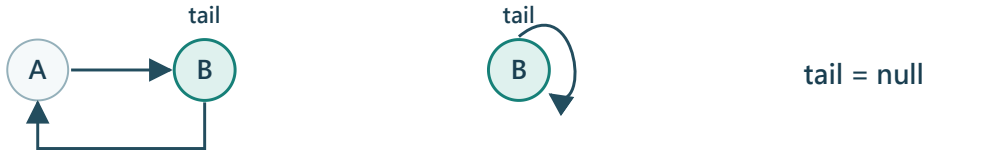
Keep only `tail`; when nonempty, the first node is `tail.next`. If `tail.next==tail`, the ring has one node. Set `tail` to null before releasing that node. Otherwise link `tail.next` to the old first node's successor, then release the old first node.

```
dequeue():
    if tail == null: return empty
    victim = tail.next
    value = copy(victim.value)
    if victim == tail: tail = null
    else: tail.next = victim.next
    destroy_and_release(victim)
    return value
```

For `A→B→C→A` with tail=C, dequeue yields `B→C→B` with tail still C. For `A→A`, the result must have tail=null. The example assumes successful copying, destruction, and return; potentially throwing generic values need a separate exception contract.

Circular queue: the final dequeue

Circular queue: removing the last node



① Start: A→B→A ② Remove A: B→B ③ Remove B: empty

victim=tail.next; for one node, set tail=null before releasing victim.

A later enqueue creates C→C; released B must not be dereferenced.

Highlighted node is tail; the empty queue has no valid tail.

Floyd's pointers: find a cycle or reach the end

Start both pointers at head. Before each iteration require that fast and fast.next are non-null; advance slow once and fast twice. A meeting implies a cycle. If the check fails, the finite chain reached its end. Nodes must remain live and the structure must not be modified concurrently.

```
slow=head; fast=head
while fast != null and fast.next != null:
    slow=slow.next; fast=fast.next.next
    if slow==fast: return meeting
return no_cycle
```

An empty chain and a single node ending in null have no cycle. A single self-linked node meets on the first iteration. The algorithm never changes links.

Why does the second meeting find the entry?

Let μ be the number of edges from head to the cycle entry and $\lambda \geq 1$ the cycle length. The first meeting is inside the cycle. If slow has taken t steps and fast $2t$, equal positions imply $t \equiv 0 \pmod{\lambda}$.

Reset one pointer to head, leave the other at the meeting, and advance each one step per iteration. After μ steps the first reaches the entry, and the second is at $(t - \mu + \mu) \bmod \lambda = 0$ relative to that entry. Earlier, the head pointer is outside the cycle while the other is inside, so they cannot meet. Their second meeting is exactly the entry.

The cyclic case takes $O(\mu + \lambda)$ time and $O(1)$ extra pointer space. A finite acyclic chain of length n takes $O(n)$. The argument compares node identity and does not assume distinct stored values.

Trace: a two-edge prefix and a three-node cycle

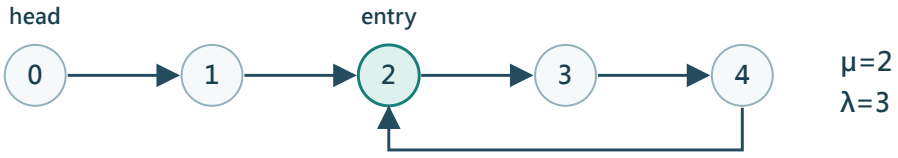
The links are `0→1→2→3→4→2`, so $\mu = 2$ and $\lambda = 3$. Successive `(slow,fast)` pairs before the first meeting are `(1,2)`, `(2,4)`, `(3,3)`. After resetting one pointer, the pairs are `(0,3)`, `(1,4)`, `(2,2)`, locating entry 2.

Compare node identity, not stored value. Locating an entry and breaking a cycle are different operations; this algorithm returns the entry without changing any `next` link.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two meetings of Floyd’ s pointers

Floyd: first meeting, then locate the entry



First phase: slow takes one step; fast takes two
(1,2) → (2,4) → (3,3)

Reset one pointer to head; advance both one step
(0,3) → (1,4) → (2,2)

Compare node identity; the first meeting need not be the entry.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Local consistency in doubly linked lists

- Nodes have prev and next. Neighboring nodes satisfy `x.next.prev=x` and `x.prev.next=x`, with endpoints handled by the sentinel policy.
- To insert x between adjacent p and q, set `x.prev=p,x.next=q,p.next=x,q.prev=x`.
- To delete known x, set `x.prev.next=x.next` and `x.next.prev=x.prev`, then release x.
- Updating a known node costs O(1). Finding a node by value still costs O(n). Removing a known node and searching then removing are different operations.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Doubly linked erasure and reference lifetime

```
doubly_erase(x):
    require x is a live owned data node, x!=H and x!=T
    p=x.prev; q=x.next
    p.next=q; q.prev=p
    release(x); size-=1
```

This procedure fixes two nondata sentinels H and T. H.prev=null and T.next=null; an empty list has H.next=T, T.prev=H, and size=0. Every data node has nonnull prev/next, making p.next and q.prev valid. A null-ended representation instead needs explicit head/tail repair when erasing an endpoint and cannot directly use this procedure.

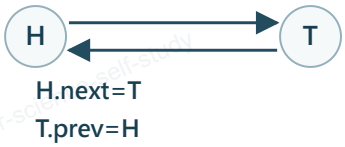
For one item H↔x↔T, p=H and q=T. Erasure restores H.next=T and T.prev=H and decreases size from 1 to 0. Never release H or T. The same sentinel links cover first/last data nodes in longer lists. Validate the live owned handle before entering pointer operations.

Deleting p↔x↔q sets both p.next=q and q.prev=p. The reciprocal conditions are restored together. References to surviving nodes remain valid; every reference to x is invalid. Updating only one direction gives inconsistent traversal sequences.

Doubly linked list: removing the only node

Doubly linked sentinels: remove the only data node

① Save p=x.prev=H and q=x.next=T
② Set p.next=q and q.prev=p
③ Release x; size changes from 1 to 0



H and T remain valid; external pointers to x must not be dereferenced.

Choosing lists or arrays

Operation or constraint	Dynamic array	Singly linked list
Indexed access	O(1)	O(n)
Insertion with known predecessor	O(n) shifts in the worst case	O(1) link changes
Sequential scan	Usually storage locality	Pointer following
Extra space per item	Unused capacity	Next link and allocation overhead

The complete workload determines the tradeoff. Link counts alone do not establish measured speed.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft forward_list, list and unique_ptr.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are recursion, linked structures, stacks, and induction.
- Describe trees precisely. Trace traversals, BST updates, and heap operations while preserving their respective invariants.
- Distinguish an arbitrary binary tree, a binary search tree, and a binary heap by their requirements rather than appearance alone.

Mathematical definition and terminology

- An undirected tree is a connected acyclic simple graph. Every pair of vertices has exactly one simple path, and an n-vertex tree has n−1 edges.
- Choosing a root gives each nonroot node a unique parent. Its children are adjacent nodes one edge farther from the root.
- Depth counts edges from the root. A node's height counts edges to its deepest descendant leaf. Tree height is root height.
- A leaf has no children. A forest is a disjoint collection of trees. We assign the empty tree height −1.

Left and right positions have meaning

- A binary-tree node has at most one left child and one right child. A lone left child and a lone right child give different structures.
- A binary tree of height h≥0 has at most `2^{h+1}-1` nodes, since depth d has at most `2^d` nodes.
- Here a full binary tree has exactly two children at every internal node. A complete tree fills every level before the last, then fills the last from left to right without gaps.
- A perfect tree has all leaves at the same depth and two children at every internal node. Fix these conventions before using terms that vary across texts.

Some sources use full for a perfect tree and strict for a tree whose internal nodes all have two children. Use the stated definition rather than relying on the label.

Leaves and branching: count edges in two ways

In a nonempty binary tree let n_0, n_1, n_2 count nodes with zero, one, and two children. Each nonroot node has exactly one incoming parent edge. The edge count is therefore both $n-1$ and n_1+2n_2.

Substitute $n = n_0 + n_1 + n_2$. Then $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$, so **$n_0=n_2+1$**. Completeness and full branching are unnecessary. The empty tree is excluded.

Example: 55 nodes, 22 with one child. From $55 = n_0 + 22 + (n_0 - 1)$, obtain $n_0=17$ and $n_2=16$. For a general rooted tree, the same count gives $n_0 = 1 + \sum_{d\geq 2}(d-1)n_d$ over $d\geq 2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Height bounds and complete-tree indices

We measure height in edges and assign the root depth 0. Depth d contains at most 2^d nodes: each node at depth d has at most two children. Summing gives $n \leq 2^{h+1} - 1$, so $h \geq \lceil \log_2(n+1) \rceil - 1$ for a nonempty tree. A chain attains maximum height n−1.

If the root is at level 1 and height counts levels, add 1 to these height values. Do not mix the two conventions in a calculation.

For zero-based complete-tree index i, left=2i+1, right=2i+2, and parent=⌊(i−1)/2⌋. A child exists only when its index is below n. With n=10 the last node with a child is i=4; leaves occupy 5 through 9. With one-based index j the corresponding formulas are ⌊j/2⌋, 2j, and 2j+1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

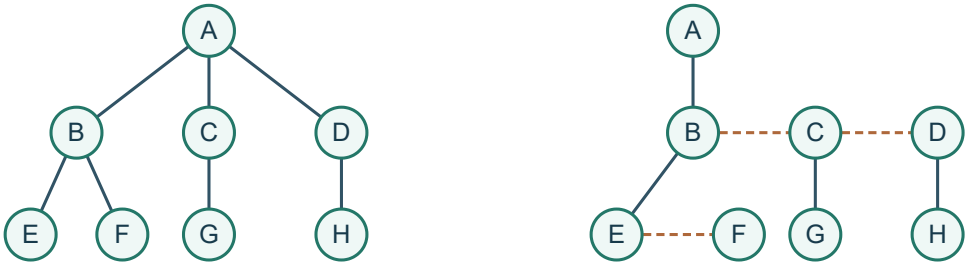
Linked and array representations

- Linked binary nodes store `{key,left,right}` and naturally accommodate irregular shapes.
- A complete tree fits a zero-based array. Node i has children `2i+1,2i+2` and parent `⌊(i−1)/2⌋`, with the parent formula restricted to i>0.
- Complete-position numbering of a chain of height h can require exponentially large indices. Array representation is not space-efficient for every binary tree.
- Check multiplication and addition for overflow before computing child indices, then check the index against the element count.

This chapter uses zero-based heap indices. Algorithm texts often use one-based indices, with parent ⌊j/2⌋ and children 2j and 2j+1. Both encode the same tree; check the convention before substituting.

First child and next sibling

An ordered tree and its first-child / next-sibling encoding



Original: A has ordered children B, C, D

Solid left = first child

Dashed right = next sibling

On the right, C is B's sibling, not B's child in the original tree.

Reading the encoding and counting shapes

A node's left link points to its first child; its right link points to its next sibling. Follow the first child's right chain to enumerate the original children in order. A forest can use right links between successive roots. This representation is a bijection for **ordered** rooted trees; discarding sibling order loses that correspondence.

Let C_n count ordered binary-tree shapes with n unlabeled nodes, including $C_0=1$. Give i nodes to the left subtree and n−1−i to the right. Then $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$, giving $C_1=1$, $C_2=2$, $C_3=5$ and $C_4=14$.

For a fixed set of n distinct totally ordered BST keys, each shape has exactly one labeling whose inorder sequence is increasing. Hence there are also C_n BST shapes. If arbitrary labels count as different in a general binary tree, there are $n!C_n$ labeled trees; these counts are not interchangeable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Catalan closed form by balanced paths

Encode the empty tree as the empty word and a nonempty tree as `(`, the left encoding, `)`, and the right encoding. Matching the root's first parentheses uniquely recovers the two subtrees. This is a bijection between n-node ordered binary-tree shapes and balanced words of n parenthesis pairs.

Read an opening parenthesis as an up-step and a closing one as a down-step. There are $\binom{2n}{n}$ paths with n of each. Reflect the prefix through the first visit to −1. This reversible map sends invalid paths to paths with n+1 up-steps and n−1 down-steps, giving $\binom{2n}{n-1}$ invalid paths.

Therefore $C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}$. At n=0 the count is 1 for the empty tree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three depth-first traversals

- Preorder visits root, left, right. Inorder visits left, root, right. Postorder visits left, right, root.
- Our original tree has root 8, left child 3 with children 1 and 6, and right child 12 with left child 10.
- Preorder is `8,3,1,6,12,10`. Inorder is `1,3,6,8,10,12`. Postorder is `1,6,3,10,12,8`.
- Applying the same rule recursively to each subtree proves by induction that each node is visited once, giving Θ(n) time.

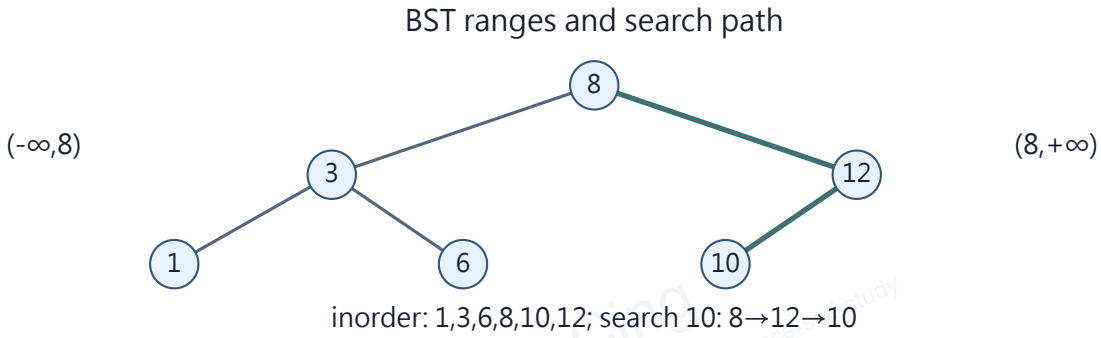
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursive traversals and complete iterative inorder states

```
visit(x,order):
    if x==null: return
    if order==pre: emit(x)
    visit(x.left,order)
    if order==in: emit(x)
    visit(x.right,order)
    if order==post: emit(x)
inorder_iterative(root):
    stack=[]; x=root
    while x!=null or stack is not empty:
        while x!=null: stack.push(x); x=x.left
        x=stack.pop(); emit(x); x=x.right
```

https://github.com/xinhi6716/com

Recursive traversals and complete iterative inorder states (continued)



Push 8,3,1 to obtain stack [8,3,1]. Pop 1 and emit [1]. Pop 3 and emit [1,3], then push 6 for stack [8,6]. Pop 6 and emit [1,3,6]. Pop 8, then follow 12's left chain to [12,10]. Pop 10 and 12, ending with [1,3,6,8,10,12]. The stack stores pending ancestors. Each node is pushed/popped once, giving Θ(n) time and O(h+1) space.

Inorder of an arbitrary binary tree need not be sorted. That property requires the entire BST subtree-range invariant, not merely this example's output.

Reconstructing a tree from preorder and inorder

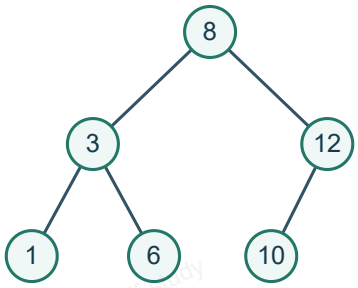
Partitioning preorder and inorder

pre: 8 | 3, 1, 6 | 12, 10
in: 1, 3, 6 | 8 | 10, 12

Root 8 splits inorder into 3 left and 2 right keys

Left: pre 3,1,6; in 1,3,6

Right: pre 12,10; in 10,12



Repeat: preorder supplies the root; inorder fixes subtree sizes.

Keys must be distinct; reject inconsistent traversal pairs.

Reconstruction: proof and limits

Assume distinct keys and two traversals of the same tree. The first preorder key uniquely selects the root. Its inorder position uniquely partitions the keys and fixes both subtree sizes. Split the remaining preorder sequence by those sizes, then recurse. Induction on node count uniquely determines each subtree and the whole tree.

Build a key-to-inorder-position map and use interval bounds with a preorder cursor. Each key is processed once: expected $\Theta(n)$ time, $\Theta(n)$ map space and $O(h+1)$ call stack. State the hash table's separate worst case. Reject repeated keys, mismatched sets, or a root outside the current inorder interval.

Preorder and postorder alone are generally ambiguous. Root A with a sole child B on the left or on the right gives preorder A,B and postorder B,A. If every internal node is additionally guaranteed to have two children and keys are distinct, the next preorder key locates the left subtree in postorder, enabling unique recursive reconstruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

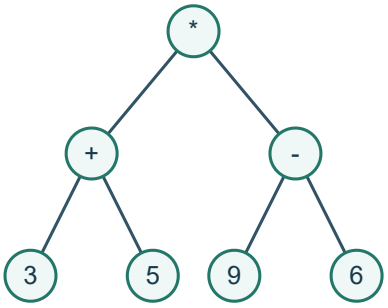
Stack and queue space in traversal

- Recursive DFS needs $O(h+1)$ extra stack space for a nonempty tree. A chain has $h=n-1$, so the bound is not always $O(\log n)$.
- Iterative inorder pushes the current left chain. It pops and outputs one node, then processes that node's right subtree.
- Level order uses a queue. The example gives `8,3,12,1,6,10`.
- Its queue needs $O(w)$ space, where w is maximum level width. Even a balanced tree may have $w=\Theta(n)$, unlike its DFS stack.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expression trees and traversal order

Expression tree: traversal and evaluation



Inorder: (3 + 5) × (9 − 6)

Preorder: × + 3 5 − 9 6

Postorder: 3 5 + 9 6 − ×

Postorder evaluation: 8 × 3 = 24

Inorder needs parentheses where precedence or associativity would alter the tree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preserving expression semantics

An operator's left and right children are its left and right operands. Postorder obtains both operands before applying their parent. For `3 5 + 9 6 − ×`, the intermediate results are 8 and 3, followed by 24. Subtraction and division are not commutative: the first pop is the right operand and the second pop is the left operand.

Preorder gives prefix notation and postorder gives postfix. Unparenthesized inorder can change the computation: `(3+5)×(9−6)` is not the same expression as `3+5×9−6`. Retain enough parentheses to preserve precedence and associativity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST invariant and search

- Keys are distinct. At every node x, all left-subtree keys are less than x.key and all right-subtree keys are greater.
- A comparison with k discards the root and one subtree. The whole-subtree invariant proves that the discarded region cannot contain k.
- Searching for 10 follows `8→12→10`. Searching for 11 ends at the empty right-child position of 10.
- Search, insertion, and deletion take O(h+1). Without balancing, increasing insertion order can produce linear height.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST insertion and duplicate policy

- Follow search to an empty child position. Adding a leaf there preserves every ancestor's allowed key interval.
- Inserting 11 into the example makes it the right child of 10. The path constrains `8<11<12` and `10<11`.
- A duplicate policy must agree with the invariant. Options include counts, unique composite keys, or explicit nonstrict subtree rules.
- In a key-value container, replacing an existing key's value differs from adding a node.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST lookup, insertion, and global-range validation

```
find(root,k):
    x=root
    while x!=null:
        if k==x.key: return x
        if k<x.key: x=x.left
        else: x=x.right
    return not_found
insert(x,k,v):
    if x==null: return Node(k,v,left=null,right=null)
    if k<x.key: x.left=insert(x.left,k,v)
    else if k>x.key: x.right=insert(x.right,k,v)
    else: x.value=v
    return x
valid(x,lower,upper):
    if x==null: return true
    return lower<x.key<upper and valid(x.left,lower,x.key)            and
valid(x.right,x.key,upper)
root=insert(root,k,v)
```


BST lookup, insertion, and global-range validation (continued)

An existing key updates its value without allocating another node. The validator starts with mathematical bounds $-\infty,+\infty$. An implementation over a bounded key type needs optional bounds; representable minimum and maximum keys cannot substitute for infinity.

Searching for 11 in the running tree compares 8<11,12>11,10<11 and reaches the empty right child of 10. Insertion allocates there and returns the roots through 10.right,12.left,8.right and the external root. Ancestor intervals are $(8,+\infty),(8,12),(10,12)$, all containing 11. Reinserting 11 changes only its value.

In the counterexample root 8, left child 3, and right child 9 below 3, every local parent-child comparison passes. Global validation rejects 9 against inherited bounds (3,8). Lookup correctness uses the same global invariant: if k<x.key, the entire right subtree has keys greater than x.key and cannot contain k. Lookup and insertion take $O(h+1)$ worst-case time; recursive insertion also uses $O(h+1)$ stack space. Validation examines every node and takes $\Theta(n)$, not $O(h)$.

Three deletion cases

1. Remove a leaf from its parent link.
2. Replace a one-child node by its child subtree.
3. For two children, copy the logical contents of the minimum node in the right subtree, the inorder successor, then delete the successor's original position.

The successor has no left child, so its removal reduces to the first two cases. External node handles require a consistent choice between moving values and transplanting nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked two-child deletion trace

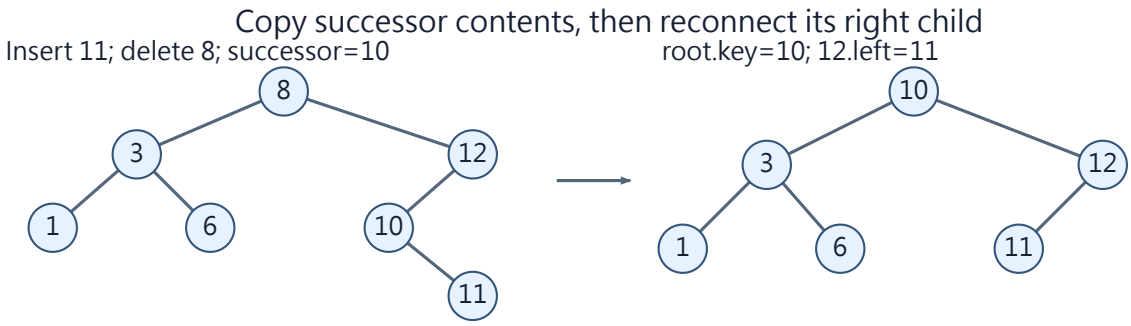
- Delete 8 from the original tree. The right-subtree minimum is 10. Change the root key to 10, then remove 12's left child 10.
- The new inorder is `1,3,6,10,12`, exactly the original sorted sequence without 8.
- The successor exceeds every key in the original left subtree and is less than every remaining right-subtree key. Replacement preserves order.
- Checking only parent-child comparisons is insufficient. A left child 3 of root 8 could have right descendant 9. Every local edge comparison holds, but the global BST invariant fails.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete recursive BST deletion and identity contract

```
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x.left==null: y=x.right; release(x); return y
        if x.right==null: y=x.left; release(x); return y
        s=x.right
        while s.left!=null: s=s.left
        x.key_and_value=copy(s.key_and_value)
        x.right=erase(x.right,s.key)
    return x
root=erase(root,k)
```


Complete recursive BST deletion and identity contract (continued)



Insert 11 to the right of 10. Deleting 8 selects successor 10, which has no left child but has right child 11. After copying 10 to the root, recursive removal returns 11 into 12.left. Inorder loses exactly 8. The caller must assign the returned root; changing only a local pointer can leave an external root dangling.

This version preserves a key-value set, not node-handle identities. Stable handles to surviving elements require node transplantation with parent/root updates instead of content copying. Payload-copy exceptions need a separate commitment policy. Search plus the successor's left chain costs O(h+1), which can be Θ(n) without balancing.

Ordinary successor: individual versus full-traversal cost

```
successor(x):
    require x is a live node and parent links are maintained
    if x.right!=null:
        x=x.right
        while x.left!=null: x=x.left
        return x
    p=x.parent
    while p!=null and x==p.right: x=p; p=p.parent
    return p
```

The successor is the smallest strictly greater key, not the numeric key plus one. If a right subtree exists, its leftmost node is the answer. Otherwise climb past ancestors whose right subtree has just been completed. The first ancestor containing the current subtree on its left is the answer; climbing beyond the root means the key is maximal. This representation stores ordinary children and parents, unlike the chapter's tagged-thread representation.

For root 11, left subtree 5(2,7), and right subtree 17(13,19), successor(5) follows the right child to 7. Successor(7) climbs through 5 to 11. Successor(19) climbs through 17 and 11 to null. Even with a node handle, one call may traverse height h and cost O(h+1), rather than O(1).

Ordinary successor: individual versus full-traversal cost (continued)

Starting at the minimum and repeatedly taking successors outputs 2,5,7,11,13,17,19. Each tree edge is traversed downward at most once and upward at most once, so initialization plus the full traversal costs Θ(n). Multiplying the maximum per-call cost by n gives only an upper bound, not a tight Θ(nh) result. Arbitrary disconnected successor queries cannot inherit the full-traversal average. Without parents, threads, or an explicit ancestor stack, the interface must search from the root again.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Merging ordered sources and building a balanced tree linearly

```
merge_unique(streams):
    H=min_heap of each nonempty stream's first record; pending=none
    while H not empty:
        (key,value,source)=H.pop_min()
        if pending==none: pending=(key,value)
        else if key==pending.key: pending.value+=value
        else: emit pending; pending=(key,value)
        advance source; if nonempty: H.push(next record from source)
    if pending!=none: emit pending
build_sorted(count):
    if count==0: return null
    left_count=count//2
    left=build_sorted(left_count)
    (k,v)=next(unique_sorted_stream)
    x=Node(k,v,left=left,right=null,parent=null,size=count)
    if left!=null: left.parent=x
    x.right=build_sorted(count-left_count-1)
    if x.right!=null: x.right.parent=x
    return x
```


Merging ordered sources and building a balanced tree linearly (continued)

Fix duplicate semantics first: let values count occurrences and sum equal-key values. Streams [2,7,17],[5,7,13],[11,19] contain eight records but seven unique keys 2,5,7,11,13,17,19; key 7 has value 2. With k sources and N input records, heap merging costs O(k+N log(k+1)), linear for fixed k. It is not O(N) for arbitrary growing k. The unique count U must be known: count and replay repeatable sources, or buffer U records for a one-pass source.

Build_sorted(7) first consumes three records into subtree 5(2,7), then consumes 11 as the root, then constructs 17(13,19). Each node consumes one record in inorder, so the resulting inorder equals the ordered input. Induction proves global BST ranges. Child counts differ by at most one, giving height O(log(U+1)). The recurrence $T(U) = T(\lfloor U/2 \rfloor) + T(U - \lfloor U/2 \rfloor - 1) + \Theta(1)$ gives $\Theta(U)$ construction time. Output nodes occupy Θ(U); the extra recursion stack uses O(log(U+1)).

Inserting U already sorted keys individually into an unbalanced BST instead creates a chain and takes 1+2+…+(U−1)=Θ(U^2) comparisons. Sorted input helps direct construction but does not accelerate an ordinary insertion routine automatically. Taking array midpoints yields the same linear construction; this version demonstrates that random indexing is unnecessary.

Threaded trees reuse empty pointers

- Tags distinguish child links from threads. Otherwise empty left and right positions can point to inorder predecessors and successors.
- For the inorder successor, follow a right thread directly. If the right link is a real child, enter that subtree and follow real left children to the end.
- A full traversal needs no recursion or explicit stack. Total time is O(n), with O(1) additional traversal state.
- Insertion and deletion must repair affected threads. Treating a thread as a recursive child can create nonterminating cycles.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Concrete threaded traversal and insertion

- In the example, leaf 6 has predecessor thread 3 and successor thread 8. Leaf 10 has predecessor 8 and successor 12.
- After 6, follow its right thread to 8. Node 8 has a real right child, so enter 12 and follow its real left child to 10.
- Insert 11 to the right of 10 where a thread previously pointed to 12. Give 11 left and right threads to 10 and 12. Change 10's right link and tag to a real child pointing to 11.
- Inorder becomes `1,3,6,8,10,11,12`. Leaving 10's right tag as a thread makes search and traversal interpret the link incorrectly. Maintain pointer and tag as one update.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inorder successor and right insertion with double threads

```
successor(x):
    if x.right_tag==thread: return x.right
    x=x.right
    while x.left_tag==child: x=x.left
    return x
insert_right(p,k):
    require p.right_tag==thread and k>p.key
    s=p.right
    require s==null or k<s.key
    x=Node(k,left=p,right=s,left_tag=thread,right_tag=thread)
    p.right=x; p.right_tag=child
    if s!=null and s.left_tag==thread and s.left==p: s.left=x
```

This variant has no header sentinel; extreme threads are null. Start at the leftmost real child and follow successors until null. Inserting11 at10's right-thread position sets new11.left=10 and 11.right=12 as threads, then10.right=11 as a child. If12.left is a real child pointing to 10, it must remain unchanged. Update it only if it was a predecessor thread to 10.

Inorder successor and right insertion with double threads (continued)

One successor may descend O(h), but a complete traversal uses real edges and threads only boundedly many times, giving O(n) total and O(1) extra traversal state. Pointer and tag jointly define representation. Updating one without the other can skip11 or produce cycles. Threaded deletion must repair these extra links in addition to ordinary BST order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Threaded-tree deletion, endpoints, and identity

Use the preceding doubly threaded representation: no header, null outer threads, and distinct keys. Search descends only through links tagged child. Reduce a two-child deletion to removing its successor with at most one real child, then repair neighboring threads and the parent link. Never recurse through a thread as though it were a subtree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Threaded-tree deletion, endpoints, and identity (continued)

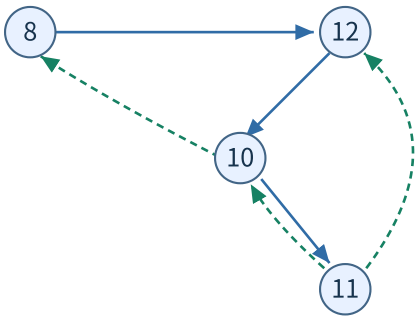
```
predecessor(x):
    if x.left_tag==thread: return x.left
    x=x.left
    while x.right_tag==child: x=x.right
    return x
erase_threaded(k):
    p=null; z=root
    while z!=null and z.key!=k:
        if k<z.key:
            if z.left_tag==thread: return absent
            p=z; z=z.left
        else:
            if z.right_tag==thread: return absent
            p=z; z=z.right
    if z==null: return absent
    if z.left_tag==child and z.right_tag==child:
        p=z; s=z.right
        while s.left_tag==child: p=s; s=s.left
```

Threaded-tree deletion, endpoints, and identity (continued)

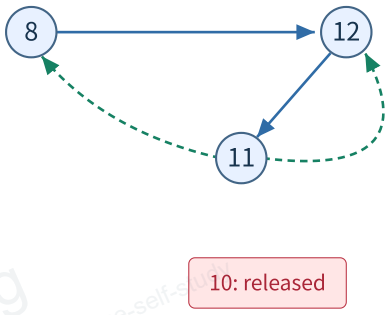
```
        z.key_and_value=copy(s.key_and_value); z=s
    pred=predecessor(z); succ=successor(z)
    replacement=null
    if z.left_tag==child: replacement=z.left
    else if z.right_tag==child: replacement=z.right
    if pred!=null and pred.right_tag==thread and pred.right==z:
        pred.right=succ
    if succ!=null and succ.left_tag==thread and succ.left==z:
        succ.left=pred
    if p==null: root=replacement
    else if p.left_tag==child and p.left==z:
        if replacement!=null: p.left=replacement
        else: p.left=pred; p.left_tag=thread
    else:
        if replacement!=null: p.right=replacement
        else: p.right=succ; p.right_tag=thread
    release(z); return removed
```


Threaded-tree deletion, endpoints, and identity (continued)

Before: 12.left=10; 11.left(thread)=10



After: 12.left=11; 11.left(thread)=8



10: released

Blue solid: real child. Green dashed: thread. Null outer threads omitted.
Subtree {1,3,6} below 8.left is unchanged and omitted.

Delete node 10 after it has right child 11 in the original example. Then p=12,pred=8,succ=11,replacement=11. Redirect 11's left thread from 10 to 8, redirect 12's real left child to 11, retain 11's right thread to 12, and only then release 10. Inorder changes from [1,3,6,8,10,11,12] to [1,3,6,8,11,12]. successor(8) descends through 12's real left child to 11; predecessor(11) follows its thread directly to 8.

Threaded-tree deletion, endpoints, and identity (continued)

Deleting leaf 6 changes parent 3's right child into a thread to successor 8; retaining its child tag is invalid. Deleting the singleton has pred=succ=replacement=null and sets root=null before release. Empty or unsuccessful deletion changes nothing. The two-child branch copies the complete key-value pair into the target and physically frees the successor. This version preserves dictionary contents, not undeleted node-handle identities; payload copy cost and exceptions are separate.

Correctness has three parts. The successor has no real left child, so the physical deletion has at most one child. Attaching replacement preserves all remaining real subtrees and inorder. Only the predecessor's successor thread, successor's predecessor thread, and parent's vacated-side tag can refer to the removed position; repairing them leaves no dangling thread. Update tags together with pointers and never overwrite a real child with a thread unconditionally. Search, successor reduction, and predecessor/successor lookup each follow at most a height-h path, for O(h+1) time and O(1) auxiliary pointers.

A min-heap has shape and order invariants

- A binary min-heap is complete, and every parent key is at most each child key. Its root is the global minimum.
- A heap imposes neither left-child versus right-child order nor whole-left versus whole-right subtree order. BST search does not apply.
- `[2,5,4,9,7,8]` is a valid original heap, not a globally sorted array. Arbitrary-key search remains Θ(n) in the worst case.
- A Priority Queue ADT offers insert, find-min, and delete-min. A binary heap is one implementation.

Sift-up and sift-down

- Insert 1 at the last position and swap upward while the parent is larger. It moves through indices 6,2,0, giving `[1,5,2,9,7,8,4]`.
- Delete-min replaces the root with the last element and swaps down with the smaller child until order is restored. An arbitrary child choice may leave a smaller child below its parent.
- Edges outside the update path are unchanged. Repairing each violated edge along the path preserves heap order and completeness.
- Heap restructuring for insert and delete-min takes worst-case O(log n). Find-min is O(1). Underlying-array growth adds allocation and movement costs, amortized across insertions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete binary min-heap sifting

```
insert(v):
    A.append(v); i=len(A)-1
    while i>0:
        p=(i-1)//2
        if A[p]<=A[i]: break
        swap(A[p],A[i]); i=p
delete_min():
    require len(A)>0
    result=A[0]; last=A.pop()
    if len(A)==0: return result
    A[0]=last; sift_down(0,len(A)); return result
sift_down(i,n):
    while i<n//2:
        j=2*i+1
        if j+1<n and A[j+1]<A[j]: j+=1
        if A[i]<=A[j]: break
        swap(A[i],A[j]); i=j
build_heap():
    for i=n//2-1 down to 0: sift_down(i,n)
```

Complete binary min-heap sifting (continued)

On [2,5,4,9,7,8], inserting 1 appends at index 6, swaps 6↔2 then 2↔0, and gives [1,5,2,9,7,8,4]. Delete-min replaces the root with final 4, giving [4,5,2,9,7,8]. Comparing children 5 and 2 selects 2, producing [2,5,4,9,7,8]. Child 8 is no smaller than 4, so sifting stops. A singleton removal returns before writing A[0].

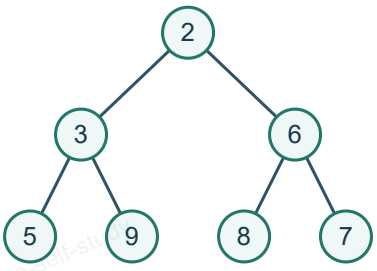
The condition `i<n//2`, with integer division, means exactly that a left child exists. It avoids computing `2*i+1` before proving the index is in range. For n=3, only i=0 is internal; for n=4, both i=0 and i=1 are internal. Compute j only inside the loop and test j+1<n before reading a right child.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Heap construction: comparisons and swaps

Building a min-heap bottom up

Index and comparison	Array after each comparison or swap (zero-based)
i=2 6 ≤ 7, 8	[9, 5, 6, 2, 3, 8, 7]
i=1 5 ↔ 2	[9, 2, 6, 5, 3, 8, 7]
i=0 9 ↔ 2	[2, 9, 6, 5, 3, 8, 7]
i=0 9 ↔ 3	[2, 3, 6, 5, 9, 8, 7]



At i=2 the child heaps already satisfy heap order; this remains true at each sift-down.

The last row is the complete array; the tree shows the same final state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why bottom-up construction is linear

- Sift down from the final internal node $\lfloor n/2 \rfloor - 1$ toward the root. Each processed node already has heap-ordered child subtrees.
- The number of nodes of height h is O(n/2^h), while a sift at height h costs O(h).
- Total work is $O(n \sum_{h \geq 0} h/2^h) = O(n)$. Reading the input takes Ω(n), so construction is Θ(n).
- Multiplying n nodes by an O(log n) maximum cost gives only a loose O(n log n) upper bound, not a tight Θ(n log n) result.

`std::priority_queue<int>` exposes the largest element by default. Specify `std::greater<int>` for a min-priority queue and match the comparator to the chosen convention.

Choosing a tree representation

Structure	Required invariant	Suited to
General binary tree	Left and right positions are distinct	Expressions and recursive structures
BST	Left-subtree keys < root < right-subtree keys	Search, ranges and ordered traversal; cost depends on height
Min-heap	Complete shape and parent key ≤ child key	Find, insert and delete minimum
Doubly threaded tree	Tags and links agree on child versus thread	Inorder traversal without an external stack

Choose the representation from the promised operations. Local heap order cannot discard an entire subtree for an arbitrary key, so lookup is generally linear. An unbalanced BST can also degrade to linear search.

References and attribution

Lin Cing, handwritten data structures notes; ISO C++ draft priority_queue.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are sets, trees, stacks, queues, priority queues, and induction.
- Choose graph representations, prove search and greedy-edge invariants, and select shortest-path algorithms using their weight conditions.
- Distinguish minimum spanning trees, shortest paths, topological orders, and critical paths by their optimization objectives.

Graph ADT and problem contract

- A graph is `G=(V,E)`. A directed edge `(u,v)` has a direction. An undirected edge `{u,v}` does not. Weights form a mapping `w:E→R`.
- Unless stated otherwise, graphs have no self-loops or parallel edges. Multigraph extensions need explicit representation and duplicate policies.
- An ADT may enumerate vertices and neighbors, add edges, and query weights. Representation determines each operation's cost.
- Weight 0 is valid. A weighted matrix cannot use the same zero value to mean both a zero-weight edge and no edge.

Adjacency matrices and lists

Representation	Space	Test a particular edge	Enumerate neighbors of u
Matrix	$\Theta(V^2)$	$O(1)$	$\Theta(V)$
Unsorted lists	$\Theta(V+E)$	$O(\deg(u))$	$\Theta(\deg(u))$

Here V and E abbreviate set sizes. An undirected edge normally has two adjacency entries. Checking for duplicates before insertion changes the insertion cost.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The BFS level invariant

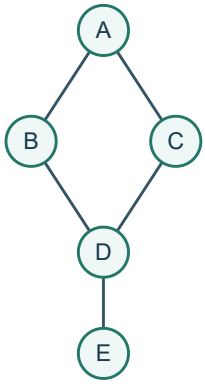
1. Set `dist[s]=0` and enqueue s. Other vertices are undiscovered.
2. Dequeue u. For each undiscovered neighbor v, set `dist[v]=dist[u]+1`, `parent[v]=u`, and enqueue v.
3. Mark discovery when enqueuing, preventing duplicate queue entries.

Queued distances are nondecreasing. At v's first discovery, a predecessor on any shorter path would already have been processed. The assigned distance is therefore the minimum edge count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS queue and predecessor trace

BFS: mark on enqueue and trace the queue



Dequeued	queue afterward	new parent
A	[B, C]	B←A, C←A
B	[C, D]	D←B
C	[D]	—
D	[E]	E←D
E	[]	—

Distances A,B,C,D,E = 0,1,1,2,3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS colors, timestamps, and directed cycles

- White vertices are undiscovered, gray vertices are active, and black vertices are finished. The recursion stack is a gray path.
- In a directed graph, a back edge to a gray ancestor proves a directed cycle. An edge to a black vertex alone does not.
- In an acyclic graph, decreasing finish time gives a valid topological order.
- A single-source search may miss other components. Restart at remaining white vertices to construct a search forest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search cost and reproducible traces

- A complete adjacency-list BFS or DFS handles each vertex and adjacency entry a constant number of times, giving $\Theta(V+E)$ time and $O(V)$ auxiliary space.
- A matrix implementation scans a full row per vertex, giving $\Theta(V^2)$. The list bound cannot be applied without its representation condition.
- Original undirected edges are `AB,AC,BD,CD,DE`. With alphabetical neighbors, BFS from A visits `A,B,C,D,E`.
- Recursive DFS visits `A,B,D,C,E`. Neighbor order changes visit order, but not reachability or BFS distances.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete BFS and DFS state machines

```
BFS(s):
    for v in V: dist[v]=infinity; parent[v]=none
    dist[s]=0; Q.push(s)
    while Q not empty:
        u=Q.pop_front()
        for v in adj[u]:
            if dist[v]==infinity:
                dist[v]=dist[u]+1; parent[v]=u; Q.push(v)
DFS(u):
    color[u]=gray; discover[u]=++clock
    for v in adj[u]:
        if color[v]==white: parent[v]=u; DFS(v)
        else if directed and color[v]==gray: report back edge
    color[u]=black; finish[u]=++clock
```

For undirected AB, AC, BD, CD, DE with alphabetical neighbors, queues after processing A, B, C, D, E are [B, C],[C, D],[D],[E],[]. Distances are (0,1,1,2,3), parents (none, A, A, B, D). DFS timestamps are A:(1,10), B:(2,9), D:(3,8), C:(4,5), E:(6,7), with C←D, E←D.

Complete BFS and DFS state machines (continued)

Gray-target edges detect ancestors in directed DFS. In undirected DFS, ignore the reverse adjacency entry of the parent tree edge. A complete forest initializes all vertices white and restarts at each remaining white vertex. Single-source BFS leaves other components unreachable. Auxiliary worst-case storage is Θ(V), excluding input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MST objective and the cut property

- A spanning tree of a connected undirected weighted graph has all vertices and V−1 edges. An MST minimizes total weight, allowing negative weights. Disconnected graphs instead have a minimum spanning forest.
- Maintain the invariant that some MST T contains the entire current selected set F. A cut $(S, V \setminus S)$ respects F when no edge of F crosses it. Let e be a lightest crossing edge.

If e∈T, containment already holds. Otherwise adding e to T creates one cycle, containing another edge f crossing the same cut. Since the cut respects F, f∉F. Also w(e)≤w(f), so T′=T−f+e is a spanning tree no heavier than optimal T. It is therefore an MST containing F∪{e}. The exchange preserves every previous selection; this proves safety relative to F.

Individual membership in some MST is insufficient. In a triangle with all three weights 1, each edge belongs to an MST, but the set of all three is a cycle and belongs to none. Safety must refer to the whole chosen set F.

Prim grows one vertex set

- Let S be the included vertices. For every outside vertex v, store the cheapest edge to S as key[v], with its parent.
- Repeatedly add an outside vertex of minimum key and update its neighbors. The selected edge is lightest across $(S, V \setminus S)$.
- With an indexed binary heap and adjacency lists, time is O((V+E)log V), or O(E log V) for connected nonsingleton graphs. Linear selection with a matrix costs $O(V^2)$.
- Key measures a single edge connecting to the current tree. It is not a cumulative source distance and cannot be substituted for Dijkstra's dist.

Start induction with F=∅. All Prim-selected edges lie inside the current S, so $(S, V \setminus S)$ respects F. Its lightest crossing edge preserves containment in an MST by the safe-edge theorem. In a connected graph the final acyclic F has V−1 edges and is itself that MST.

Kruskal and disjoint sets

1. Sort all edges by weight.
2. Process `(u,v)` from lightest to heaviest. If endpoints lie in different components, select the edge and union the components.
3. In a connected graph, stop after V−1 selections. Otherwise finish the minimum spanning forest.

Path compression and union by rank give an O(m α(V)) amortized bound for m disjoint-set operations. Sorting normally dominates at O(E log E). Here α is inverse Ackermann, not the hash-table load factor.

For Kruskal, take S as the entire current F-component containing u. A chosen e=(u,v) joins different components, so it crosses a cut respecting F. Any lighter crossing edge would have been processed earlier while its endpoints were still separate; it would have been selected and merged them, a contradiction. Thus e is lightest across this cut, preserving the same containment invariant. Include Θ(V) initialization: total O(V+E log(E+1)+E α(V)) time and Θ(V) DSU space.

Complete disjoint-set forest operations

A disjoint-set union structure (DSU) represents a partition of vertices. Every nonempty set has one root with parent[root]=root. find returns that root; union merges two different sets and reports whether the partition changed. Allocate V array entries before make_set and require valid vertex IDs.

```
make_set(v): parent[v]=v; rank[v]=0
find(v):
    r=v
    while parent[r]!=r: r=parent[r]
    while parent[v]!=v:
        next=parent[v]; parent[v]=r; v=next
    return r
union(u,v):
    a=find(u); b=find(v)
    if a==b: return false
    if rank[a]<rank[b]: swap(a,b)
    parent[b]=a
    if rank[a]==rank[b]: rank[a]+=1
    return true
```

Complete disjoint-set forest operations (continued)

The second find loop saves next before overwriting parent; otherwise it loses the uncompressed suffix. Rank records a historical height bound, not the current height after compression, and is never decreased after compression. The Boolean union result is exactly Kruskal's edge-acceptance test.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disjoint-set states and a self-contained cost bound

Use the same MST's selected edges BC, AB, DE, CD, calling union in that endpoint order. On equal ranks retain the first root. Parent entries below are ordered A,B,C,D,E.

Completed operation	parent[A..E]	Roots and ranks
Five make_set calls	[A,B,C,D,E]	A0,B0,C0,D0,E0
union(B,C)	[A,B,B,D,E]	A0,B1,D0,E0
union(A,B)	[B,B,B,D,E]	B1,D0,E0
union(D,E)	[B,B,B,D,D]	B1,D1
union(C,D)	[B,B,B,B,D]	B2
find(E)	[B,B,B,B,B]	B2

The final find first follows E→D→B, then links E directly to B. Membership is unchanged. Later edges BD, AC, CE have equal representatives B. Their unions return false without increasing ranks or accepting another edge.

Disjoint-set states and a self-contained cost bound (continued)

For correctness, induct on the forest invariant. find redirects a node only to its own ancestral root, preserving the partition and acyclicity. union connects two different roots and merges exactly their sets. Rank strictly increases on every nonroot-to-parent edge: a lower rank joins a higher rank; a tie raises the winning root's rank. Compression to an ancestor preserves the inequality. Initially each rank-0 set has one vertex. A rank-r root represents at least 2^r vertices, since a rank increase merges two sets of at least 2^{r-1} vertices each. Thus $\mathrm{rank} \leq \lfloor \log_2 V \rfloor$, every path and single find/union cost O(1+log(V+1)), initialization costs Θ(V), and storage is Θ(V). This fully proves the usable Kruskal bound O(V+E log(E+1)+E log(V+1)).

The finer inverse-Ackermann layering theorem gives O(V+m α(V)) for V initial make_set calls followed by m find/union operations. The rank-height proof above does not prove that sharper theorem. Neither the implementation nor the proved logarithmic bound depends on the omitted fine-grained analysis. Sorting and initialization of all vertices remain part of the total.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked MST trace and counterexample

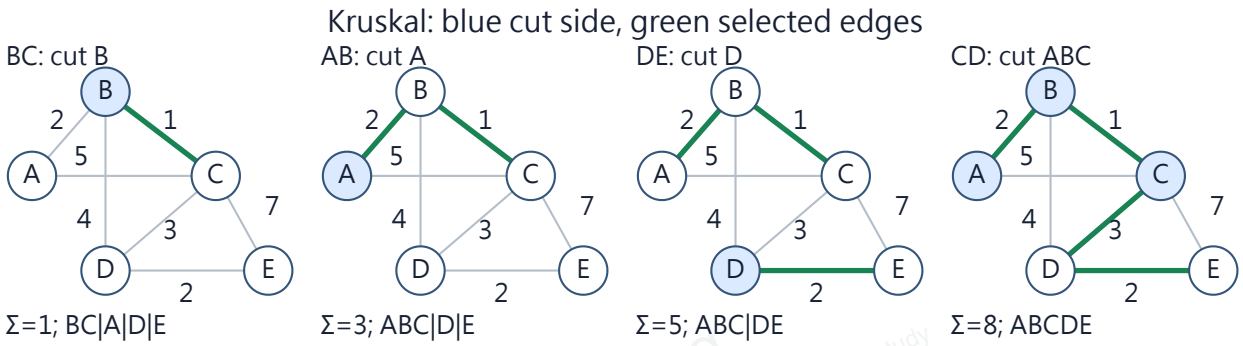
- Weights are `AB=2,AC=5,BC=1,BD=4,CD=3,CE=7,DE=2`.
- Kruskal can select `BC,AB,DE,CD`, totaling 1+2+2+3=8. Tie order may vary without changing the optimal total.
- Prim from A can select `AB,BC,CD,DE`, the same set in another order.
- An MST does not guarantee shortest source-to-vertex paths. For triangle `AB=2,BC=2,AC=3`, the MST path from A to C costs 4 while the graph's shortest distance is 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete Prim and Kruskal choices and states

```
Prim(s):
    key[*]=infinity; parent[*]=none; key[s]=0
    while some vertex is unselected:
        u=unselected vertex with minimum key
        if key[u]==infinity: start a new component or report disconnected
        select u
        for (u,v,w) in adj[u]:
            if v unselected and w<key[v]: key[v]=w; parent[v]=u
Kruskal():
    make_set(v) for all v; result=[]
    for (u,v,w) in sorted_edges:
        if find(u)!=find(v): result.append((u,v,w)); union(u,v)
    return result
```


Complete Prim and Kruskal choices and states (continued)



For AB2, AC5, BC1, BD4, CD3, CE7, DE2, Prim from A gives outside keys after A: B2, C5, D∞, E∞; after B: C1, D4, E∞; after C: D3, E7; after D: E2; then E completes the tree. Parents are B←A, C←B, D←C, E←D, total 8. Key reductions 5→1 and 7→2 are single-edge costs, not accumulated source distances.

Kruskal order is BC1, AB2, DE2, CD3, BD4, AC5, CE7. The first four join different components; the last three would create cycles. DSU establishes component membership, while the cut-exchange theorem proves optimality. Ties may yield multiple distinct minimum trees.

Shortest paths and relaxation

- A path length sums its edge weights. $\delta(s,v)$ is the minimum source-to-v length when a finite minimum exists. Unreachable vertices have $+\infty$.
- If s reaches a negative cycle that can reach v, walk lengths to v are unbounded below. There is no finite shortest value.
- Relax (u,v) when d[u] is finite and $d[u]+w(u,v)<d[v]$. Update d[v] and its parent.
- Every finite d represents a discovered walk length and is an upper bound on the optimal distance. Machine-integer sums additionally need overflow protection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra requires nonnegative weights

- Initialize d[s]=0. Repeatedly select an unsettled u of minimum d, settle it, then relax its outgoing edges.
- The proof requires that the remaining edges of a path cannot lower its accumulated length. All relevant edge weights must be nonnegative.
- In the original counterexample $s\to a=2, s\to b=5, b\to a=-4$, permanently settling a at 2 misses its true distance 1. No negative cycle is needed for this failure.
- Zero-weight edges are legal. Strict positivity is not required. For negative-edge graphs choose an algorithm whose conditions hold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked Dijkstra trace and implementation choices

Directed edges are `s→a=4`, `s→b=1`, `b→a=2`, `a→t=1`, `b→t=6`.

Settled	d[a]	d[b]	d[t]
s	4	1	∞
b	3	1	7
a	3	1	4
t	3	1	4

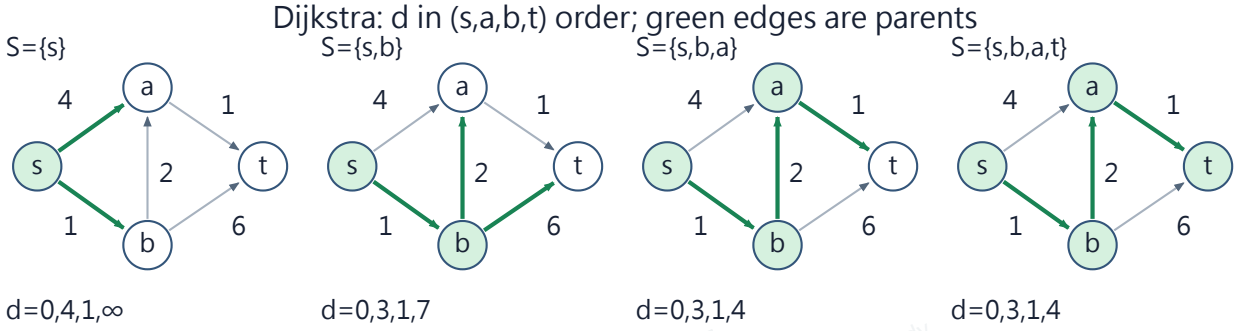
An indexed binary heap gives O((V+E)log V). With duplicate insertions instead of decrease-key, skip stale distances. Space can reach O(V+E), with the conservative time bound O((V+E)log(V+E)).

The matrix version scans all unsettled distances to select a minimum and scans the selected row, taking $O(V^2)$ time and $\Theta(V^2)$ matrix space. Adjacency lists with an indexed heap select and update differently. The relaxation rule and nonnegative-weight requirement are shared; their costs are not interchangeable.

Dijkstra queue entries, stale values, and predecessors

```
Dijkstra(s):
    require all relevant edge weights>=0
    d[*]=infinity; parent[*]=none; d[s]=0
    H.push((0,s)); settled={}
    while H not empty:
        (du,u)=H.pop_min()
        if du!=d[u] or u in settled: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            if d[u]+w<d[v]:
                d[v]=d[u]+w; parent[v]=u; H.push((d[v],v))
```

Dijkstra queue entries, stale values, and predecessors (continued)



For s→a4, s→b1, b→a2, a→t1, b→t6, start with H=[(0, s)]. Removing s creates (1, b),(4, a). Removing b adds (3, a),(7, t). Removing a=3 adds (4, t). Old (4, a) is skipped because du differs from current d[a]; old (7, t) is also skipped. Final distances are s0, a3, b1, t4 with a←b, b←s, t←a.

If a shorter path to settled u existed, let y be its first unsettled vertex and x its settled predecessor. Relaxation by x makes d[y] no greater than that path prefix. Nonnegative remaining edges make the prefix no longer than the alleged shorter path, contradicting minimum d selection. Negative s→a2, s→b5, b→a−4 breaks that step without a negative cycle. Strict improvement avoids gratuitous equal-distance parent changes that can create cycles around zero-weight edges.

Bellman–Ford induction on edge count

- Perform V−1 full edge-relaxation passes. In a synchronous-array version, pass k gives shortest walk lengths using at most k edges.
- An in-place version can propagate farther within a pass. After pass k it has nevertheless covered every path of at most k edges.
- Without a reachable negative cycle, a shortest walk can be simplified to a path with at most V−1 edges. A further possible relaxation proves a source-reachable negative cycle.
- In-place relaxation takes O(V+VE) time. The synchronous version here copies V entries per round and takes O(V(V+E)) time, with O(V) distance/parent space. Initialization only at s detects source-reachable negative cycles, not necessarily every negative cycle in the graph.

Floyd–Warshall recurrence on intermediate vertices

- Without relevant negative cycles, let `D^(k)[i,j]` be the shortest distance whose intermediate vertices belong to `{0,…,k-1}`.
- The recurrence is `D^(k+1)[i,j]=min(D^(k)[i,j],D^(k)[i,k]+D^(k)[k,j])`.
- Initialize diagonal entries to 0, direct edges to their weights, and all other entries to infinity. The k loop must be outermost to match this invariant. Check finiteness before adding distances.
- Time is $\Theta(V^3)$ and space $\Theta(V^2)$. A final `D[v,v]<0` detects a negative cycle. If i reaches such a v and v reaches j, the i-to-j optimum is unbounded below. The computed finite number must not be reported as a finite shortest distance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman–Ford edge layers and Floyd intermediate layers

```
BellmanFord_synchronous(s):
    d[*]=infinity; d[s]=0
    repeat V-1 times:
        next=copy(d)
        for (u,v,w) in E:
            if finite(d[u]): next[v]=min(next[v],d[u]+w)
        d=next
    if any finite(d[u]) and d[u]+w<d[v]: report reachable_negative_cycle
Floyd():
    D=direct_edges_and_zero_diagonal
    for k in V:
        for i in V:
            for j in V:
                if finite(D[i,k]) and finite(D[k,j]):
                    D[i,j]=min(D[i,j],D[i,k]+D[k,j])
```

For s→a2, s→b5, b→a−4, a→t3, synchronous BF vectors (s, a, b, t) are round 0=(0,∞,∞,∞), round 1=(0,2,5,∞), round 2=(0,1,5,5), round 3=(0,1,5,4). An extra pass makes no improvement. In-place relaxation may reach the final vector in its first pass; it does not satisfy the same exact edge-layer equality.

Bellman–Ford edge layers and Floyd intermediate layers (continued)

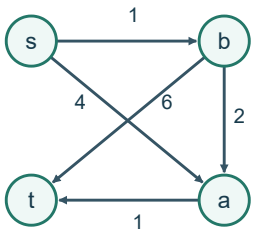
For the nonnegative example in order (s, a, b, t), initial Floyd matrix is `[[0,4,1,∞],[∞,0,∞,1],[∞,2,0,6],[∞,∞,∞,0]]`. Intermediate s changes nothing. Intermediate a gives D[s, t]=5, D[b, t]=3. Intermediate b gives D[s, a]=3, D[s, t]=4. Intermediate t changes nothing. Final matrix is `[[0,3,1,4],[∞,0,∞,1],[∞,2,0,3],[∞,∞,∞,0]]`, whose s row matches Dijkstra.

Return to the negative-edge example, whose b→a weight is −4. Adding a→b of weight 1 creates a reachable −3 cycle; adding that edge to the nonnegative example with b→a weight 2 would not. After BF's extra pass, forward search from affected endpoints classifies a, b, t as −∞ while s remains 0. Floyd marks only pairs reaching and leaving a k with D[k, k]<0. A cycle-existence flag is not full affected-vertex classification.

Do not hide copy(d) inside edge-relaxation cost. For a legal graph with $V \geq 2$ and $E=0$, the fixed $V-1$ rounds copy $V(V-1)$ entries, taking $\Theta(V^2)$. The in-place version needs $\Theta(V)$ initialization and $O(V)$ empty edge-loop checks. In general each synchronous round takes $\Theta(V+E)$, so its total is $\Theta(V(V+E))$; the in-place bound is $O(V+VE)$. Both simplify to $O(VE)$ only with an additional $E=\Omega(V)$ assumption. Exact edge-layer states cost two-array maintenance. An early-stop-on-no-change variant terminates after one empty-edge round and must be specified separately.

Floyd updates as intermediates are admitted

Floyd–Warshall: adding allowed intermediate vertices



Initially: D[s,a]=4, D[s,b]=1, D[s,t]=∞

k=a: D[s,t]=min(∞,4+1)=5

D[b,t]=min(6,2+1)=3

k=b: D[s,a]=min(4,1+2)=3

D[s,t]=min(5,1+3)=4

Intermediate order s, a, b, t; k is outermost. The final s→b→a→t path has length 4.

The k=a phase establishes b→a→t before k=b uses it to improve s→t.

Induction for shortest-path recurrences and negative cycles

```
// Synchronous edge-count recurrence:
D[0,s]=0; D[0,v!=s]=infinity
D[k,v]=min(D[k-1,v], min_{(u,v) in E}(D[k-1,u]+w(u,v)))
// Allowed-intermediate recurrence, with k outermost:
F[k+1,i,j]=min(F[k,i,j],F[k,i,k]+F[k,k,j])
```

For synchronous Bellman–Ford, zero edges reach only the source. A walk using at most k edges either already uses fewer edges, or ends in some edge u→v with a prefix using at most k−1 edges. Minimizing over these alternatives proves both directions of the recurrence. When finite shortest distances exist, removing nonnegative cycles yields a simple path with at most V−1 edges, so V−1 rounds suffice.

An improvement in the extra round yields an at-most-V-edge walk better than every at-most-(V−1)-edge walk. It repeats a vertex. If all its cycles were nonnegative, removing a cycle would contradict that improvement; a reachable negative cycle therefore exists. Conversely, if all edges satisfied $d[v] \le d[u]+w(u,v)$, summing these inequalities around a reachable negative cycle would give $0 \le$ a negative number. Thus some edge remains relaxable. Only vertices reachable from affected endpoints have distance $-\infty$; unrelated components remain unaffected.

Induction for shortest-path recurrences and negative cycles (continued)

Floyd's induction concerns permitted intermediate vertices rather than edge counts. Without a relevant negative cycle, choose a shortest path without repeated vertices. Upon admitting k, it either avoids k or splits into i→k and k→j, each using the old intermediate set. Joining optimal subpaths gives the reverse inequality and proves equality. In-place updates work because D[k,k]=0 leaves D[i,k] and D[k,j] unchanged in this round. With a negative cycle this finite-distance argument fails and reachability must classify $-\infty$ pairs. Three nested loops alone do not justify moving i to the outermost position.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shortest-path conditions compared

Method	Condition	Problem and cost
BFS	Equal positive edge cost, or count edges only	Single source O(V+E)
Topological relaxation	Directed acyclic graph, negative edges allowed	Single source O(V+E)
Dijkstra	Nonnegative weights	Single source, see “Dijkstra’s priority queue, stale entries, and predecessors” for heap variants
Bellman–Ford	Negative edges allowed	Single source; in-place O(V+VE), synchronous O(V(V+E)); detects reachable negative cycles
Floyd–Warshall	Negative edges allowed	All pairs $\Theta(V^3)$, classify negative-cycle effects

Weight restrictions, acyclicity, and single-source versus all-pairs requirements determine the method before data structure preferences.

Bottleneck paths: objective, relaxation, and full trace

```
bottleneck(s):
    B[*]=infinity; parent[*]=none; B[s]=-infinity
    H.push((-infinity,s)); settled={}
    while H not empty:
        (bu,u)=H.pop_min()
        if u in settled or bu!=B[u]: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            candidate=max(B[u],w)
            if candidate<B[v]:
                B[v]=candidate; parent[v]=u; H.push((candidate,v))
```

A path's bottleneck is its maximum edge weight. The objective is $\min_{P:s\rightsquigarrow t} \max_{e\in P} w(e)$, rather than the sum of its weights. The empty path has max-identity $-\infty$ and unreachable vertices $+\infty$. Nonnegative inputs permit source label 0, but with negative edges that initialization would incorrectly raise an all-negative path's bottleneck to zero.

Use undirected edges s–p=6,p–t=6,s–q=2,q–t=9,p–q=7. After extracting s, B(p,q,t)=(6,2,∞). Extract q to obtain (6,2,9) with t.parent=q. Extract p to obtain (6,2,6) with t.parent=p. Extract t and return bottleneck 6 along s,p,t. The shortest-sum path is instead s,q,t of length 11, while s,p,t totals 12.

Bottleneck paths: objective, relaxation, and full trace (continued)

For correctness, consider the first unsettled vertex on an allegedly better path. Its settled predecessor has already proposed a value at most that path prefix's bottleneck. Extending a path with max cannot decrease its value, so this candidate is at most the whole better path's bottleneck, contradicting extraction of the minimum-labeled u. This monotonicity holds even with negative edges, unlike additive Dijkstra. Stale-entry bounds match the earlier implementation; indexed binary heaps give O((V+E)log(V+1)).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The MST bottleneck property and threshold connectivity

```
threshold_bottleneck(s,t):
    require undirected graph and s!=t
    make_set(v) for all vertices
    for (u,v,w) in edges sorted by weight:
        union(u,v)
        if find(s)==find(t): return w
    return unreachable
```

In any undirected MST T, let β be the maximum weight on its unique s–t path, and remove a path edge e of weight β. This splits s,t across a cut. A lighter crossing edge f would replace e and strictly reduce tree weight, contradicting minimality. Every s–t path therefore crosses an edge of weight at least β. The tree path achieves β, proving it is minimax.

This is not a shortest-sum property. The example MST is {s–q2,s–p6,p–t6}, total weight 14. Its s–t path totals 12 rather than the graph's shortest 11, but its maximum edge 6 is optimal. Negative edges do not invalidate the exchange or max argument. The tree-path theorem does not transfer directly to directed graphs, where an ordinary MST is not a directed reachability tree.

The MST bottleneck property and threshold connectivity (continued)

Equivalently, find the smallest threshold λ at which s, t become connected in $G_{\leq\lambda}$. Union edges in increasing weight order and return the first connecting weight. Sorting costs O(E log(E+1)); DSU work is amortized O(E α(V)). This solves a single pair's bottleneck, not its additive distance. Repeated queries can use maximum-edge queries on an MST, but that needs separate preprocessing and data-structure analysis.

Initializing all V disjoint sets takes Θ(V) time and Θ(V) DSU space, giving total O(V+E log(E+1)+E α(V)) time. Storing edges for sorting adds a separate Θ(E) edge buffer; sorting workspace depends on the chosen algorithm. With E=0 and V isolated vertices, all sets are still initialized before returning unreachable, taking Θ(V), rather than zero.

Weight layers and mandatory edges across multiple MSTs

```
classify_MST_edges(G):
    require G undirected and connected
    DSU=singleton components
    for all equal-weight groups in increasing weight:
        Q=multigraph whose vertices are current DSU components
        for edge (u,v) in this group:
            if find(u)==find(v): mark edge as never_in_MST
            else: add its original edge ID between these two vertices in Q
        mark Q bridges as in_every_MST; other Q edges as optional
        for (u,v) in this group: union(u,v)
```

At threshold λ, let c_λ count components of $G_{\leq\lambda}$, the original graph restricted to edges of weight at most λ. Every MST has exactly $n - c_\lambda$ edges at or below this threshold. Otherwise some threshold component remains disconnected inside the tree's low-weight forest. A low-weight edge reconnects those parts; adding it creates a cycle containing an edge heavier than λ. Replacing that edge strictly lowers tree weight, a contradiction. A forest spanning each threshold component has exactly n−cλ edges.

Weight layers and mandatory edges across multiple MSTs (continued)

Subtract consecutive threshold counts: every MST selects the same number of edges at each weight, though identities can differ. Alternative edges need not all share one weight globally. Contract strictly lighter components; at the current weight choose a spanning forest in each component of quotient multigraph Q. Bridges must be chosen; cycle edges can be included or omitted; contracted self-loops are never chosen. Classify the entire group before unioning it, or early unions falsely reject equal-weight alternatives.

Q can contain parallel edges even when the original graph is simple. Parallel edges between two components substitute for each other. Bridge DFS must skip the parent edge by ID, not every edge to the parent vertex. Sorting plus linear DFS over all quotient groups costs O(E log(E+1)), with additional amortized DSU work. The result certifies whether an edge is necessary in every optimum, beyond finding one optimum.

Complete bridge low-link procedure and proof

```
bridges(Q):
    tin[*]=unset; clock=0; result=[]
    for v in Q.vertices:
        if tin[v]==unset: dfs(v,none)
    return result
// dfs shares tin, low, clock, result with the enclosing routine.
dfs(v,parent_edge):
    tin[v]=low[v]=clock; clock+=1
    for (u,edge_id) in Q.adj[v]:
        if edge_id==parent_edge: continue
        if tin[u]==unset:
            dfs(u,edge_id); low[v]=min(low[v],low[u])
            if low[u]>tin[v]: result.append(edge_id)
        else: low[v]=min(low[v],tin[u])
```

Tin is first discovery time. Low[v] is the minimum discovery time reachable by descending zero or more DFS tree edges inside v's subtree and then using at most one nonparent back edge. It is not a shortest-path distance. For tree edge v→u, low[u]>tin[v] means no edge returns from u's subtree to v or an ancestor, so removing the edge disconnects it. If low[u]≤tin[v], a return route bypasses the parent edge and it is not a bridge.

Complete bridge low-link procedure and proof (continued)

Let Q have X,Y,Z, parallel edges e1,e2 between X,Y, and e3 between Y,Z. Enter Y via e1 with tin(X)=0,tin(Y)=1. Skip e1 at Y, but e2 returns to X and sets low(Y)=0. Visit Z with tin(Z)=low(Z)=2. Since 2>1, e3 is a bridge. Returning to X, low(Y)=0 is not greater than 0, so e1 is not a bridge; neither is back edge e2. Skipping every edge to X would incorrectly classify e1.

Each vertex is discovered once and each undirected adjacency entry is processed once, giving $\Theta(|V_Q|+|E_Q|)$ time and $O(|V_Q|)$ extra space. For MST classification, create quotient vertices only for components incident to the current group. Their total over groups is O(E); allocating a full original-V array for every group would invalidate the claimed total linear group processing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Articulation points: deleting a vertex

An articulation point in an undirected graph is a vertex whose deletion together with incident edges increases the number of connected components. The definition also applies to a disconnected graph; start a DFS tree in each undiscovered component.

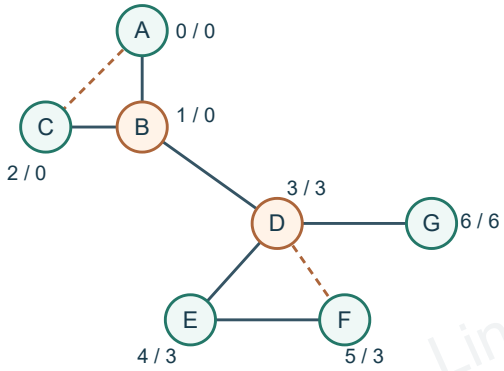
Let tin[v] be discovery time and low[v] the minimum discovery time reachable from v's DFS subtree by zero or more downward tree edges followed by at most one back edge. After finishing tree child u, propagate low[u] into low[v].

A nonroot v is an articulation point exactly when some DFS child u satisfies $low[u] \geq tin[v]$. A DFS root instead needs at least two DFS children; it has no parent-side component. A bridge requires low[u]>tin[v]. The equality case matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A numerical articulation-point and bridge trace

Articulation points and bridges in one DFS tree



Labels are tin / low

Solid: DFS tree edge

Dashed: back edge to ancestor

B: low[D]=3 ≥ tin[B]=1

D: low[E]=3 ≥ tin[D]=3

Bridges: BD, DG (strict >)

Root A has only one DFS child B, so A is not an articulation point.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why ≥ for a cut vertex but > for a bridge?

For nonroot v and tree child u, low[u]≥tin[v] means the subtree has no back edge to a strict ancestor of v. Deleting v disconnects that subtree from the parent side. Conversely, low[u] <tin[v] supplies a bypass for each child subtree, so deleting v does not separate that subtree from the parent side.

At equality, a back edge can reach v, so the tree edge (v,u) has an alternative route and is not a bridge. Deleting v removes that return route as well, so v can still be an articulation point. Distinct DFS-root child subtrees cannot have an edge between them, or DFS would have discovered one from the other. At least two such children are therefore required and sufficient.

In the figure low[E]=tin[D]=3: DE is not a bridge, but D is a cut vertex. low[D]=3>tin[B]=1 makes BD a bridge, and low[G]=6>tin[D]=3 makes DG a bridge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete articulation-point DFS

```
articulation_points(G):
    tin[*]=unset; clock=0; answer={}
    for v in V:
        if tin[v]==unset: dfs(v, parent_edge=none)
    return answer

dfs(v, parent_edge):
    tin[v]=low[v]=clock; clock+=1; children=0
    for (u, edge_id) in adj[v]:
        if edge_id==parent_edge: continue
        if tin[u]==unset:
            children+=1; dfs(u,edge_id)
            low[v]=min(low[v],low[u])
            if parent_edge!=none and low[u]>=tin[v]: answer.add(v)
        else: low[v]=min(low[v],tin[u])
    if parent_edge==none and children>=2: answer.add(v)
```

The forest includes isolated vertices. Skip the specific incoming edge ID, rather than every edge to the parent vertex, to handle parallel edges. With adjacency lists the time is Θ(V+E), auxiliary space is O(V), and recursion depth can be Θ(V).

AOV networks and topological sorting

- An activity-on-vertex network represents activities by vertices and precedence by directed edges. A feasible dependency order requires a DAG.
- Kahn's algorithm enqueues all indegree-zero vertices. Output one, remove its outgoing edges, and enqueue any neighbor whose indegree becomes zero.
- Remaining indegrees count only unremoved edges. If vertices remain but none has indegree zero, the remaining graph contains a directed cycle.
- Fewer than V output vertices detects a cycle. Time is Θ(V+E). Multiple eligible vertices allow multiple valid orders.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AOE networks and earliest event times

- An activity-on-edge network assigns durations to edges and events to vertices. An activity starts only after its preceding event completes.
- Durations d(u, v) are nonnegative. The graph is a DAG with a common start and finish. Add zero-duration dummy edges when there are multiple starts or finishes.
- In topological order, compute `E[v]=max_{(u,v)}(E[u]+d(u,v))`, with E[start]=0.
- This is a longest-path computation on a DAG, not Dijkstra. The maximum expresses the requirement that every prerequisite event must finish.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Latest events, slack, and critical activities

- Set `L[t]=E[t]`, then in reverse topological order compute `L[u]=min_{(u,v)}(L[v]-d(u,v))`.
- Activity `(u,v)` has earliest start E[u] and latest start L[v]−d(u, v). Total slack is their difference.
- Zero-slack activities are critical. Several critical paths may exist. Finding one does not imply all other activities can be delayed.
- The model assumes unlimited resources and fixed durations. Shared workers, machine conflicts, and random durations require further scheduling models.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked critical-path data

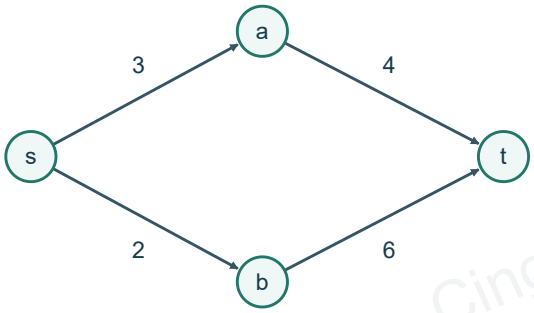
Activities are `s→a:3,s→b:2,a→t:4,b→t:6`.

- Earliest events are E[s]=0, E[a]=3, E[b]=2, E[t]=8.
- Latest events are L[t]=8, L[a]=4, L[b]=2, L[s]=0.
- Both s→a and a→t have total slack 1. Both s→b and b→t have slack 0, yielding critical-path duration 8.
- The two noncritical activities cannot both independently consume one extra unit without delaying completion. Slack shared by one path cannot be spent twice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AOE event times and activity slack

AOE: event times and activity slack



Event	earliest E	latest L
s	0	0
a	3	4
b	2	2
t	8	8

s→b→t takes 8; s→a→t takes 7 and shares one unit of slack.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete topological and AOE forward/backward computation

```
Kahn():
    indegree=counts of all incoming edges; Q=all zero-indegree vertices
    order=[]
    while Q not empty:
        u=Q.pop(); order.append(u)
        for (u,v) in outgoing: indegree[v]-=1; if indegree[v]==0: Q.push(v)
    require len(order)==V
AOE(order):
    E[*]=-infinity; E[start]=0
    for u in order:
        for (u,v,d): E[v]=max(E[v],E[u]+d)
    L[*]=infinity; L[finish]=E[finish]
    for u in reverse(order):
        for (u,v,d): L[u]=min(L[u],L[v]-d)
```

For s→a3, s→b2, a→t4, b→t6, indegrees start (0,1,1,2). Output s makes a, b zero. Output a lowers t to 1; output b lowers t to 0; output t completes s, a, b, t. Earliest E starts (0,−∞,−∞, −∞), becomes (0,3,2,−∞) after s, t=7 after a, and t=8 after b. Backward L starts with t=8, gives b=2, a=4, then s=min(4−3,2−2)=0.

Complete topological and AOE forward/backward computation (continued)

Activity tuples (earliest start, latest start, total slack) are s→a:(0,1,1), s→b:(0,0,0), a→t:(3,4,1), b→t:(2,2,0). Fixed durations and unlimited resources give critical path s, b, t of length 8. The length 7 a-path shares one unit of slack; increasing both durations by 1 makes length 9. Add zero-duration virtual boundaries for multiple starts/finishes and restrict analysis to activities reachable between them.

References and attribution

Lin Cing, handwritten data structures notes; MIT 6.006 lecture notes on graph search and shortest paths.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are arrays, heaps, recursion, expectation, and Chapter 1 complexity definitions.
- Explain each algorithm's invariant, correctness, and cost. Distinguish stability, in-place storage, and the comparison model.
- Choose using existing order, key domain, memory limits, and worst-case requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sorting contract and stability

- The output must be a permutation of the input and satisfy `A[i]≤A[i+1]`. Order alone does not rule out lost or duplicated elements.
- Stability preserves the original relative order of equal-key records. Sorting `(2,A),(1,X),(2,B)` must leave A before B.
- Here in-place means O(1) extra data-array space. Recursion-stack space is reported separately.
- The comparator must express a consistent order. C++ sorting requires strict weak ordering. Using $\leq$ as an ordinary comparator violates that requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion-sort invariant

1. Before iteration i, `A[0,i)` is sorted and is a permutation of the original first i elements.
2. Save key=A[i]. Shift prefix elements strictly greater than key rightward until its insertion slot is available.
3. Write key there, obtaining sorted prefix `A[0,i+1)`.

Moving only strictly greater keys preserves stability. Example `[5,2,4,2]` becomes `[2,5,4,2]`, `[2,4,5,2]`, then `[2,2,4,5]`.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion sort and inversions

- An inversion is a pair i<j with A[i]>A[j]. Each rightward shift removes one inversion involving the current key.
- With I inversions, standard insertion sort takes $\Theta(n+I)$.
- Sorted input has I=0 and best time $\Theta(n)$. Distinct reverse-sorted input has $I=n(n-1)/2$ and worst time $\Theta(n^2)$.
- The inversion count explains suitability for small or nearly sorted inputs without an unsupported claim about average speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quicksort partition contract

- Three-way partition divides the range around pivot value p into `<p`, `=p`, and `>p`. Recurse only on the outer regions.
- Maintain `[l,lt)<p`, `[lt,i)=p`, unclassified `[i,gt)`, and `[gt,r)>p`.
- For a smaller value, swap to lt and increment lt and i. For equality, increment i. For a larger value, decrement gt, swap there, and keep i to inspect the incoming value.
- The measure gt−i decreases each iteration. At termination every element is classified. Partition time is Θ(r−l).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-way partition indices and recursion contract

```
partition(A,l,r,p):
    lt=l; i=l; gt=r
    while i<gt:
        if A[i]<p: swap(A[lt],A[i]); lt+=1; i+=1
        else if A[i]>p: gt-=1; swap(A[i],A[gt])
        else: i+=1
    return (lt,gt)
quicksort(A,l,r):
    if r-l<2: return
    choose p from A[l:r]
    (lt,gt)=partition(A,l,r,p)
    quicksort(A,l,lt); quicksort(A,gt,r)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For [4,2,4,7,1,4], states (A, lt, i, gt) are initially (A,0,0,6), then (A,0,1,6), ([2,4,4,7,1,4],1,2,6), (same,1,3,6), ([2,4,4,4,1,7],1,3,5), (same,1,4,5), and ([2,1,4,4,4,7],2,5,5). Recurse only on [0,2) and [5,6).

Three-way partition indices and recursion contract (continued)

Choosing the pivot from the subarray guarantees a nonempty equal region and smaller recursive problems. An absent extreme pivot could leave a recursive side equal to the original range. Expected bounds require random selection, not a fixed middle-position value. Smaller-side recursion bounds stack depth but leaves quadratic worst-case time possible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quicksort worst-case and expected time

- For `[4,2,4,7,1,4]` and p=4, a partition is `[2,1] | [4,4,4] | [7]`. Order inside regions is unspecified.
- With distinct keys and extreme pivots, `T(n)=T(n−1)+Θ(n)`, yielding worst-case $\Theta(n^2)$.
- For a fixed distinct-key input, uniformly random pivots in every subproblem give expected Θ(n log n). In the classic model comparing the pivot once with each other key, the exact expectation is `2(n+1)H_n−4n`. Randomness comes from the algorithm.
- Three-way partition removes equal keys together. The distinct-key exact formula does not count comparisons in every three-way implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expected quicksort comparisons and bounded recursion depth

```
quicksort_small_side(A,l,r):
    while r-l>=2:
        p=uniformly_random_value_from(A[l:r])
        (lt,gt)=partition(A,l,r,p)
        if lt-l<r-gt:
            quicksort_small_side(A,l,lt); l=gt
        else:
            quicksort_small_side(A,gt,r); r=lt
```

Rank the distinct keys $1, \dots, n$, and let X_{ij} indicate whether ranks i, j are ever compared directly. The classic two-way version compares each pivot exactly once with every other key in its subproblem. Two keys are compared exactly when the first pivot chosen from the rank interval $[i, j]$ is one of its endpoints. The $j - i + 1$ keys are symmetric, so $\Pr(X_{ij} = 1) = 2/(j - i + 1)$. By linearity of expectation,

$$\mathbb{E}[C] = \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{d=1}^{n-1} \frac{n-d}{d+1} = 2(n+1)H_n - 4n = \Theta(n \log n),$$

Expected quicksort comparisons and bounded recursion depth (continued)

where $H_n = \sum_{k=1}^{n} 1/k$.

The three-way implementation tests <, >, and = sequentially, so its distinct-key partitions still use only a bounded number of comparisons per key. Expected O(n log n) remains valid, but the exact classic comparison count does not transfer unchanged. Each pivot must be compared with every nonpivot key at least once. The same random rank splits therefore also give the indicator-sum lower bound, establishing expected Θ(n log n) for distinct keys. All-equal input needs only one three-way partition and Θ(n) work.

Space has a separate deterministic proof: the actual recursive child above is the smaller nonequal region, of length at most (r−l−1)/2. Active frame sizes therefore halve at each nesting, giving worst-case O(log n) frames. The larger side uses the current frame's while loop. Even repeated extreme pivots still scan total length n+(n−1)+…=Θ(n^2); bounded stack space does not remove that time worst case.

Recursion space and depth protection

- Recursing on both sides can have $\Theta(n)$ worst-case stack depth. In-place partition does not imply constant total auxiliary space.
- Recurse only on the smaller side and process the larger side in a loop. Each actual recursive subproblem is at most half as large, limiting stack space to $O(\log n)$.
- This improves space but does not eliminate $\Theta(n^2)$ worst-case comparison work.
- Introspective sorting additionally switches to heapsort when recursion becomes too deep. It is an engineering extension.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

Merge sort and stable merging

- For sorted halves, compare their first unconsumed values and output the smaller. Choose the left value on equality to preserve stability.
- The output is the sorted merge of consumed input, and its last value is no larger than either unconsumed remainder.
- Merging `[2_A,5]` and `[2_B,4]` gives `[2_A,2_B,4,5]`. Taking right first on equality reverses the equal-key records.
- Standard array merging uses a $\Theta(n)$ buffer. The recurrence `T(n)=2T(n/2)+Θ(n)` gives $\Theta(n \log n)$ worst-case time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete stable merge and merge-sort

```
merge(L,R):
    i=0; j=0; out=[]
    while i<len(L) and j<len(R):
        if L[i].key<=R[j].key: out.append(L[i]); i+=1
        else: out.append(R[j]); j+=1
    append remaining L[i:]; append remaining R[j:]
    return out
merge_sort(A):
    if len(A)<2: return A
    m=len(A)//2
    return merge(merge_sort(A[:m]),merge_sort(A[m:]))
```

Merging L=[2_A,5], R=[2_B,4] first takes A on equality, with cursors (1,0) and output [A]. Taking B gives (1,1) and [A,B]. Taking 4 gives (1,2) and [A,B,4]. Appending 5 completes [A,B,4,5]. Choosing the smaller remaining head proves order, and consuming every input exactly once proves permutation preservation.

The slicing merge_sort is semantic pseudocode. The shared-buffer C++ implementation avoids retaining needless copies across levels.

Heapsort's sorted suffix

- For ascending order, build a max-heap, swap the root with the last heap element, and exclude that position from the heap.
- The prefix is a max-heap, the suffix is increasing, and every prefix key is at most every suffix key.
- The extracted root is the largest remaining key and belongs at the front of the sorted suffix. Sift down the replacement root.
- Construction is Θ(n). Extractions take Θ(n log n) worst-case total time. An iterative implementation uses O(1) extra space and is generally unstable.

Complete heapsort driver and sorted-suffix trace

```
heap_sort(A):
    build_max_heap(A)
    for end=len(A)-1 down to 1:
        swap(A[0],A[end])
        sift_down_max(A,0,end)    // heap is [0,end), sorted suffix excluded
```

Heapsort on the same keys starts with max-heap [5,2,4,2]. Removing 5 gives [2,2,4|5], sifted to [4,2,2|5]. Removing 4 gives [2,2|4,5], then [2|2,4,5]. The fixed sorted suffix never reenters sifting. Equal value results do not establish record-identity stability.

Max-sifting chooses the larger child and stops when its parent is at least as large; indices match the min version with reversed comparisons.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting sort assumes a bounded key domain

- Integer keys in `[0,K)` use K counters. Prefix sums give the end position of each key's output block.
- Scan original records right to left, placing key k at `--end[k]`. This preserves equal-key order.
- Time and combined count/output space are Θ(n+K). A huge K relative to n may be impractical.
- Directly indexing by key obtains more than pairwise comparison information, so the pure comparison lower bound does not apply directly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stable counting allocation and record-by-record scatter

```
stable_count(records,K):
    out=array(len(records)); count=[0]*K
    for x in records: require 0<=x.key<K; count[x.key]+=1
    end=prefix_sum_inclusive(count)
    for x in reverse(records): end[x.key]-=1; out[end[x.key]]=x
    return out
```

For [(5,A),(2,B),(4,C),(2,D)],K=6, count=[0,0,2,0,1,1] and end=[0,0,2,2,3,4]. Right-to-left scattering sends D→out[1],C→out[2],B→out[0],A→out[3]. Output [B,D,C,A] preserves B before D. LSD passes require this stability, not an arbitrary swap-based equivalent.

Stability in LSD radix sort

- With base b and d digits, sort stably by each digit from least significant to most significant.
- After pass k, values are ordered by their lowest k digits. Stability in the next pass preserves lower-digit order within equal next digits.
- For `31,14,22,11`, the units pass gives `31,11,22,14`. A stable tens pass gives `11,14,22,31`.
- Stable counting sort costs Θ(n+b) per pass and Θ(d(n+b)) overall. Signed values, variable-length strings, and Unicode need explicit encoding and ordering rules.

Complete LSD driver and digit-order invariant

```
LSD(A,b,d):
    for position=0..d-1:
        A=stable_count_by_digit(A,position,b)
    return A
```

LSD assumes fixed width d and base b≥2, digit=(key//b^position) mod b, and nonnegative or order-preservingly encoded keys.

For 31,14,22,11, the units digits are 1,4,2,1, so the stable first pass gives 31,11,22,14. The tens digits are then 3,1,2,1, giving 11,14,22,31. Stability preserves lower-digit order inside equal next digits, proving the low-k-digit invariant. Arbitrary-precision division would invalidate constant-time digit extraction in Θ(d(n+b)).

Decision-tree lower bound

- For n distinct keys, assume a deterministic algorithm learns order only by comparing two keys. Comparison histories form a binary decision tree.
- All n! relative input permutations need distinguishable leaves. A height-h binary tree has at most 2^h leaves, so $h \geq \lceil \log_2(n!) \rceil$.
- Since $n! \geq (n/2)^{n/2}$, we have $\log_2(n!) = \Omega(n \log n)$. Worst-case comparisons are therefore Ω(n log n).
- The model concerns distinct keys and arbitrary permutations. It neither forbids linear sorting of bounded-domain keys nor requires n log n comparisons on every input.

Preserve qualifications in comparisons

Method	Best case	Worst case	Main auxiliary space	Stable
Insertion	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Yes, move only strictly greater keys
Randomized quicksort	Depends on partitions and duplicates	$\Theta(n^2)$, expected $\Theta(n \log n)$ for distinct keys	Stack can be $O(\log n)$	Generally no
Merge	$\Theta(n \log n)$, standard version here	$\Theta(n \log n)$	$\Theta(n)$	Yes, take left on equality
Heap	Depends on duplicates and implementation	$\Theta(n \log n)$	$O(1)$, iterative version	No
LSD radix	$\Theta(d(n+b))$, version here	Same	$\Theta(n+b)$	Every pass must be stable

Preserve qualifications in comparisons (continued)

Assume constant-time key operations. Incompatible cost models do not establish which method is faster.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mistakes and verification

- Check order, the element multiset, and record stability where promised. is_sorted alone can miss data loss.
- Equal keys, sorted and reverse-sorted inputs, empty and singleton arrays, and many duplicates expose different partition boundaries.
- `examples/sorting.cpp` compares original insertion, merge, and three-way quicksort with standard-sort results on fixed-seed data, including negative values and duplicates.
- Executable cases are not a proof. Preserve partition invariants and recursive termination arguments.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources and extensions

- The MIT 6.006 2008 index, L10–L11, identifies course material on lower bounds and noncomparison sorting.
- Comparator requirements were checked against C++ sorting requirements. An interface requirement does not mandate one fixed implementation algorithm.
- Inversion explanations, partition traces, data, and exercises are original. Introsort is an explicit extension.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are the dictionary ADT, modular arithmetic, expectation, linked lists, and amortized analysis.
- Separate hash functions, collision resolution, and resizing policies. State the assumptions behind each performance claim.
- Trace chaining, linear probing, tombstone deletion, and rehashing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dictionary ADT and collisions

- A dictionary maps each unique key to a value. Operations include find, insert-or-update, and erase.
- A hash function $h : U \to \{0, \ldots, m-1\}$ maps a key universe into m buckets. Collisions are unavoidable when $|U| > m$.
- Equal keys must have equal hashes. The converse fails. Different colliding keys still need equality comparison.
- A hash is not a unique identifier. Comparing hashes without keys turns a collision into a data error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash functions and probability spaces

- The division method is h(k)=k mod m. Keys with matching modular structure may all enter one bucket.
- Randomly selecting a function differs from assuming uniform input. A universal family requires collision probability at most 1/m for every distinct fixed x, y, over the random choice of h.
- If input is fixed before h is selected, universal hashing bounds expected collisions through internal randomness. Adaptive adversaries may require stronger assumptions.
- Cryptographic hashing and ordinary hash tables address different threat models. Output bit length alone does not establish resistance to attacks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate chaining and load factor

- Each bucket stores a list or another collection of keys sharing a hash. Load factor α=n/m can exceed 1.
- With uniform hashing, an independent unsuccessful query has expected bucket length α. For a universal collision bound and a fixed query key outside a fixed stored set, the expectation is at most α. Both give expected O(1+α) search under constant-time key operations.
- Successful-search expectation also depends on insertion order and query distribution. Standard uniform models give O(1+α).
- All n keys can still occupy one bucket, giving Θ(n) worst-case search. An expected guarantee does not eliminate the worst case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked chaining example

- Let m=7, h(k)=k mod 7, and insert `10,17,24,8`.
- Appending within a bucket gives bucket 3 as `[10,17,24]` and bucket 1 as `[8]`. Other buckets are empty.
- Finding 24 scans three keys. Finding absent 31 also scans bucket 3. A small global n/m does not guarantee a short particular search.
- Tree-based buckets can support other requirements or control severe collisions. Their different implementation and costs are not automatic properties of basic chaining.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete chaining dictionary operations

```
find(k):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): return reference(node.value)
    return absent
insert_or_update(k,v):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): node.value=v; return updated
    prepend newly constructed pair(k,v) to bucket[j]; n+=1
erase(k):
    j=hash(k) mod m; predecessor=dummy_before(bucket[j])
    while predecessor.next!=null:
        if equal(predecessor.next.key,k): unlink and release; n-=1; return true
        predecessor=predecessor.next
    return false
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete chaining dictionary operations (continued)

This code prepends, unlike the earlier append-based example. After 10,17,24 enter bucket 3, the chain is [24,17,10]. Find 24 compares once; find 31 compares three times. Erasing 17 leaves [24,10]; reinserting 24 updates its value without increasing n. Both insertion orders implement the same dictionary but differ in individual successful-search costs.

A dummy predecessor may be temporary rather than stored user data. Removing the first node must update the bucket head. Expected chain length depends on the hashing model; collisions can still make worst-case search Θ(n). Skipping duplicate checks changes a unique-key dictionary into multimap semantics.

Whole-table collision pairs versus a fixed query

```
// S is fixed before the random function h is selected.
tau=0
for each unordered pair {x,y} in S:
    tau += indicator(h(x)==h(y))
E[tau] = sum_{unordered {x,y}} Pr(h(x)==h(y))
```

An unsuccessful fixed query compares one key with n stored keys, giving expected chain length at most n/m. Whole-table collision counting has $\binom{n}{2}$ unordered pairs. A universal family bounds each pair's collision probability by $1/m$, hence $\mathbb{E}[\tau] \le n(n-1)/(2m)$. Exact pair probability 1/m gives equality. Both calculations use linearity without independence among all pair events, but n/m is not the table's total collision count.

If $m = \Theta(n^{4/3})$, total expected collisions are $O(n^{2/3})$, while a fixed unsuccessful query has $\mathbb{E}[X] \le n/m = O(n^{-1/3})$, plus the unavoidable bucket-access constant. With m=Θ(n), Θ(n) total expected pairs remain compatible with expected O(1) per query. A bucket of b keys contributes b(b−1)/2 collision pairs, not b−1.

Whole-table collision pairs versus a fixed query (continued)

Worst-case expectation here means $\max_{|S|=n} \mathbb{E}_h[\tau(S,h)]$. Fix any S before choosing h. Universality gives the same bound for each such set. This differs from $\mathbb{E}_h[\max_{|S|=n} \tau(S,h)]$, which allows the input to depend on the observed function. The quantifier order changes the model. For nonnegative τ and $a > 0$, Markov's inequality gives $\Pr(\tau \geq a) \leq \mathbb{E}[\tau]/a$. A small expectation does not guarantee zero collisions for every selected function.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Constructing a universal family and rejecting false randomization

```
// p prime, keys are distinct vectors in F_p^2.
choose a,b independently and uniformly from 0..p-1
h(a,b,(x1,x2))=(a*x1+b*x2) mod p
// A random output translation of fixed g:
g_c(x)=(g(x)+c) mod m
```

For distinct vectors x,y, difference δ=(x1−y1,x2−y2) is nonzero. If δ1≠0, fixing any b leaves exactly one solution a to aδ1+bδ2=0 because nonzero field elements are invertible. Collision probability is exactly 1/p. If δ1=0, then δ2≠0 and solve uniquely for b instead. The proof includes the function a=b=0; one all-colliding function does not invalidate the family's expected guarantee.

For p=11 and keys (0,0),(1,0),(0,1),(1,1), six pairs each collide under exactly 11 of 121 coefficient choices. Total expected collisions are 6/11. Function a=b=0 makes all six pairs collide, an allowed low-probability outcome within the same expectation.

Constructing a universal family and rejecting false randomization (continued)

By contrast, randomly translating a fixed g preserves collisions: g_c(x)=g_c(y) exactly when g(x)=g(y). With g(x1,x2)=x1, keys (3,1),(3,7) always collide, so this is not universal. Reducing a large integer modulo p before hashing can likewise collapse distinct keys prematurely. Encode keys $0..p^2-1$ injectively as two base-p digits rather than discarding the higher digit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

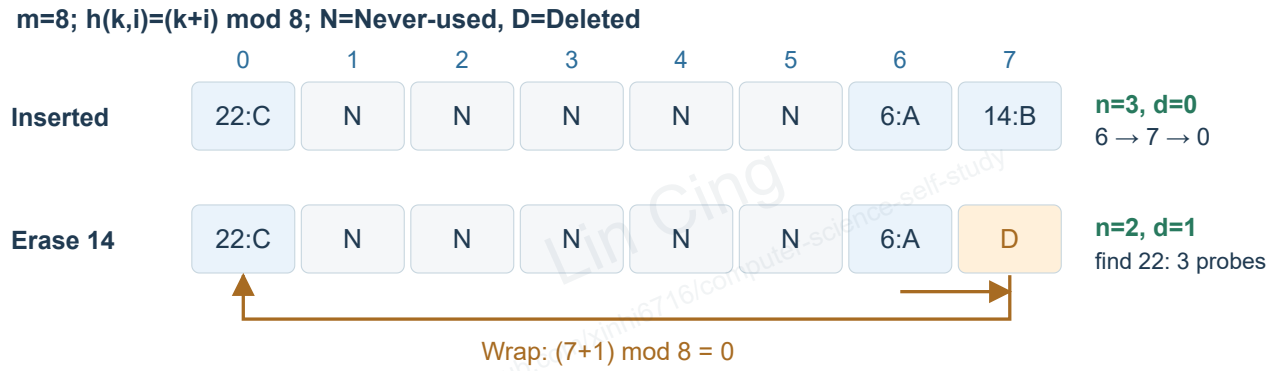
Slot states in open addressing

- Every element resides in the table. A key searches probe sequence `h(k,0),h(k,1),…`. Normally n<m is required.
- Slots are Never-used, Occupied, or Deleted. The three states have different meanings.
- Search may stop at Never-used because insertion could not have passed it to place the same key later. Deleted does not permit stopping.
- Insertion may remember the first Deleted slot but must continue checking for an existing equal key, avoiding duplicate dictionary keys.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear probing: a collision run wraps around

Fix m=8 and $h(k,i) = (k+i) \bmod 8$. Insert (6,A), (14,B), (22,C); every home slot is 6. Let n count live keys and d count Deleted slots. N means Never-used since the latest table initialization.



Deleting 14 changes slot 7 to D. Finding 22 still probes 6,7,0: three probes. Making slot 7 N instead would incorrectly stop after two. A D slot retains no readable old pair; it records that search must continue.

Primary clustering: different homes extend the same run

Start again from the three-key state before deletion, then insert 8 and 1. Cells show keys only; N is Never-used. Neither insertion needs growth.

Action	0	1	2	3	4	5	6	7
Initial	22	N	N	N	N	N	6	14
Insert 8: probe 0→1	22	8	N	N	N	N	6	14
Insert 1: probe 1→2	22	8	1	N	N	N	6	14

Key 8 has home 0 and key 1 has home 1, unlike the original home 6. Both extend the circular run 6→7→0→1→2. A new key with its home inside this run must scan beyond its end. Primary clustering affects more than keys with identical homes. Consecutive probing has locality, but not the probability model of an independent uniform permutation.

The same Deleted slot, two different decisions

After erasing 14, run `insert_or_update(22,C*)`, then `insert_or_update(30,Z)`.

Update 22 → C*

- ① slot 6: key=6, different
- ② slot 7: Deleted; remember 7
- ③ slot 0: key=22 found
- Update 0; leave candidate 7 unused

n=2, d=1; exactly one key 22

Insert 30 → Z

- ① slot 6: key=6, different
- ② slot 7: Deleted; remember 7
- ③ slot 0: key=22, different
- ④ slot 1: Never-used proves absence

Write slot 7; n=3, d=0

The update finds 22 at slot 0 and leaves candidate 7 unused. Inserting 30 reaches N at slot 1 before writing the earliest D at 7. Final slots 0,6,7 contain 22:C*,6:A,30:Z.

Why Never-used stops search but Deleted does not

Invariant: no live key has N before its stored position along its probe sequence. Insertion uses the first N as evidence of absence. Erasing O→D creates no new N, and reusing D changes no N before another live key. These operations preserve the invariant.

Incorrect change	Probe or write outcome	Broken contract
Erase 14 by making slot 7 N	Find 22 stops at 6→7, missing slot 0	Reachability of a live key
Update 22 by writing at the first D	Slots 7 and 0 both contain key 22	Dictionary key uniqueness

An `insert_new` operation may skip duplicate checking if its caller proves absence. Our `insert_or_update` has no such precondition: find the key, reach N, or examine all m slots before deciding. Algorithms Chapter 11 calls this same stopping state EMPTY; it corresponds to Never-used here. Neither course stops at Deleted.

Backshift deletion is a different strategy: moves must respect homes and probe intervals and preserve reachability. It is not an in-place change from tombstone to N.

With no Never-used slot, stop after one cycle

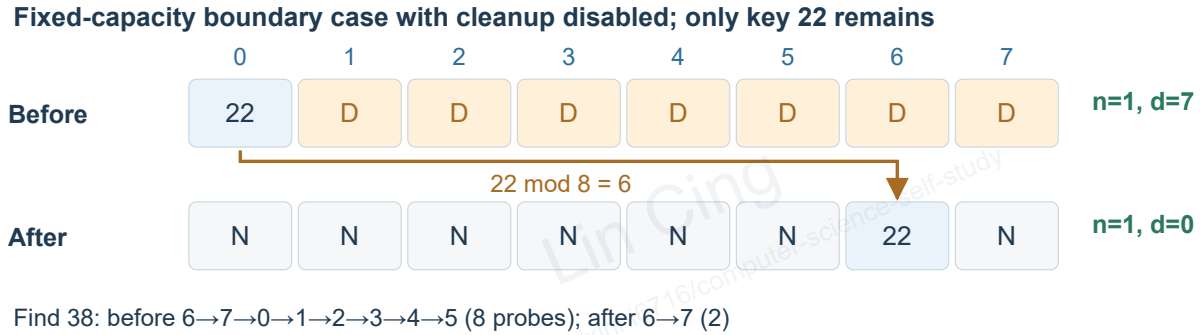
Test the fixed-capacity `locate_or_slot` with automatic resizing and cleanup disabled. Insert 6,14,22,1,2,3,4,5 to fill eight slots, then erase all seven keys other than 22. Now n=1,d=7 and no N remains.

For absent key 38, home=6. Probe 6,7,0,1,2,3,4,5, remembering the first D at 6. After eight nonmatches, return absent(6); never wait indefinitely for N. If all eight slots are Occupied with other keys, return full.

This boundary tests the locator. The later automatic policy cleans earlier and does not retain d=7. **A valid fixed-capacity state need not be a state retained by that maintenance policy.**

Same-capacity rehash: relocate live keys

Keep only key 22 from the fixed-capacity example. Allocate eight N slots and reinsert each live pair using the new table's probe sequence.



Key 22 moves from 0 to 6; n stays 1 and d becomes 0. Finding 38 now probes only 6,7. Merely clearing old D slots to N while keeping 22 at 0 would make search stop at home 6. Cleanup removes markers that searches must cross. Its cost includes scanning and reinserting live keys; the later maintenance thresholds account for that work.

Double hashing and probe coverage

- Double hashing uses $h(k,i)=(h_1(k)+i\cdot h_2(k)) \bmod m$. The step and m must be coprime to cover all m slots.
- With m=8 and step 2, only four slots of the same parity are visited. Insertion can fail despite empty slots elsewhere.
- A prime m with $1\le h_2(k)<m$ ensures coprimality. Power-of-two capacities can use odd steps.
- Quadratic probing needs specified coefficients, capacity, and load restrictions. Writing $h+i^2$ alone does not guarantee reaching every empty slot.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Locate or select a candidate before writing

```
locate_or_slot(key):
    first_deleted=none
    for i=0..m-1:
        j=probe(key,i)
        if state[j]==Occupied and equal(keys[j],key): return found(j)
        if state[j]==Deleted and first_deleted==none: first_deleted=j
        if state[j]==Never_used:
            return absent(first_deleted if first_deleted!=none else j)
    if first_deleted is set: return absent(first_deleted)
    return full
```

Probe at most m slots and require full coverage. Read keys[j] only for Occupied; Deleted owns no live pair. Return found, absent(slot), or full so the caller separates update, insertion, and lack of capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Automatic updates: locate again after rebuilding

```
insert_or_update(k,v):
    result=locate_or_slot(k)
    if result is found(j): values[j]=v; return
    if n+1>3*m/4: rehash(2*m); result=locate_or_slot(k)
    else if deleted>m/8: rehash(m); result=locate_or_slot(k)
    j=result.slot
    if state[j]==Deleted: deleted-=1
    construct pair(k,v) at j; state[j]=Occupied; n+=1

erase(k):
    if locate_or_slot(k) is found(j):
        destroy item; state[j]=Deleted; n-=1; deleted+=1
        if m>minimum_capacity and n<m/4:
            rehash(max(minimum_capacity,m//2))
```

Handle an existing key before checking maintenance thresholds. Rehash can relocate keys, so locate again rather than reuse an old slot index. The eight-slot update/insertion trace has $d=1\le m/8$ and a live load below the growth threshold; it causes no extra rebuild. The seven-slot exercise explicitly fixes capacity and disables this layer.

The model behind expected probe bounds

- Ideal uniform hashing assumes each key's probe permutation is uniform and independent of the prior table history. With $\alpha<1$ and no tombstones, unsuccessful search takes at most $1/(1-\alpha)$ expected probes.
- In the same model, equally likely successful queries among keys inserted without deletion have bound $(1/\alpha)\ln(1/(1-\alpha))$ for $\alpha>0$.
- At $\alpha=0.9$ the unsuccessful bound is 10. It depends on load and diverges as α approaches 1.
- These are not exact linear-probing formulas. They omit tombstones, hash computation, and long-key equality costs.

Deriving ideal probe upper bounds

- Assume m−n Never-used slots, no tombstones, and a uniform random probe permutation independent of the occupied set. The first i probes all hit occupied slots with probability $(n)_i/(m)_i \le \alpha^i$, using falling factorials.
- For probe count X, the tail-sum identity gives $\mathbb{E}[X] = \sum_{i\ge 0} \Pr(X > i) \le \sum_{i\ge 0} \alpha^i = 1/(1-\alpha)$. Both α<1 and absence of tombstones enter this proof.
- For equally likely successful queries among n inserted, undeleted keys, average over insertion loads j/m. Since 1/(1−x) increases, its left rectangle sum is bounded by its integral: $\mathbb{E}[X] \le \frac{1}{n}\sum_{j=0}^{n-1} \frac{1}{1-j/m} \le \frac{1}{\alpha}\ln\frac{1}{1-\alpha}$.
- This derives the bound from the probability model. Linear probing's contiguous clusters do not satisfy the random-permutation premise.

Resizing must recompute placement

- Changing capacity changes the key-to-slot mapping. Reinsert existing keys rather than merely copying old slots into a larger array.
- Doubling at a fixed load threshold scans and allocates Θ(n) storage when m=Θ(n). Reinsertion also has expected Θ(n) cost under suitable random assumptions and a load bounded away from 1. It need not be worst-case linear under arbitrary collisions.
- These conditions give expected amortized O(1) insertion. The claim combines probability assumptions with accounting across a sequence. It is not worst-case constant time per operation.
- Equal growth and shrink thresholds can trigger repeated rebuilding under alternating updates. Distinct thresholds avoid this oscillation.

Rehash relocation and expected analysis

```
rehash(new_m):
    allocate new_m Never_used slots
    choose compatible new hash parameters if required
    for each old Occupied pair (k,v):
        insert into new table using its new probe sequence
    commit new table; deleted=0; preserve n; release old table
```

For keys 10,17,24,8 with m=7, buckets are 3:[10,17,24],1:[8]. At m=14 using k mod14, buckets become 10:[10,24],3:[17],8:[8]. Old positions cannot remain unchanged. Open addressing likewise reinserts using new probe sequences and removes tombstones.

For a fixed unsuccessful query key $x \notin S$ and a fixed stored set S with $|S| = n$, define

$$X = \sum_{y \in S} \mathbf{1}[h(y) = h(x)].$$

Linearity of expectation and universality give

$$\mathbb{E}[X] = \sum_{y \in S} \Pr(h(y) = h(x)) \le n/m.$$

Rehash relocation and expected analysis (continued)

This calculation does not need independence among collision events, but both x and S must be fixed before the function is selected. Adaptive attacks are a different model. If $x \in S$, the self-collision contributes exactly 1, giving the bucket-length bound $1 + (n-1)/m$. The unsuccessful-query bound cannot be reused in that case. A successful-query distribution concentrated on the longest chain cannot inherit the average for uniform queries.

Scanning costs Θ(m+n). When m=Θ(n), bounded load and suitable randomness make reinsertion expected Θ(n). Colliding insertions with duplicate checks may take quadratic time, so worst-case linear rehash is not unconditional. Separate 3/4 growth and 1/4 shrink thresholds plus tombstone cleanup support the combined expected-amortized update guarantee.

Cleanup-threshold counterexample and amortized accounting

```
// m starts at a power-of-two minimum_capacity>=8.
// Before inserting a genuinely absent key:
if n+1>3*m/4: rehash(2*m)
else if deleted>m/8: rehash(m)
// After deleting a present key:
if m>minimum_capacity and n<m/4:
    rehash(max(minimum_capacity,m//2))
// Rehash resets deleted=0 and reinserts all n live entries.
```

Resizing and tombstone cleanup need different triggers. The rule "rebuild at the same capacity if n+deleted+1>3m/4" fails near n=3m/4. Delete one key, giving n=3m/4−1 and deleted=1; reinserting it could reuse that tombstone, but the rule rebuilds immediately. With m=128 and 96 keys, 500 repetitions of deleting and reinserting key 95 cause 500 rebuilds and 47,500 record reinsertions. Constantly many operations cannot pay Θ(m) per repetition.

Cleanup-threshold counterexample and amortized accounting (continued)

The corrected rule cleans at the same capacity only after deleted>m/8, requiring Ω(m) deletions since the previous rebuild. Lookup precedes resizing, so updating an existing value never resizes. A genuinely new key first tests live load 3/4, then the deleted threshold. After every insertion, n+deleted≤7m/8: without rebuilding, n≤3m/4 and deleted≤m/8; rebuilding clears tombstones; reusing one does not increase used slots. Deletion only converts Occupied to Deleted, preserving that sum. Consequently at least m/8 Never-used slots remain even after subsequent deletions.

Growth leaves live load about 3/8, while shrinking leaves it at most 1/2. Reaching the opposite resize threshold requires Ω(m) live-count changes, preventing one-delete/one-insert oscillation. Charge same-capacity cleanup separately to its accumulated deletions. Under the stated uniform-probing or compatible random model, each rebuild costs expected Θ(m+n), giving expected amortized O(1) probe work over updates. Worst-case collisions still allow linear operations or quadratic reinsertion. Odd double-hash steps cover power-of-two capacities, but coverage alone does not establish the ideal random-permutation model.

Overflow and engineering conditions

- In chaining, overflow means longer bucket contents. A full open-addressed table must be detected rather than probed forever.
- Hashing an L-symbol string can cost $\Theta(L)$. Probe count alone omits key-processing time.
- An attacker controlling many colliding keys can invalidate distribution assumptions. Possible responses include random seeds, suitable keyed hashing, or another container.
- Sorted iteration, range queries, and predecessors are more direct with balanced search trees, since hash tables do not preserve key order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and further reading

- The MIT 6.006 2008 index, L5–L7, identifies hashing course material.
- C++ unordered associative requirements distinguish average from worst-case costs and require compatible hash and equality functions. Average guarantees do not mean worst-case constant time for every call.
- MIT 6.006 2014 Recitation 10, § 1.1: Deleted search semantics and duplicate-key updates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are binary heaps, linked trees, induction, logarithms, and amortized analysis.
- Derive structures from meld requirements. Trace leftist, binomial, Fibonacci, and standard two-pass pairing heaps.
- Distinguish actual work, worst-case bounds, and amortized bounds. State whether decrease-key already has a valid node handle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mergeable priority queue contract

- A heap represents a multiset, allowing duplicate keys. Meld combines all elements of two heaps while preserving element identities.
- We use destructive meld. Source heaps no longer own transferred nodes afterward. Keeping independently mutable source copies has different costs.
- Decrease-key uses a valid node handle and lowers that node's key. Searching for an arbitrary key cannot be hidden inside an O(1) or O(log n) bound.
- With tied minima, any minimum may be returned. Stable order requires a composite key such as key plus insertion sequence number.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Leftist heaps: make the path used by meld short

Concatenating two binary-heap arrays and rebuilding takes linear time. A leftist heap uses a linked binary tree: **the whole tree need not be shallow; following right pointers must reach null quickly.** Meld descends through one right child at each level, so that is the path we constrain.

Set null rank to zero. For a nonempty node,

$$r(x) = 1 + \min\{r(x.left), r(x.right)\}.$$

A leaf has rank 1. A node with only a left child also has rank 1. Rank measures the shortest path to a null child position, **not height or child count**.

Maintain min-heap order and $r(left) \geq r(right)$. Thus $r(x) = 1 + r(x.right)$: rank counts nonempty nodes on the right spine. An n-node left-only chain is valid, with rank 1 and height n−1.

Meld: keep the smaller root, repair on return

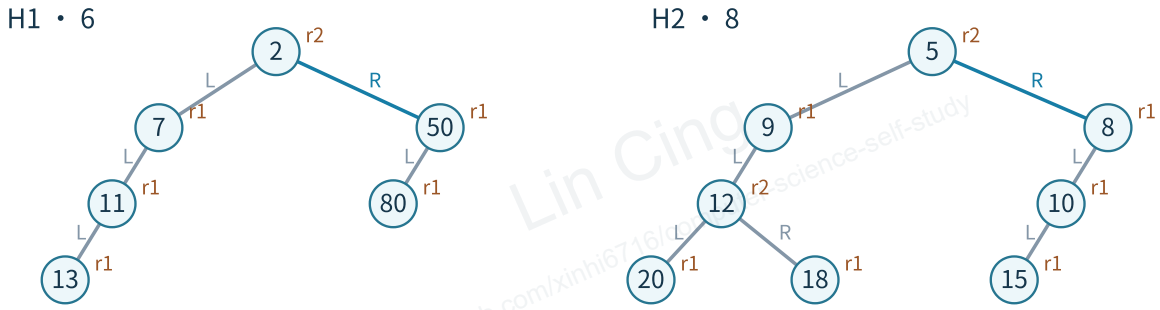
Inputs must be valid heaps with disjoint node sets. This destructive meld keeps the first root on a tie. Nodes store key, left, right and rank, with no parent pointers.

```
rank(null) = 0
meld(a, b):
    if a == null: return b
    if b == null: return a
    if a.key > b.key: swap(a, b)
    a.right = meld(a.right, b)
    if rank(a.left) < rank(a.right):
        swap(a.left, a.right)
    a.rank = 1 + rank(a.right)
    return a
```

Compare keys on descent; compare ranks on return. Keep a's left subtree and recursively meld its right subtree with the other heap. The new right rank is known only after that call returns. Swap pointers, never keys or payloads.

Meld inputs: 14 nodes and two right spines

Circles contain keys; r labels give ranks. L and R identify child pointers. An omitted edge is null, with rank 0. Blue edges mark the original right spines; other edges are not traversed by this recursive descent.



H1 has 6 nodes and spine 2→50. H2 has 8 nodes and spine 5→8. Both roots have rank 2. Node 9 has a deep left subtree but rank 1, because its right pointer is null.

Recursive descent: do not swap children yet

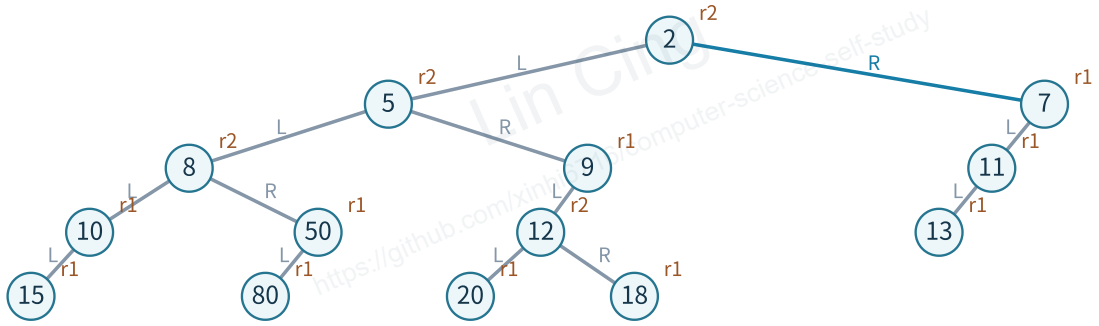
Keep the smaller root and recurse through its right child. Retained-root ranks have not yet received their return-time updates.

Call	Comparison and retained root	Left subtree kept	Next call
meld(2,5)	2<5: keep 2	Root 7	meld(50,5)
meld(50,5)	5<50: exchange local roots, keep 5	Root 9	meld(8,50)
meld(8,50)	8<50: keep 8	Root 10	meld(null,50)
meld(null,50)	Empty base case; no key comparison	—	Return root 50

There are **three root-key comparisons**. Calls at 2,5,8 remain pending. A temporarily assembled min-heap is not yet a repaired leftist heap. Unwind in order 8→5→2.

Unwinding: why roots 5 and 2 swap children

Returning root	Attached left/right roots	Left/right ranks	Action and new rank
8	10 / 50	1 / 1	No swap; r(8)=2
5	9 / 8	1 / 2	Swap: left 8, right 9; r(5)=2
2	7 / 5	1 / 2	Swap: left 5, right 7; r(2)=2



Why meld is correct

Induct on the total input size. If one heap is empty, returning the other preserves the node set and both invariants. Otherwise select the smaller root a. Recursion processes only a's right subtree and the other heap, strictly reducing total size.

1. **No lost or duplicated nodes.** Disjoint inputs partition into a, its original left subtree and the recursive inputs. Induction preserves recursive nodes; attachment and child swaps create or delete none.
2. **Heap order.** Root a is no greater than any key in its own subtrees or the other heap. It is no greater than the new child roots. The child heaps are valid by induction; exchanging their sides preserves these comparisons.
3. **Leftist property.** Both children are valid heaps. Put the higher-rank child on the left. Assigning $r(a) = 1 + r(a.right)$ now agrees with the defining minimum.

Strictly decreasing size also proves termination. meld(H,H) violates disjointness and may create a cycle. The ownership interface must exclude it; this operation does not copy the heap.

Worst-case cost: follow the original right spines

First bound rank. A subtree of rank at least r has two child subtrees of rank at least r−1. If N(r) is the minimum size for rank at least r, N(0)=0 and induction gives

$$N(r) \geq 1 + 2N(r-1) \geq 1 + 2(2^{r-1} - 1) = 2^r - 1.$$

Hence an n-node heap has $r \leq \lfloor \log_2(n+1) \rfloor$. The base case r=0 holds.

During descent no return-time swaps have occurred. Each step replaces one root by its **original right child**, reducing the two current ranks' sum by exactly one. When both heaps are nonempty that sum is at least two. Input ranks r1,r2 therefore permit at most r1+r2−1 nonempty levels, plus one empty base case.

Constant work per level gives **worst-case** time and stack space $O(1 + \log(n_1 + 1) + \log(n_2 + 1))$. The 14-node example has rank sum four and exactly three comparisons; other inputs can finish earlier.

Delete-min: meld the two child heaps

Remove root 2 from the 14-node result and meld roots 5 and 7. Reject deletion on an empty heap. Removing a singleton returns null.

Call or return	Decision
meld(5,7)	Keep 5 and left subtree 8; recurse on meld(9,7)
meld(9,7)	Keep 7 and left subtree 11; recurse on meld(null,9)
meld(null,9)	Return 9 with all its descendants
Return to 7	Children 11,9 both have rank 1; no swap; r(7)=2
Return to 5	Children 8,7 both have rank 2; no swap; r(5)=3

Result: `5:(8,7)`, `8:(10,50)`, `7:(11,9)`; all other edges remain. The right spine is 5→7→9, minimum is 5, and size is 13. **Rank can increase after deletion**: it measures shape and is not monotone in element count.

Insertion and rank conventions

Insert creates a singleton with `left=right=null, rank=1` and melds it with the heap. Insert key 1 into the preceding 13-node heap: keep 1, and meld(null,5) returns the old heap. The new root has left rank 0 and right rank 3. Swap children, placing the old heap on the left. The new root has rank 1. This uses two calls and one comparison, without searching through the old nodes.

Insert and delete-min take worst-case $O(1 + \log(n + 1))$; find-min reads a nonempty root in O(1). Allocation and arbitrary-precision key costs are excluded.

Here **null rank=0 and leaf rank=1**. Under null npl=−1 and leaf npl=0, the same position has $npl = r - 1$. Its right spine has npl+1 nonempty nodes; its subtree has at least $2^{npl+1} - 1$ nodes. Convert formulas and base cases together.

This differs from binomial/Fibonacci degree (direct child count) or $\log_2 size$ in splay analysis. These are different quantities, not universally convertible by shifting one. The right-spine bound does not bound overall height.

Binomial trees B_k

- B_0 is a singleton. Form B_k from two B_{k-1} trees by making one root a child of the other.
- Induction gives 2^k nodes, height k, root degree k, and child subtrees B_0 through B_{k-1}.
- A binomial heap stores heap-ordered binomial trees with at most one root of each degree.
- The binary expansion of n determines the degrees. For n=13=1101_2, the forest contains B_0, B_2, B_3.

Binomial meld as binary carrying

- Merge root lists by degree. For two equal-degree trees, compare roots and link the larger root below the smaller.
- Two B_k trees become B_{k+1}, like a binary carry. With three equal-degree trees, retain one and link the other two.
- At most $\lfloor \log_2 n \rfloor + 1$ roots exist. Standard eager consolidation meld takes worst-case O(log n).
- Melding sizes $5=101_2$ and $3=011_2$ carries through degrees 0,1,2, leaving one B_3 with eight nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial linking and dedicated insertion

```
binomial_link(a,b):
    require degree(a)==degree(b)
    if a.key>b.key: swap(a,b)
    prepend b to a.child list; b.parent=a
    a.degree+=1; return a
binomial_insert(x):
    carry=singleton(x); d=0
    while tree[d] exists: carry=link(carry,tree[d]); clear tree[d]; d+=1
    tree[d]=carry; update min pointer
```

`swap(a,b)` exchanges local root pointers, not node keys or payloads. Both inputs are Bk trees. The winner has child degrees k−1,…,0; prepending the losing Bk gives k,k−1,…,0, exactly the children of B(k+1). Keeping the smaller root preserves heap order on the new edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

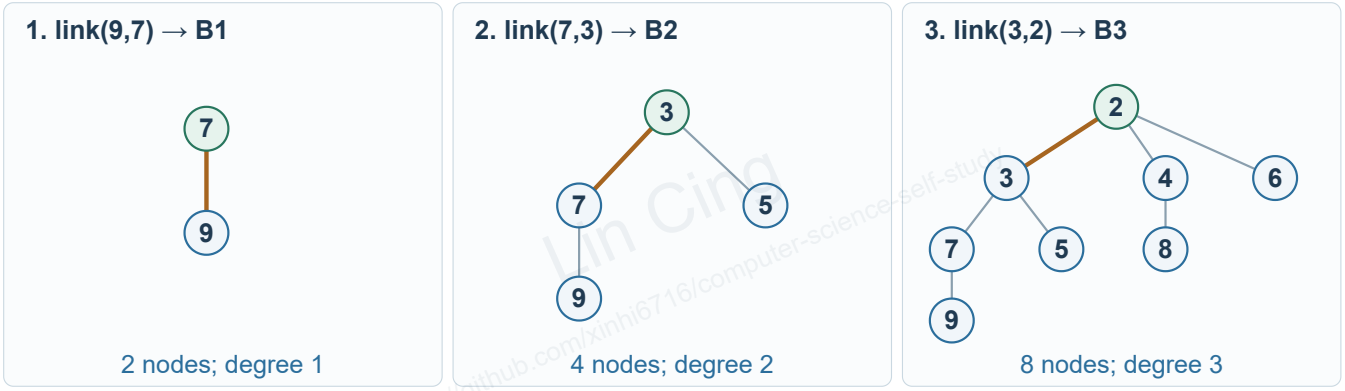
Binomial meld: input forests and list order

All keys are distinct. Notation `x:[child roots]` gives child-list order; unlisted nodes are leaves. H1 is `9:[], 2:[4,6], 4:[8]`; H2 is `7:[], 3:[5]`. Root lists use increasing degree; child lists use decreasing degree. A link makes the losing root the winner's **first child**.

H1 has 1+4=5 nodes; H2 has 1+2=3. The first root need not be the minimum: H1's minimum is 2, not 9.

Binomial meld: three comparisons and subtree links

Link B0 roots 9 and 7, then link with B1 root 3, then with B2 root 2. Each step compares only two roots and moves the entire losing subtree.



Brown marks the new parent→child edge; all other edges remain. Root 2 ends with children 3,4,6 of degrees 2,1,0. Each of the eight keys occurs once; no payload is exchanged or losing subtree discarded.

Binomial meld: what remains at each degree?

Step	Link	Remaining roots: key(degree)	Roots
Input	None	H1: 9(0),2(2); H2: 7(0),3(1)	4
degree 0	9 and 7 → 7	7(1),3(1),2(2)	3
degree 1	7 and 3 → 3	3(2),2(2)	2
degree 2	3 and 2 → 2	2(3)	1

One link removes one tree, not a node. Four roots become one in exactly three links. Linking equal B_k trees preserves the binomial shape and gives $2^k + 2^k = 2^{k+1}$ nodes. Keeping the smaller root preserves heap order on new and retained edges. The final degree 3 also matches $5+3=8=1000_2$.

Three links in this example do not establish O(1) general meld. The algorithm also visits input root lists, whose size is O(log n) for valid binomial heaps. A Fibonacci heap may have Θ(n) unconsolidated roots, so this root-list bound does not transfer.

Why not link three equal-degree trees into one?

For $7=111_2$ plus $3=011_2$, each degree receives at most two input roots and one carry.

degree	H1 roots	H2 roots	Incoming carry	Retained here	Outgoing carry
0	1	1	0	0	1
1	1	1	1	1	1
2	1	0	1	0	1
3	0	0	1	1	0

Three B1 trees have six nodes: retain one B1 and link the other two into B2. Six is not a power of two, so all three cannot form one binomial tree. The final B1 and B3 contain 2+8=10 nodes, matching 1010_2. Choosing a different retained tree can change the final shape; degree, key conservation, and heap order must still hold.

Conditions for other binomial operations

- A dedicated insertion adds B_0 and carries from low degrees, stopping as soon as carrying ends. Its worst case is O(log n), but carry accounting gives amortized O(1). Calling a meld that always scans the entire root list does not automatically obtain this insertion bound.
- Delete-min finds and removes the minimum root, reverses its child list into increasing degree order, then melds. Worst-case time is O(log n).
- Find-min costs O(log n) if it scans roots. A maintained minimum-root pointer makes it O(1).
- Decrease-key can exchange contents up the parent chain in O(log n). Node-identity handles require compatible handling of those content exchanges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial meld: carry through a degree table

```
meld_roots(R1,R2):
    by_degree=empty
    for x in concatenation of detached root snapshots R1,R2:
        while by_degree[x.degree] contains y:
            clear by_degree[x.degree]
            x=binomial_link(x,y)
        by_degree[x.degree]=x
    return roots in degree order; recompute minimum pointer
```

Each snapshot root enters once. On collision, clear the old degree slot, link, and retry at the new degree. Each link removes one root, so there are at most one fewer links than input roots. Valid input forests have O(log(n1+n2)) roots, giving O(log(n1+n2)) total visits and links. This root bound does not apply to unconsolidated Fibonacci heaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

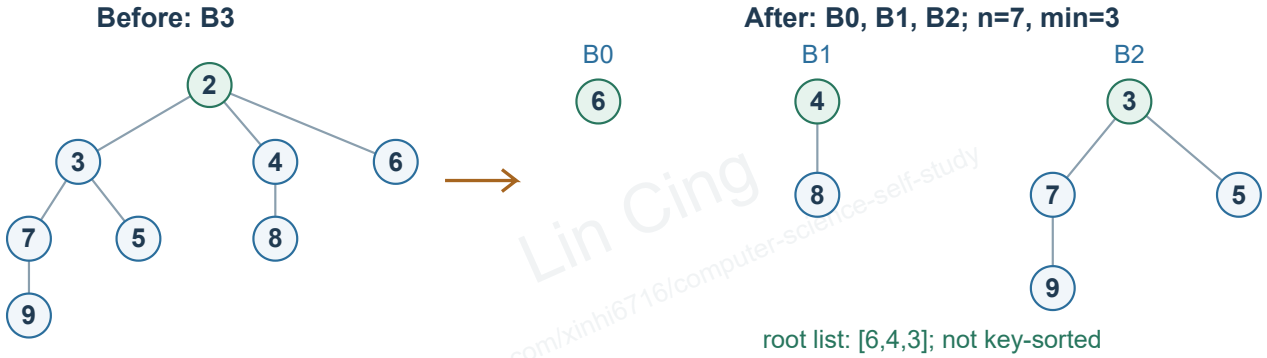
Binomial removal and decrease: restore parent and min

```
delete_min():
    require roots not empty
    z=minimum root
    remove z from roots
    children=detach z.children; clear their parent pointers
    release z
    roots=meld_roots(roots,children)
decrease_key(x,new):
    require new<=x.key; x.key=new
    while x.parent!=null and x.key<x.parent.key:
        swap complete key-value contents and update handle mapping
        x=x.parent
    update minimum pointer
```

Detach children and clear their parents before releasing the old root; meld recomputes min. Decrease-key may swap complete key-value contents, but logical-element handles require the handle-to-node mapping to follow those contents.

Delete-min: turn the same B3 back into a forest

Remove 2 and keep its three subtrees. Reverse child degrees 2→1→0 into root degrees 0→1→2; clear each new root's parent.



Root list [6,4,3] holds 1+2+4=7 nodes and min=3. No other roots remain, so no link is needed. **Reverse by degree, not key.** A degree-table variant can accept unreversed input.

Fibonacci heaps defer consolidation

- A Fibonacci heap has a heap-ordered forest, circular doubly linked root list, minimum-root pointer, and degree and mark fields per node.
- Insert adds a singleton root. Meld splices root lists and updates the minimum pointer. Neither immediately consolidates equal degrees.
- Delete-min promotes the removed root's children, links equal-degree roots using a degree table, then rebuilds the minimum pointer.
- Nodes can lose children, so trees do not retain fixed binomial shapes. Cascading cuts are needed for the degree bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decrease-key and cascading cuts

1. Lower x.key. If it remains at least its parent's key, no cut is needed.
2. Otherwise cut x from parent p, add x to the root list, and clear x.mark.
3. If p is not a root, mark it on its first lost child. If already marked, cut p and continue upward.

A nonroot node can lose at most one child after its latest attachment and still remain under that parent. Roots need not be marked on losing children.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci consolidation and mark operations

```
link_equal_degree_roots(a,b):
    require a.degree==b.degree
    if a.key>b.key: swap(a,b)
    roots=ring_remove(roots,b)
    a.child=ring_prepend(a.child,b)
    b.parent=a; b.mark=false; a.degree+=1
    return a
cut(x,p):
    p.child=ring_remove(p.child,x); p.degree-=1
    x.parent=null; x.mark=false; roots=ring_prepend(roots,x)
cascade(p):
    g=p.parent
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci consolidation and mark operations (continued)

```
    if g==null: return
    if not p.mark: p.mark=true
    else: cut(p,g); cascade(g)
decrease_key(x,new):
    require new<=x.key; x.key=new
    p=x.parent
    if p!=null and x.key<p.key: cut(x,p); cascade(p)
    if x.key<min.key: min=x
delete_min():
    require min!=null
    return extract_root(min)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Consolidating singleton roots [7,3,8,5] in that order puts 7 at degree 0. Root 3 links 7 below it to form degree 1. Root 8 occupies degree 0; root 5 links 8 below it, then collides with degree 1 root 3. Root 3 wins, with child 7 and child 5→8, degree 2. Iterate a snapshot or use mutation-safe root traversal while linking.

Fibonacci consolidation and mark operations (continued)

For marked nonroots p and g in a valid state, illustrate r=2→g=3→p=5→x=7 with g and p marked. Decreasing x to 1 cuts x, p, g into roots. Root count changes 1→4 and marked count 2→0, so Φ changes 5→4, ΔΦ=−1. Scaling potential pays fixed cut costs with constant endpoints left over. This state illustrates accounting only; the general degree bound still requires the attachment-time and one-lost-child proof.

Insert adds an unmarked singleton root and updates min. Destructive meld splices root cycles and transfers ownership. Empty delete-min rejects, and removal of the last node sets min=null. Arbitrary-node deletion uses the complete erase(handle) and extract_root procedures below without changing keys. A forced cut notifies the old parent through cascade even when heap order was not violated.

Complete circular-list operations and shared consolidation

```
ring_remove(head,x):
    if x.right==x: head=null
    else:
        x.left.right=x.right; x.right.left=x.left
        if head==x: head=x.right
    x.left=x.right=x; return head
ring_prepend(head,x):
    require x.left==x and x.right==x
    if head!=null:
        x.left=head.left; x.right=head
        head.left.right=x; head.left=x
    return x
snapshot_of_root_ring():
    S=[]
    if roots==null: return S
    x=roots
    repeat: S.append(x); x=x.right
    until x==roots
```

Complete circular-list operations and shared consolidation (continued)

```
    return S
consolidate():
    A=[]                        // missing degree entries mean null
    for x in snapshot_of_root_ring():
        if x.parent!=null: continue
        while len(A)<=x.degree: A.append(null)
        while A[x.degree]!=null:
            y=A[x.degree]; A[x.degree]=null
            x=link_equal_degree_roots(x,y)
            while len(A)<=x.degree: A.append(null)
        A[x.degree]=x
    roots=null; min=null
    for x in nonnull entries of A:
        x.left=x.right=x; x.parent=null; x.mark=false
        roots=ring_prepend(roots,x)
        if min==null or x.key<min.key: min=x
```


Complete circular-list operations and shared consolidation (continued)

Ring_remove requires x to belong to the designated ring. A singleton yields null. Otherwise reconnect both neighbors and move the representative if necessary. Make the removed x a singleton before passing it to ring_prepend. Both operations change constantly many pointers without a predecessor search. Using them for root and child rings makes cut, promotion, and link explicit.

Snapshot root pointers before linking modifies the ring. Equal-degree roots compare keys; remove the larger root from roots, attach it to the smaller root's child ring, clear its mark, and increase the parent's degree, using link_equal_degree_roots from the earlier page. Clear the old table entry before carrying to the new degree. Each surviving table entry represents one valid heap-ordered tree.

Complete circular-list operations and shared consolidation (continued)

Rebuild roots and min from the table without traversing the discarded old ring. Never read old min.key after releasing its node, or skip comparisons merely because z was not the old minimum. Grow the degree table as needed, using O(1+log(n+1)) entries. With t′ roots after cutting and target degree d, this snapshot version additionally stores O(t′+d) root pointers, worst-case O(n); auxiliary space is not just the degree table. One operation may also take Θ(n) actual time. Amortized time and instantaneous space are different claims.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why Fibonacci degrees stay logarithmic

- Order x's k retained children by attachment time. When child i was linked, x already had at least i−1 earlier retained children. Equal-degree linking means the new child then had degree at least i−1.
- A child retained under x can have lost at most one child. Its current degree is at least max(0, i−2).
- A minimum-subtree-size recurrence then gives at least F_{k+2} nodes for degree k, hence degree O(log n).
- The bound connects permitted child losses to degree. Omitting cascading cuts invalidates this argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving the Fibonacci size bound

- Let S(k) be the minimum possible subtree size among nodes of degree at least k, including the root. This definition is nondecreasing in k. S(0)=1 and S(1)≥2.
- Any node of degree at least k has k children with degree lower bounds max(0, i−2). Thus for k≥2, $S(k) \geq 1 + 2S(0) + \sum_{j=1}^{k-2} S(j)$.
- Inductively substitute $S(j) \geq F_{j+2}$ to obtain $S(k) \geq 3 + \sum_{j=1}^{k-2} F_{j+2} = F_{k+2}$. The last identity is a Fibonacci prefix sum, with F_0=0, F_1=1.
- Since $F_{k+2} \geq \varphi^k$ for $\varphi = (1 + \sqrt{5})/2 > 1$, every degree in an n-node heap satisfies $k \leq \log_\varphi n$. This justifies an O(log n) degree table.

Potential accounts for deferred work

- Use $\Phi = t + 2m$, where t counts roots and m counts marked nonroots. Amortized cost is $\hat{c} = c + \Phi_{\text{after}} - \Phi_{\text{before}}$.
- Insert increases roots by one and potential by a constant. Meld combines additive potentials and takes constant amortized work.
- Cutting a marked ancestor adds one root but clears one mark, reducing potential by one. Repeated cuts release potential to pay their actual work, apart from constant endpoint changes.
- Multiplying Φ by a sufficient fixed implementation constant pays for the constant pointer work per cut or link. Starting empty, nonnegative potential makes total actual work at most total amortized work plus initial potential. This is a worst-sequence guarantee, not a random average.

One Fibonacci potential and complete amortized bounds

For Fibonacci decrease-key with k≥1 cuts, root count rises by k. Beyond the first cut, each cut ancestor was marked; at most the final stopping ancestor acquires a new mark. Thus $\Delta m \le 1-(k-1)=2-k$ and $\Delta(t+2m)\le 4-k$. If actual work is at most ak+b, choose scaled potential $A(t+2m),\ A\ge a$. Amortized cost is at most $ak+b+A(4-k)\le b+4A$, independent of k. With no cut, key/minimum updates and comparisons are constant work.

For delete-min, let the initial root count be t, removed degree d, and maximum degree D. Actual work is at most a(t+d+D)+b. Promotion, linking, and clearing root marks do not increase the marked count. Consolidation leaves at most D+1 roots, so $\Delta\Phi \le A(D+1-t)$. Choosing A≥a yields amortized cost at most $a(d+D)+A(D+1)+b=O(D+1)=O(1+\log(n+1))$, since d≤D. This is the algebra paying for root scans; one actual operation may still cost Θ(n).

- Fibonacci heaps yield Dijkstra's abstract O(E+V log V) bound with valid handles and decrease-key, under Dijkstra's nonnegative-weight condition.

Erase without changing a key and extraction of an arbitrary root

```
erase(h):
    z=resolve_live_owned_handle(h)   // reject stale/foreign handle first
    if z.parent!=null:
        p=z.parent; cut(z,p); cascade(p)
    return extract_root(z)
extract_root(z):
    require z is a live root owned by this heap
    result=copy(z.item)             // fixed-size nonthrowing model
    while z.child!=null:
        c=z.child
        z.child=ring_remove(z.child,c); z.degree-=1
        c.parent=null; c.mark=false; roots=ring_prepend(roots,c)
    roots=ring_remove(roots,z); min=null
    invalidate_handle(z); release(z); n-=1
    if n==0: roots=null; return result
    consolidate()
    return result
```

Erase without changing a key and extraction of an arbitrary root (continued)

Each node stores key, item, parent, child, degree, mark, and circular-list left/right pointers. Roots and child designate root and child rings; null denotes an empty ring. A handle identifies a particular live node, not an arbitrary node with an equal key. Only the erased handle becomes invalid; other items and node identities are never swapped. Resolve an opaque external handle through its owning container's live-node registry before accessing a node. Reading an already released raw pointer to inspect an alive flag is invalid. Pointer-work bounds assume a valid handle; additional lookup/validation costs are separate.

For nonroot z, cut it into the root ring and notify its old parent p through cascade. An unmarked nonroot p becomes marked; a marked p is cut and the process continues upward; a root stops the cascade. This cut is required for deletion even without a key decrease. Omitting cascade would allow a nonroot to lose two children and invalidate the degree proof. A target already at a root enters extract_root directly.

Erase without changing a key and extraction of an arbitrary root (continued)

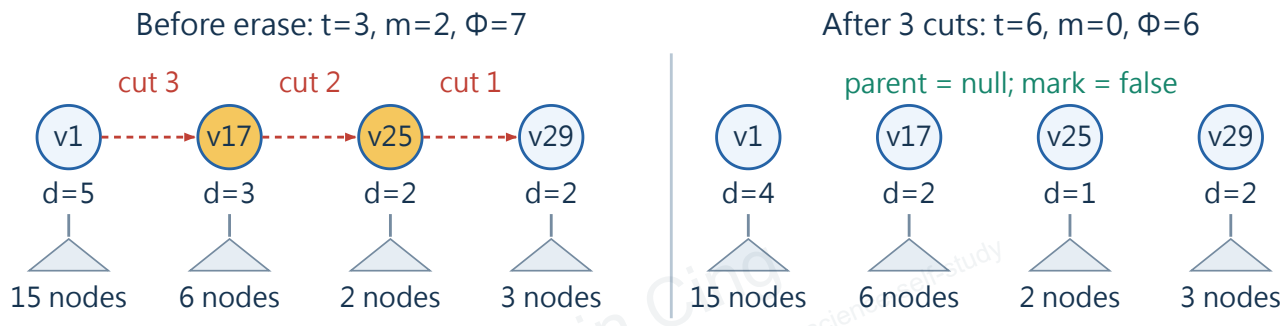
Extract_root does not require z=min. Remove each child c from z's ring before decrementing degree, clearing parent/mark, and adding its entire subtree to roots. Remove z, clear min to avoid a dangling pointer, invalidate its handle, release it, and decrement n exactly once. If nodes remain, consolidate all roots and recompute the actual minimum. No key is changed. Reject empty delete-min before access; an empty heap has no valid erase handle. Removing the last node gives n=0 and roots=min=null. Reusing that handle must fail. After destructive meld, accepted handles follow the container's ownership contract; the emptied source must not erase destination-owned nodes.

Legal erase setup and three cuts

```
for i=0..32: h[i]=insert(i)     // v_i is permanent identity, initially key i
delete_min()                    // removes v_0; consolidates to root v_1
decrease_key(h[18],-10)
decrease_key(h[26],-11)
erase(h[29])                    // key stays 29 until release
```

Prepend to root and child rings and traverse snapshots through right links. After inserting 0..32 and removing v0, root v1 has degree 5. Its child v17 originally has children v25,v21,v19,v18; v25 has v29,v27,v26; v29 has v31,v30; v31 has v32. Lowering v18 to −10 cuts it and marks v17. Lowering v26 to −11 cuts it and marks v25. Before erase, roots are v26(key−11,d0),v18(key−10,d0),v1(key1,d5), with marked nonroots v17,v25. Target v29 is not the minimum and still has two child subtrees.

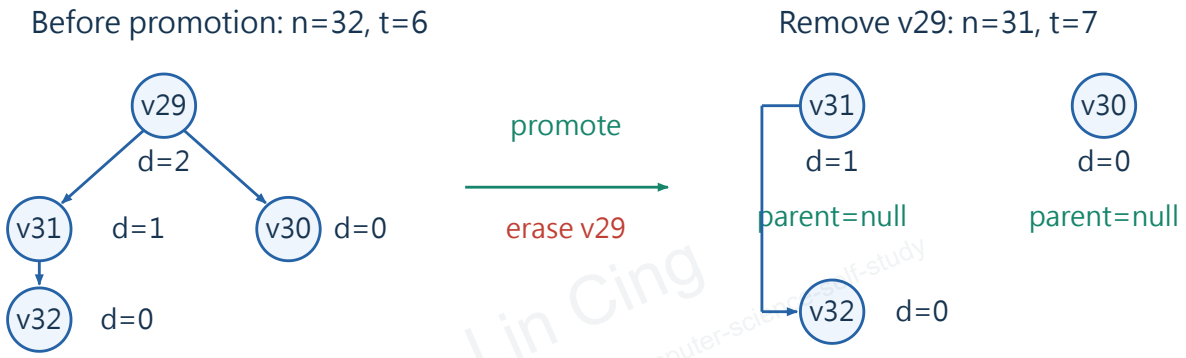
Legal erase setup and three cuts (continued)



Other roots: v18(key −10), v26(key −11). Gold = mark true.
Triangles retain child forests; counts exclude the parent. Both states have n=32.

The four displayed nodes, unchanged child forests of sizes 15,6,2,3, and two other roots account for all 32 nodes. Cutting changes parent links, not node identities. Each already marked ancestor becomes an unmarked root; root v1 is not marked when the cascade stops.

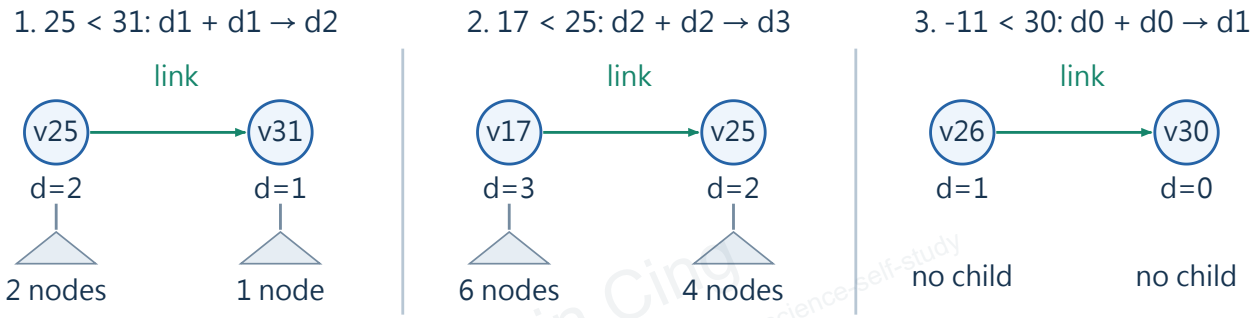
Promote target children and remove exactly one node



Unchanged roots: v1, v17, v25, v26, v18 (28 nodes); only h29 expires.

Moving v31 preserves its child v32, so it remains a two-node tree. Together with singleton v30, the promoted trees contain three nodes. Only v29 is removed; v26 with key −11 remains the minimum, and all surviving handles still identify their original nodes.

Three degree-table carry links



A[0]=v18, A[1]=v26, A[3]=v17, A[4]=v1; min=v26 (key −11)
Subtree sizes: 1 + 2 + 12 + 16 = 31; t=4, m=0, Φ=4.

The first link makes v25 a five-node tree of degree 2; the second attaches it to v17, producing degree 3 and twelve nodes. Linking v30 under v26 produces degree 1 and two nodes. Unchanged roots v18 and v1 have one and sixteen nodes, respectively.

Erase state table and identity conservation

Completed step	Roots t	Marked nonroots m	$\Phi = t + 2m$	Required change
Before erase	3	2	7	v29.parent=v25; v25.parent=v17
Cut v29	4	2	8	v25.degree falls 2→1
Cascading cut v25	5	1	7	Clear v25.mark; v17.degree falls 3→2
Cascading cut v17	6	0	6	Clear v17.mark; v1.degree falls 5→4
Promote v31,v30	8	0	8	Both parents become null; v29.degree=0
Remove/release v29	7	0	7	n falls 32→31; h29 is invalid
After three links	4	0	4	Distinct degrees; recompute min=v26

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Erase state table and identity conservation (continued)

The root snapshot is v30(d0),v31(d1),v17(d2),v25(d1),v26(d0),v18(d0),v1(d4). Equal-degree v25 wins v31 and reaches degree 2; v17 then wins v25 and reaches degree 3. Root v26(key−11) wins v30 and reaches degree 1. Final table entries are 0→v18,1→v26,3→v17,4→v1. There are 31 nodes and the minimum remains key−11. Promoted v31,v30 later become children again, without changing node identities. Promotion describes an intermediate root state, not a promise to remain a root after consolidation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness and amortized derivation for arbitrary erase

```
// Potential Phi_A=A*(t+2*m), with a constant A large enough for pointer work.
// Cut/cascade phase: k cuts, actual <= a*k+b, Delta(t+2m)<=4-k.
// Root extraction: degree d, initial t' roots, final at most D(n)+1 roots.
// Actual <= a*(t'+d+D(n)+1)+b; marks never increase in this phase.
```

Correctness has three parts. Cutting z removes one parent edge and preserves every subtree's internal heap order. Cascading enforces the first-loss mark and second-loss cut rule needed by the degree proof. Promoting z's children preserves all their elements; only z.item is removed and only z's handle becomes invalid. Finally, linking equal-degree roots under the smaller root preserves order and node sets. Taking the least final root yields the global minimum, whether or not z was previously minimal.

Correctness and amortized derivation for arbitrary erase (continued)

If cutting uses k≥1 cuts, roots increase by k. Apart from the initial target, each further cut ancestor was marked, and at most one final ancestor acquires a new mark. Thus $\Delta m \leq 2 - k$ and $\Delta(t + 2m) \leq 4 - k$. For actual cost≤ak+b, choose scaled potential $\Phi_A = A(t + 2m)$, A≥a. Amortized cut cost is at most $ak + b + A(4 - k) \leq b + 4A = O(1)$. A target already at a root needs only constant checks in this phase.

After cutting, let there be t′ roots and let z have degree d. Maximum degree during consolidation has bound $D(n) = O(\log(n + 1))$. Extraction costs at most $a(t' + d + D(n) + 1) + b$. Promotion and linking never increase marks, and at most D(n)+1 roots survive, so $\Delta\Phi_A \leq A(D(n) + 1 - t')$. With A≥a and d≤D(n), amortized extraction is at most $a(d + D(n) + 1) + A(D(n) + 1) + b = O(1 + \log(n + 1))$. The two potential differences telescope using the same potential, proving the complete erase bound.

Correctness and amortized derivation for arbitrary erase (continued)

One operation can still cost Θ(n), due to long cascading cuts or a large root scan/snapshot. Start-from-empty sequence analysis uses the same nonnegative Φ_A; arbitrary startup adds its initial value. Key/result operations and node release follow the fixed-cost model, and handle lookup is separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing heap link

- A standard pairing heap is a multi-child min-heap. Meld compares two roots and makes the larger a new child of the smaller, changing a constant number of pointers.
- Insert melds a singleton. Decrease-key usually cuts the lowered node with its subtree and melds it with the root.
- Constant-time removal from the sibling list requires sibling, parent, predecessor, or equivalent backlink information. A next-only list without a way to locate its predecessor may require scanning. Require a live handle owned by this heap before access.
- A small immediate pointer count does not justify assigning every operation O(1) amortized cost across a sequence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing sibling backlinks and detachment

```
// Pairing roots passed to link have parent=prev=next=null.
detach(x):
    p=x.parent
    if x.prev==null: p.first_child=x.next
    else: x.prev.next=x.next
    if x.next!=null: x.next.prev=x.prev
    x.parent=x.prev=x.next=null
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    b.parent=a; b.prev=null; b.next=a.first_child
    if b.next!=null: b.next.prev=b
    a.first_child=b; return a
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing sibling backlinks and detachment (continued)

Each node stores parent, previous/next siblings, and first_child. Detach applies only to a nonroot. The first child's prev is null; other prev links point to the previous sibling. Removing the first child updates its parent's first_child, while removing another updates the previous sibling's next. Both cases repair the next sibling's prev. The entire subtree below x remains intact. Lowering the root needs no detach. Before delete-min pairs children, save each next pointer and clear parent/prev/next so every child becomes a detached root.

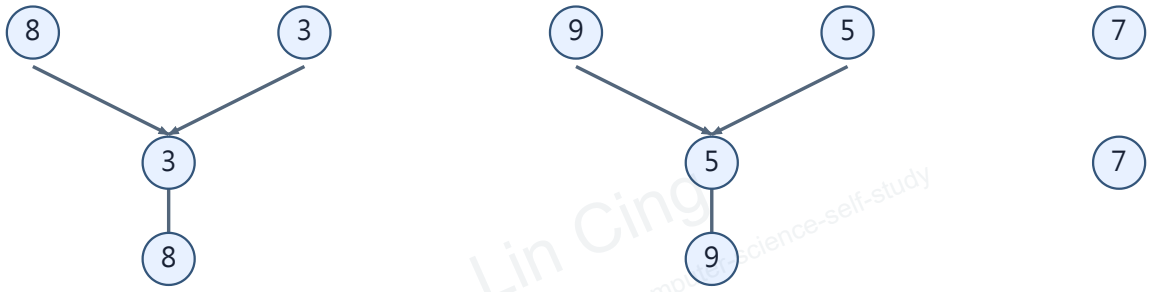
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete two-pass pairing and deferred costs

```
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    prepend b to a.children; return a
delete_min():
    require root!=null
    children=detach_all_children(root); release(root)
    paired=[]
    for adjacent (a,b) left-to-right: paired.append(link(a,b))
    if odd last child remains: paired.append(last)
    root=null
    for heap in reverse(paired): root=link(heap,root)
decrease_key(x,new):
    require new<=x.key; x.key=new
    if x!=root: detach x in O(1) using backlinks; root=link(root,x)
```


Complete two-pass pairing and deferred costs (continued)

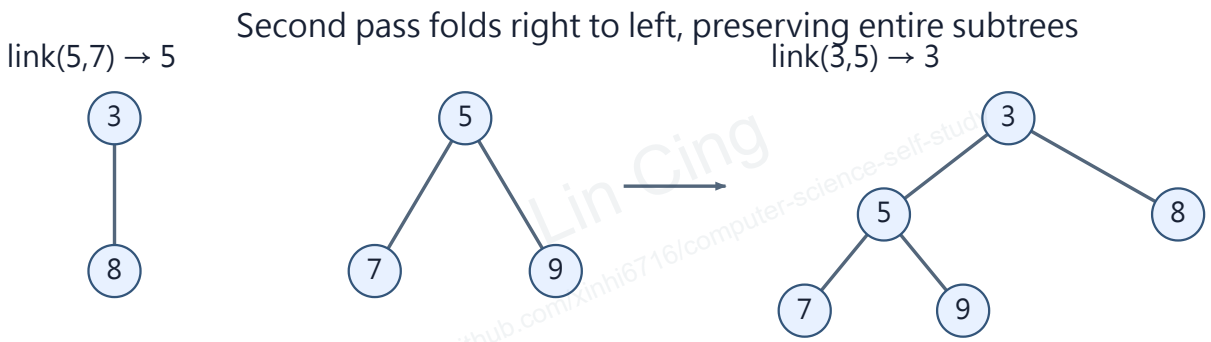
Pairing first pass: adjacent pairs; unpaired 7 remains



Roots 8,3,9,5,7 pair (8,3) into 3 with child 8, (9,5) into 5 with child 9, leaving 7. Right-to-left accumulation links 7 with empty, then 5 with 7 to obtain children 7,9, then 3 with 5 to obtain children 5,8. Subtree 5 retains 7,9. No paired subtree is dismantled. Immediate delete-min link work is proportional to the old root degree and can be Θ(n).

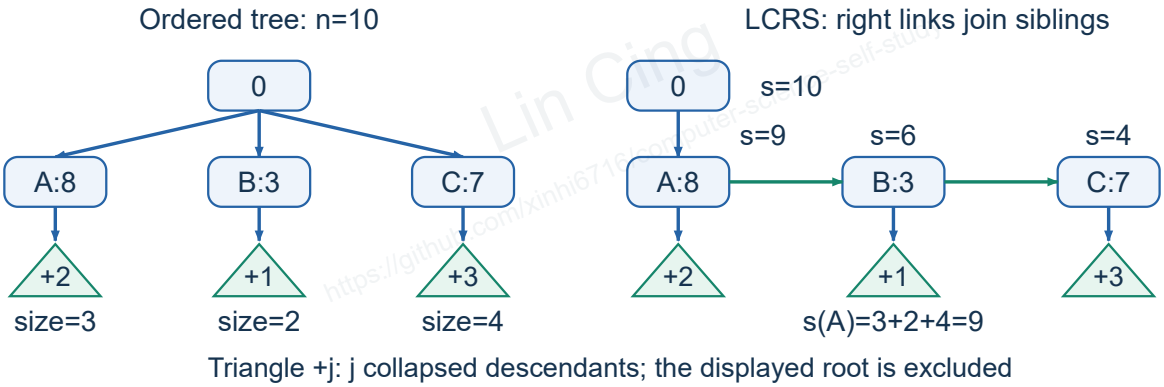
Actual subtree shapes during the pairing second pass

```
paired=[root3(child8),root5(child9),root7]
acc=link(root7,null); acc=link(root5,acc)
acc=link(root3,acc)       // 4 nonempty links across both passes
```



LCRS representation and a nonnegative potential

View the ordered multiway tree as a left-child/right-sibling (LCRS) binary tree. Let s(v) count all nodes reachable from v through these two binary links, $\rho(v) = \log_2 s(v)$, and $\Phi = \sum_v \rho(v)$. This includes later sibling subtrees: **s(v) is not v's original multiway subtree size**. Every s(v)≥1, so Φ≥0; an empty heap has Φ=0. These are analysis variables, not metadata recomputed during operations.



LCRS representation and a nonnegative potential (continued)

Root 0 has child subtrees of sizes 3,2,4. In the binary representation s(A)=9,s(B)=6,s(C)=4, and s(root)=10. Removing the root leaves the ordered forest of children: only the root's $\log_2 10$ disappears; every other s is unchanged.

For independent live heaps, total potential is the sum of their separate potentials; roots are not sibling-linked. During delete-min, define a temporary forest with virtual right-sibling adjacency in component order. If an array implementation clears actual next pointers while detaching roots, interpret the virtual chain from this component order. Do not assign unrelated heap potentials to each cleared-pointer intermediate state. Each macro-link includes constant detach/reconnect work; scanning, temporary storage, and releasing the old root cost O(k+1). Both endpoint potentials match the actual heaps.

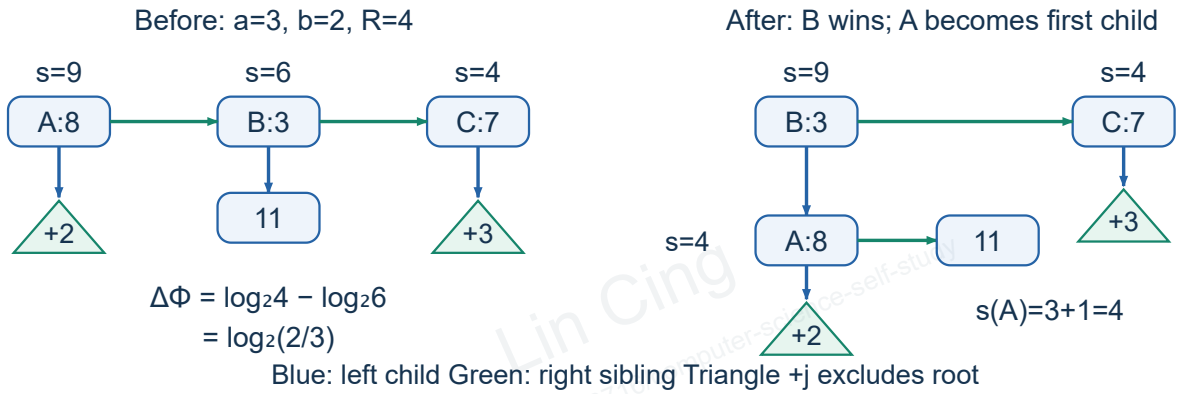
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An adjacent link changes only two root ranks

Consider adjacent components A,B in one forest, with original multiway sizes $a,b \geq 1$ and a suffix containing $R \geq 0$ nodes. Before linking, s(A)=a+b+R and s(B)=b+R. Link makes the losing root the winner's **first child**. Resolve equal keys consistently.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An adjacent link changes only two root ranks (continued)



After linking, the winner has s=a+b+R and the loser has s=a+b−1. All other sizes are unchanged: a root in the prefix reaches the same suffix nodes, and old child chains retain their contents. Either winner therefore gives

$$\Delta\Phi_{\text{link}} = \log_2(a+b-1) - \log_2(b+R)$$

First-pass inequality with a positive suffix

Write A=a+b and T=A+R. If R>0, the exact adjacent-link formula gives

$$1 + \Delta\Phi_{\text{link}} = \log_2 \frac{2(A-1)}{b+R} \le \log_2 \frac{2A}{R} \le 2\log_2 \frac{T}{R}$$

The first inequality uses A−1≤A and b+R≥R. The second is equivalent to $2AR \le (A+R)^2$, whose right side minus left side is $A^2 + R^2 \ge 0$. The argument depends only on sizes, not which root key wins.

Number the pairs processed left-to-right. Let $T_j = a_j + b_j + R_j$ be the size still present from pair j to the right end. The untouched suffix of one pair is precisely the total starting at the next pair, so $R_j = T_{j+1}$. Summing the logarithms cancels intermediate terms. A final pair with R=0 cannot use this division by R and is handled separately below.

Telescoping the first pass for odd degree

Before deletion there are n nodes; after removing the root N=n−1 remain. Suppose its degree is k=2q+1. The first pass makes q links and leaves the rightmost tree of size r≥1 unpaired. This tree lies in every processed pair's suffix, so every $R_j > 0$.

$$q + \Delta\Phi_1 \le 2\sum_{j=1}^{q} \log_2 \frac{T_j}{R_j} = 2\log_2 \frac{N}{r} \le 2\log_2 N$$

Consequently $\Delta\Phi_1 \le -q + 2\log_2 N$. The negative −q term is needed to pay for a linear number of links. A bound saying only that potential increases by O(log n) would omit this payment.

In the existing five-singleton example 8,3,9,5,7, N=5,q=2,r=1. The two exact link changes are $\log_2(1/4)$ and $\log_2(1/2)$, totaling −3. For k=1, q=0 and r=N: the empty sum is 0, and no link occurs. No log0 is needed.

The final zero-suffix pair for even degree

If k=2q≥2, the last pair has $R_q = 0$. Write $A_q = a_q + b_q \ge 2$. The first q−1 pairs telescope to

$$(q-1) + \Delta\Phi_{\text{previous}} \le 2\log_2(N/A_q)$$

Use the exact formula for the last pair: $\Delta\Phi_{\text{last}} = \log_2(A_q - 1) - \log_2 b_q \le \log_2 A_q$. Therefore

$$\Delta\Phi_1 \le -q + 1 + 2\log_2(N/A_q) + \log_2 A_q \le -q + 1 + 2\log_2 N$$

When q=1 the earlier sum is empty and $N = A_q$, so its upper bound is 0. A uniform odd/even bound is $\Delta\Phi_1 \le -q + 2\log_2 N + 1$, where q=⌊k/2⌋. Degree zero is a separate singleton case.

Second pass and the complete delete-min bound

The first pass leaves m=⌈k/2⌉ trees. Accumulate right-to-left. If the next left tree has size a and the accumulated right tree has size b, then R=0, and

$$\Delta\Phi_{\text{link}} = \log_2(a+b-1) - \log_2 b \le \log_2 \frac{a+b}{b}$$

The next accumulator size is a+b, so $\Delta\Phi_2 \le \log_2(N/b_0) \le \log_2 N$, where $b_0 \ge 1$ is the rightmost component size. If m≤1 there are no links. The five-child example has second-pass ratios 2 and 4/3; including the first pass and removal of the original root gives total potential ratio (1/6)(1/4)(1/2)(2)(4/3)=1/18.

For k≥2, combine root removal and both passes: $\Delta\Phi \le -\log_2 n - q + 3\log_2 N + 1 \le -q + 3\log_2 n + 1$. Actual work is at most ck+d for fixed constants, and k≤2q+1. Choose the same scaled potential $\Psi = C\Phi$ with $C \ge 2c$. $\text{actual} + \Delta\Psi \le (2c-C)q + 3C\log_2 n + c + d + C = O(1+\log n)$. This pays for scanning and all k−1 links, not just first-pass links.

Second pass and the complete delete-min bound (continued)

For k=0, n=1 and deletion leaves an empty heap: ΔΦ=0 and actual cost O(1). For k=1, the sole child becomes root, $\Delta\Phi = -\log_2 n$, and actual cost is O(1). Reject empty delete-min. The array procedure stores O(k) roots, possibly O(n) space; the proof does not imply logarithmic temporary space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Meld, insert, and decrease under the same potential

Meld two independent nonempty heaps of sizes a,b, with n=a+b. Initially their root ranks are $\log_2 a$ and $\log_2 b$. Afterward the winner has s=n and the loser s=n−1; other sizes do not change. Thus $\Delta\Phi_{\text{meld}} = \log_2 \frac{n(n-1)}{ab} \le \log_2 n$, since $ab \ge a + b - 1 = n - 1$. An empty operand gives ΔΦ=0. Inserting into a nonempty heap melds a singleton and has exactly $\Delta\Phi = \log_2 n$; inserting the first node has ΔΦ=0. Although both operations take O(1) immediate work, **this analysis charges O(1+log(n+1)) amortized for each**.

Decreasing the root's key does not change shape or potential. For a nonroot x of original multiway subtree size a, first cut it into a separate heap. The binary size of x loses its right-sibling suffix. Binary ancestors that previously reached the removed subtree only lose reachable nodes; other sizes are unchanged. Thus cutting has $\Delta\Phi \le 0$. Melding it back adds at most $\log_2 n$. A valid handle and the stated parent/prev/next backlinks make the actual cut O(1). The same Ψ therefore charges decrease-key O(1+log(n+1)); it does not establish constant amortized decrease-key.

Sequence cost and the scope of this analysis

Operation	Immediate actual cost	Amortized charge under this same Ψ
find-min	O(1)	O(1)
insert	O(1)	O(1+log(n+1))
meld	O(1)	O(1+log(n+1)), n combined size
decrease-key	O(1), valid handle/backlinks required	O(1+log(n+1))
delete-min	O(k+1), worst-case O(n)	O(1+log(n+1))

Starting with empty heaps, $\Psi_0=0$ and the final $\Psi \ge 0$. Hence $\sum c_i = \sum \hat{c}_i + \Psi_0 - \Psi_{\text{final}} \le \sum \hat{c}_i$. If N is the maximum live-node count and I,M,D,K,F count insertions, melds, deletions, decreases, and find-min operations, total cost is $O((I + M + D + K)(1 + \log_2(N + 1)) + F)$, plus O(1) for each empty-heap creation. A nonempty initial state contributes Ψ_0.

Sequence cost and the scope of this analysis (continued)

This proves one compatible collection of bounds for the standard two-pass variant. Substituting a different analysis's constant amortized insert/meld into this proof does not establish $O(I + M + (D + K) \log N + F)$. Constant immediate cut work also does not imply constant amortized decrease-key. Changing pass directions or the prepend-child rule requires checking the inequalities again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complexity comparison and selection

Structure	find-min	meld	delete-min	decrease-key
Leftist	O(1) worst	O(log n) worst	O(log n) worst	No fast bound supplied here
Eager binomial	O(1) with minimum pointer	O(log n) worst	O(log n) worst	O(log n) worst
Fibonacci	O(1) worst	O(1) worst	O(log n) amortized	O(1) amortized
Two-pass pairing	O(1) worst	O(1) immediate pointer work	O(log n) amortized	O(log n) amortized under this potential

Every decrease-key bound assumes a valid handle. Amortized bounds for different operations must belong to one compatible analysis. Independently selecting optimistic bounds can produce an invalid combination.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complexity comparison and selection (continued)

Keep constant terms for empty and singleton cases. Melding two empty heaps still tests pointers and takes O(1); the precise leftist bound is O(1+log(n1+1)+log(n2+1)). Delete-min requires nonempty input. Reject an empty request instead of invoking log 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and further reading

- Fibonacci heaps: Fredman–Tarjan paper.
- Pairing heaps: original paper and authors' notes, and Pettie's advanced analysis.
- Binomial heaps: Stanford CS166 course.
- Prepend-child linking, pass directions, and backlinks for the standard two-pass variant: Sinnamon–Tarjan, § 2. The LCRS potential and compatible charges used in this chapter are derived in full above.
- Chicago CS223: binomial representation, first-child linking, and root-list order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are BSTs, rotations, recursion, probability-weighted costs, dynamic programming, and amortized analysis.
- Distinguish optimal BST's static weighted objective, AVL and red-black worst-case height guarantees, and splay sequence guarantees.
- Check rotations using inorder order and structural invariants instead of memorizing shapes alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Which cost is being optimized?

- Minimizing height reduces worst-case search depth but need not minimize average search cost.
- If key k_i is queried with known probability p_i, expected successful comparisons are $\sum_i p_i(\mathrm{depth}(k_i) + 1)$.
- Accounting for unsuccessful queries requires n+1 gap probabilities $q_0,\ldots, q_n$ and a stated dummy-leaf cost convention.
- Optimal BSTs suit relatively stable distributions and static indexes. Frequent updates require separately accounting for reoptimization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Successful-search optimal BST recurrence

- Let C[i, j] be the minimum weighted comparison cost for ordered keys in closed interval [i, j]. Empty intervals cost 0.
- Define $W[i, j] = \sum_{k=i}^{j} p_k$. Choosing root r adds one comparison to every key in the interval relative to its child-subproblem cost.
- Thus `C[i,j]=W[i,j]+min_{r=i..j}(C[i,r-1]+C[r+1,j])`.
- Fill increasing interval lengths. Prefix sums give O(1) W queries. Time is $O(n^3)$, space $O(n^2)$. Store optimal roots to reconstruct the tree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked weighted examples

- For probabilities p=[0.1,0.2,0.7], root key 2 gives cost 1+0.1+0.7=1.8.
- Root key 3 with key 2 at its left and key 1 below that gives 0.7 • 1+0.2 • 2+0.1 • 3=1.4.
- The minimum-height choice is not the minimum-expected-cost choice. Placing a frequent key near the root can offset deeper rare keys.
- Greedily taking the highest-probability key is still not always optimal. For p=[0.4,0.35,0.25], the best tree rooted at key 1 costs 1.85, while root key 2 gives 1.65.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A standard unsuccessful-search convention

- Set `e[i,i-1]=q_{i-1}`, charging one dummy comparison in an empty subtree. Define $w[i,j] = \sum_{k=i}^{j} p_k + \sum_{k=i-1}^{j} q_k$.
- The recurrence is `e[i,j]=w[i,j]+min_r(e[i,r-1]+e[r+1,j])`. Under this convention the empty cost cannot be 0.
- This computes $\sum_i p_i(\text{depth}(\text{key}_i)+1) + \sum_j q_j(\text{depth}(\text{dummy}_j)+1)$. Counting only real-key comparisons subtracts one per failed query, a constant total $\sum_j q_j$, so the optimal tree is unchanged.
- Knuth's O(n²) optimization requires the applicable nonnegative weights and monotone-root property. An arbitrary interval DP does not inherit it merely by name. Its full proof is outside this edition.

Complete optimal-BST interval table and failed-query example

```
for i=1..n+1: C[i,i-1]=0
for length=1..n:
    for i=1..n-length+1:
        j=i+length-1; C[i,j]=infinity
        for r=i..j:
            candidate=W(i,j)+C[i,r-1]+C[r+1,j]
            if candidate<C[i,j]: C[i,j]=candidate; root[i,j]=r
build(i,j):
    if i>j: return null
    r=root[i,j]; return Node(k[r],build(i,r-1),build(r+1,j))
```

For successful probabilities [.1,.2,.7], singleton costs are .1,.2,.7. C12 candidates are .5,.4; C23 candidates are 1.6,1.1; C13 candidates are 2.1,1.8,1.4. Reconstruct root 3 with left child 2 and left grandchild 1. Direct weighted comparisons give .1×3+.2×2+.7×1=1.4.

Complete optimal-BST interval table and failed-query example (continued)

For one key with p1=.6, q0=.1, q1=.3, dummy-counting initialization e10=.1, e21=.3, w11=1 yields e11=1.4. Counting only real comparisons gives 1 because every query compares the sole real key once. The difference is q0+q1=.4. Do not initialize empty costs to 0 and still claim the dummy-counting result.

For a fixed root r, ordering uniquely partitions key ranges. Replacing a nonoptimal child tree by an optimal tree on the same keys lowers total cost without changing root contribution W, contradicting optimality. Induction on interval length proves the DP without a highest-frequency greedy assumption.

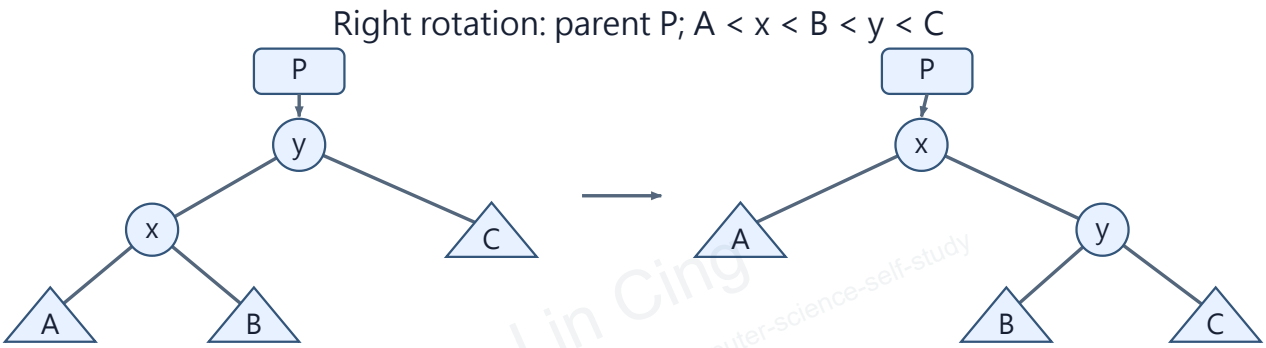
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rotations preserve inorder order

- Before a right rotation, y is root and x its left child, with order `A<x<B<y<C`. Afterward x is root and y its right child. Inorder order is unchanged.
- Preserve B, set y.left=B and x.right=y, then reconnect the original parent. If parents are stored, update B and both rotated nodes' parent fields.
- Update height, subtree size, or other augmentation first at lowered y, then at raised x.
- Rotation preserves BST order, not every extra invariant. Colors and AVL heights require their own repair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rotations preserve inorder order (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL local height constraint

- Set h(null)=−1 and h(leaf)=0. Balance factor `bf(x)=h(left)-h(right)` must be −1,0, or 1.
- Minimum nodes at height h satisfy `N(h)=1+N(h-1)+N(h-2)`, for h≥1, with N(−1)=0, N(0)=1. The taller child has height h−1 and the other at least h−2.
- Fibonacci growth gives h=O(log n). Search, insertion, and deletion have O(log n) worst-case path lengths.
- The constraint holds at every node, not only the root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four AVL insertion shapes

- LL means a left-heavy x with insertion in its left child's left side. Right-rotate x. RR is the symmetric left rotation.
- LR first left-rotates the left child, then right-rotates x. RL is symmetric.
- Inserting 30,10,20 produces LR. Raise 20 above 10, then above 30, giving root 20 with children 10,30.
- Repair at the first unbalanced insertion ancestor restores that subtree's preinsertion height. No further balancing rotations are required above it, but metadata must still be correct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL deletion may repair repeatedly

- First perform BST deletion. Update toward the root from the position where height actually decreased. Local repair need not restore predeletion height.
- If bf(x)=2 and bf(left)$\geq$0, right-rotate. If the left child's factor is negative, use LR. Right-heavy cases are symmetric.
- The deletion case with child bf=0 cannot be handled by remembering only insertion direction.
- At most O(log n) local updates along the root path give O(log n) worst-case total time. Stopping after one repair can leave a higher ancestor unbalanced.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL recomputation and insertion/deletion repair

```
fix(x):
    if x==null: return null
    recompute_height(x)
    if bf(x)==2:
        if bf(x.left)<0: x.left=rotate_left(x.left)
        return rotate_right(x)
    if bf(x)==-2:
        if bf(x.right)>0: x.right=rotate_right(x.right)
        return rotate_left(x)
    return x
insert(x,k):
    if x==null: return leaf(k)
    if k<x.key: x.left=insert(x.left,k)
    else if k>x.key: x.right=insert(x.right,k)
```

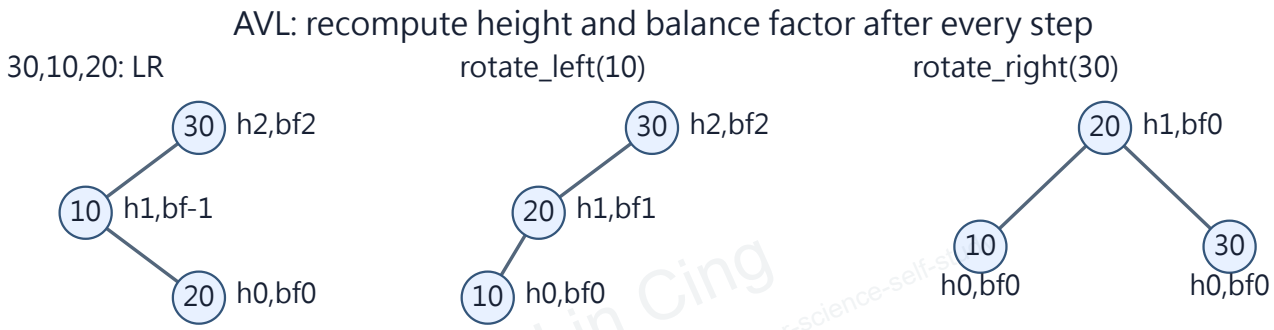
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL recomputation and insertion/deletion repair (continued)

```
    else: update/reject duplicate
    return fix(x)
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x has at most one child:
            child=the nonnull child or null; release(x); return child
        s=minimum(x.right); x.contents=copy(s.contents)
        x.right=erase(x.right,s.key)
    return fix(x)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL recomputation and insertion/deletion repair (continued)



Each rotation recomputes the lowered node before the new root. For 30,10,20, initially 30 has (h2,bf2), 10 has (h1,bf−1), and 20 has (h0,bf0). Left rotation at 10 leaves 30 at (h2,bf2), with left child 20 at (h1,bf1). Right rotation at 30 yields root 20 at (h1,bf0) and two height-0 leaves.

AVL recomputation and insertion/deletion repair (continued)

For root 4 with left subtree 2(1,3) and right leaf 5, deleting 5 gives bf(4)=2 and bf(2)=0. A single right rotation gives root 2, left child 1, right child 4, and 4.left=3. Local height stays 2 in this case. Other deletion repairs reduce height and must continue upward. Constant rotations and metadata work per logarithmic-depth frame give O(log n).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General red-black invariants

- Every node is red or black, the root is black, and all NIL leaves are black.
- Children of red nodes are black. Every path from a node to a descendant NIL contains the same number of black nodes.
- This chapter uses general node-colored red-black trees. It does not require all red links to lean left. Princeton's LLRB imposes a stronger representation convention.
- Include implicit NIL leaves in the black-height convention when checking a diagram.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black height bound

- Define bh(x) as black nodes strictly below x through and including NIL. A subtree with bh=b has at least $2^b - 1$ real nodes.
- For induction, each real child's black height is at least b−1. Its subtree therefore has at least $2^{b-1} - 1$ nodes. Adding both children and x yields at least $2^b - 1$. NIL gives the zero-node base.
- Red nodes cannot be consecutive, so root-to-NIL length is at most roughly twice the root black height. Combining $n \geq 2^{\mathrm{bh}(\mathrm{root})} - 1$ gives the standard height bound $h \leq 2\log_2(n+1)$, with endpoint constants consistent with edge height.
- This is a worst-case guarantee with no random-key or uniform-query requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black insertion repair

- Color the new data leaf red, preserving black height. A black parent needs no repair. A red parent creates a red-red violation.
- If the uncle is red, color parent and uncle black, color grandparent red, and continue at the grandparent.
- With a black uncle, first rotate the parent if needed to turn a bent path into a straight one. Rotate the grandparent and exchange the appropriate colors. Mirror cases are symmetric.
- Finally color the root black. Inserting 10,5,1 right-rotates 10, yielding black root 5 with red children 1,10.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete ancestor loop for red-black insertion

```
insert_fix(z):
    while color(z.parent)==red:
        p=z.parent; g=p.parent
        if p==g.left:
            u=g.right
            if color(u)==red:
                p.black; u.black; g.red; z=g
            else:
                if z==p.right:
                    z=p; rotate_left(z); p=z.parent; g=p.parent
                p.black; g.red; rotate_right(g)
        else: apply exact left/right mirror
    root.black
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

New data leaves are red with black NIL children. Use a black parent sentinel for the root or explicitly guard null parents. Insert 10 as a black root, 5 as a red left child, then 1. A black uncle gives the straight case. Right-rotate 10 to obtain black root 5 with red children 1,10. Insert 7 under 10. Red parent 10 and uncle 1 become black, and grandparent 5 becomes red before the root becomes black again. Every root-to-NIL path has three blacks.

Complete ancestor loop for red-black insertion (continued)

A red uncle can propagate z repeatedly. In the bent black-uncle case, recompute p/g after the first rotation. Using stale parents can misattach the second rotation. Each loop moves upward or ends with constant rotations, so total time is O(log n). These branches implement general red-black invariants, not LLRB's stronger leaning rule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deletion and a black-height deficit

- Removing a red node does not reduce black height. If a removed black node has a red replacement child, color that child black.
- Otherwise track one missing black at replacement position x. This is an analysis state, not a required third stored color.
- Suppose x is a left child with sibling w. If w is red, rotate and recolor to obtain a black sibling before proceeding.
- If w is black with two black children, color w red and move the deficit to the parent. A previously red parent can become black and absorb the deficit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Near and far nephews in deletion

- With x on the left, w's left child is the near nephew and its right child is the far nephew.
- If w is black, the far nephew black, and the near nephew red, right-rotate w and recolor, producing the far-red case.
- With a red far nephew, give w the parent's old color, color parent and far nephew black, and left-rotate the parent. This removes the deficit on x's side.
- All cases mirror when x is on the right. Each step does constant local work or moves upward, giving O(log n) worst-case repair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rotation and the transplant entry to red-black deletion

```
right_rotate(y):
    x=y.left; require x!=NIL
    b=x.right; p=y.parent
    y.left=b
    if b!=NIL: b.parent=y
    x.parent=p
    if p==NIL: root=x
    else if p.left==y: p.left=x
    else: p.right=x
    x.right=y; y.parent=x
    pull(y); pull(x)
transplant(u,v):
    if u.parent==NIL: root=v
    else if u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent       // including NIL: deletion needs its deficit parent
RB_erase(z):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rotation and the transplant entry to red-black deletion (continued)

```
    y=z; removed_color=y.color
    if z.left==NIL:
        x=z.right; p=z.parent; transplant(z,x); pull_up(p)
    else if z.right==NIL:
        x=z.left; p=z.parent; transplant(z,x); pull_up(p)
    else:
        y=minimum(z.right); removed_color=y.color; x=y.right
        if y.parent==z: x.parent=y
        else:
            p_old=y.parent; transplant(y,x)
            pull_up(p_old)          // repair old path before y changes parent
            y.right=z.right; y.right.parent=y
        transplant(z,y); y.left=z.left; y.left.parent=y; y.color=z.color
        pull(y); pull_up(y.parent)  // repair y at its new position
    release(z)
    if removed_color==black: delete_fix(x)
```


Rotation and the transplant entry to red-black deletion (continued)

This page uses a shared black NIL sentinel rather than a data node. The root's parent is NIL, NIL is always black, and delete_fix is the four-case loop in "Complete control flow for red-black deletion repair". Left rotation is the exact left/right mirror. The complete augmentation here is subtree size: NIL.size=0 and pull(x)=1+size(left)+size(right); pull_up recomputes bottom-up until NIL; pure red-black metadata needs no automatic recoloring during rotation. Save middle subtree b and external parent p before modifying links.

Two-child deletion moves successor node y into z's position rather than copying a key and releasing y. The color removed from y's old position determines the black deficit, not the color y receives from z. Node x is y's sole right child or NIL. If y directly follows z, assign x.parent=y. Otherwise transplant y's right child into its old location, then attach z's right subtree to y. Attach the left subtree and original z color, and release z. All other element identities remain intact.

Rotation and the transplant entry to red-black deletion (continued)

All empty children may share NIL, but deletion repair needs the parent of the current deficient x. Rotation does not overwrite NIL.parent for an arbitrary empty middle subtree b. Only an explicit transplant or direct-successor branch sets that parent. Deleting the sole black root gives root=NIL; delete_fix skips its loop because x==root, leaving a valid empty tree.

The proof separates responsibilities: BST transplant preserves inorder and all undeleted elements; copying z's color preserves black counts at y's new location; only y's old position can have a deficit. The four-case loop preserves every other subtree while propagating or eliminating that single deficit. Search, successor lookup, and repair follow O(log n) paths; each transplant or rotation uses constant pointer work. The two additional pull_up paths each take O(log n) and cannot be omitted.

Subtree sizes, selection, and both repair paths

```
pull(x):
    require x!=NIL
    x.size=1+x.left.size+x.right.size
pull_up(x):
    while x!=NIL: pull(x); x=x.parent
select(k):
    require 1<=k<=root.size          // one-based rank
    x=root
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
    unreachable                      // sizes and rank invariant forbid this
```

Every data node satisfies size(x)=1+size(x.left)+size(x.right), with NIL.size=0. A new leaf starts at size 1. After BST insertion, pull_up from its parent before insert_fix. Each rotation pulls the descending node before the new local root.

Subtree sizes, selection, and both repair paths (continued)

Insert 4,2,6,1,3,5,7 legally, then erase red leaf 1. No color repair or rotation is needed, yet metadata must change:

Field	Before deletion	Transplant without pulls	Correctly updated
size(2)	3	3	2
size(4)	7	7	6
select(3)	Node 3	Reaches NIL	Node 4

With stale left size 3, root 4 claims rank 4 and sends rank 3 left. Node 2 then leads the search through node 3 to NIL. With correct left size 2, root 4 is exactly rank 3. Valid red-black colors and black heights do not establish valid augmentation.

Subtree sizes, selection, and both repair paths (continued)

For at most one child, transplant changes subtree sets only at z's old parent and above; pull_up from saved p. For a detached successor y, transplant its right child x into the old position, then immediately pull_up from p_old while that ancestor chain is intact. Move y to z's position, attach both children, pull(y), and pull_up from its new parent. A direct successor omits the old-position path but still needs the new-position repair. Overlapping paths add only a constant factor.

Bottom-up induction proves sizes: subtrees outside the changed paths are unchanged, and each pull sees already correct child sizes. Subsequent recoloring leaves sizes unchanged. A rotation changes child sets only at two local roots; pulling the lower before the upper fixes them. The local subtree's total element set is unchanged, so its external ancestors keep correct sizes. Selection maintains that k is a valid desired rank within the current subtree; exact left sizes justify each discarded interval, and finite height ensures termination.

Two repair paths, constant rotations, search, and successor lookup retain O(log n) worst-case update time; select is also O(log n). Height augmentation requires separate upward propagation when a rotation changes local height; the unchanged-element-count argument is specific to size.

Complete control flow for red-black deletion repair

```
while x!=root and color(x)==black:
    if x==x.parent.left:
        p=x.parent; w=p.right
        if color(w)==red:
            w.black; p.red; rotate_left(p); w=p.right
        if color(w.left)==black and color(w.right)==black:
            w.red; x=p
        else:
            if color(w.right)==black:
                w.left.black; w.red; rotate_right(w); w=p.right
            w.color=p.color; p.black; w.right.black
            rotate_left(p); x=root
    else: apply exact left/right mirror
x.black
```

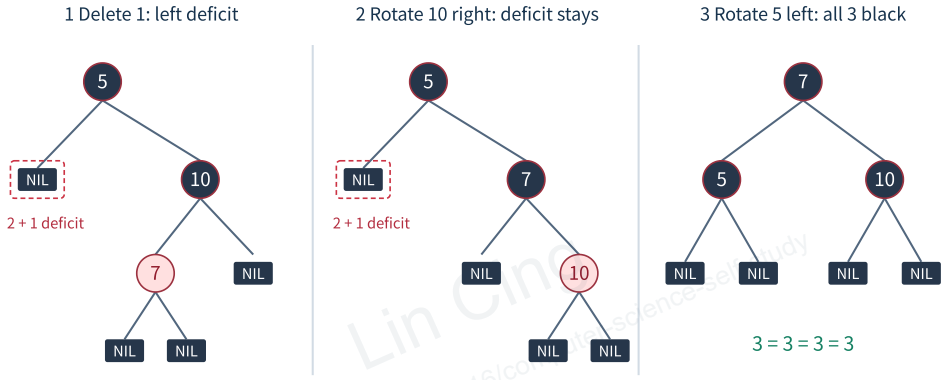
Assume BST deletion/transplant is complete and x identifies the deficient position, possibly a NIL sentinel with its parent available. Skip repair if the physically removed color was red. Color assignments create no nodes. Rotations update external parents and root.

Complete control flow for red-black deletion repair (continued)

Use black root 5, black left child 1, black right child 10, and red node 7 as 10.left. Delete 1, leaving a deficient left NIL, black sibling 10, red near nephew 7, and black far NIL. Color 7 black and 10 red, then right-rotate 10. The new sibling 7 is black with red far nephew 10. Give 7 the old parent color, color 5 and 10 black, then left-rotate 5. The final black root 7 has black children 5,10. Every root-to-NIL path has three blacks, and inorder changes [1,5,7,10]→[5,7,10].

A red sibling conversion does not itself eliminate the deficit. Two black nephews can propagate it to the root. The cases cannot be replaced by unrelated one-level early returns.

Red-black deletion: deficit and two rotations



B: black / R: red; NIL is black. Inorder: 5,7,10.

Red-black deletion: deficit and two rotations (continued)

Start with the valid tree 5B(1B,10B(7R,NIL)) and delete 1. All NIL nodes are black; the dashed red outline denotes an analytical one-black deficit, not a third node color. In the first state the left route contains only 5B and NIL, two actual black nodes, while every right route contains three. Adding the deficit makes the counts temporarily equal. In state 2, recolor near nephew 7 black and 10 red, then rotate 10 right. The deficit remains at 5's left NIL. In state 3, 7 inherits 5's black color, 5 and far nephew 10 become black, and a left rotation at 5 makes all four root-to-NIL routes contain three black nodes. Only then is the deficit gone.

Every rotation preserves inorder [5,7,10] and updates the external root connection and parent pointers. These path counts include the local root and NIL, unlike the earlier bh(x) convention excluding x. Stopping after state 2 leaves the black deficit unresolved: converting a near red nephew into a far red nephew is a preparatory case, not completion.

Splay zig, zig-zig, and zig-zag

- After accessing x, rotate it to the root. If only a parent remains, perform one zig.
- If x and its parent are both left children or both right children, zig-zig first rotates the parent over the grandparent, then x over its parent.
- With opposite directions, zig-zag first rotates x over its parent, then x over its grandparent.
- These preserve inorder order without a fixed balance factor or an O(log n) height guarantee after every operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

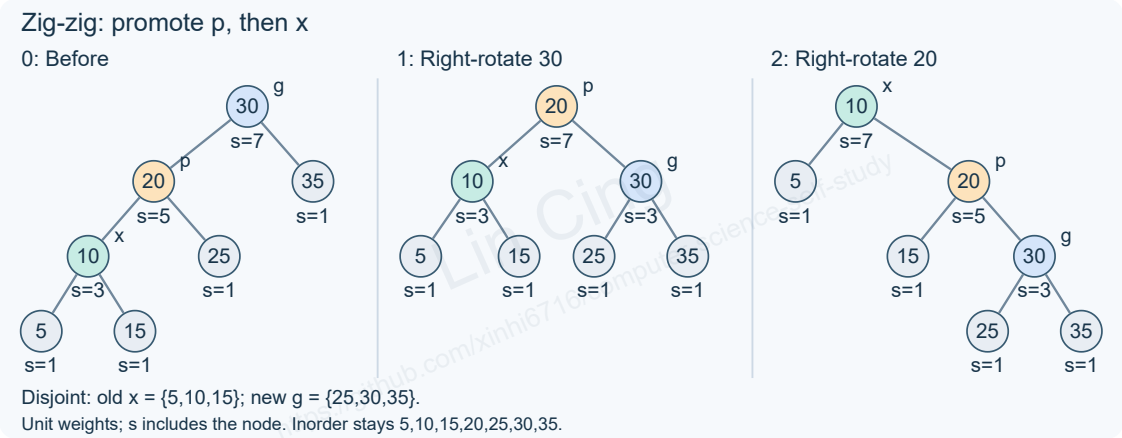
The splay access lemma

- Assign positive fixed node weights. Let s(x) be subtree weight, rank $r(x) = \log_2 s(x)$, and potential $\Phi = \sum_x r(x)$.
- The access lemma bounds a splay's amortized work by $O(1 + \log(s(\text{root})/s(x)))$.
- Unit weights give amortized O(log n) operations. An arbitrary initial tree requires an initial-potential term. A single operation is not worst-case logarithmic.
- In the right chain 1→2→3→4, searching for 4 visits four nodes. Repeated access after 4 reaches the root costs constant work, illustrating sequence adaptation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zig: promote the parent first

Fix x=10, p=20, g=30. Promote orange p over g, then green x over p. The letters identify the same nodes throughout; do not rename them after a rotation.



The intermediate root is 20; only the final root is 10. Rotating x over p first and then over g produces a different tree. Moving the target to the root alone does not specify splay's rotation order.

A complete potential account for zig-zig

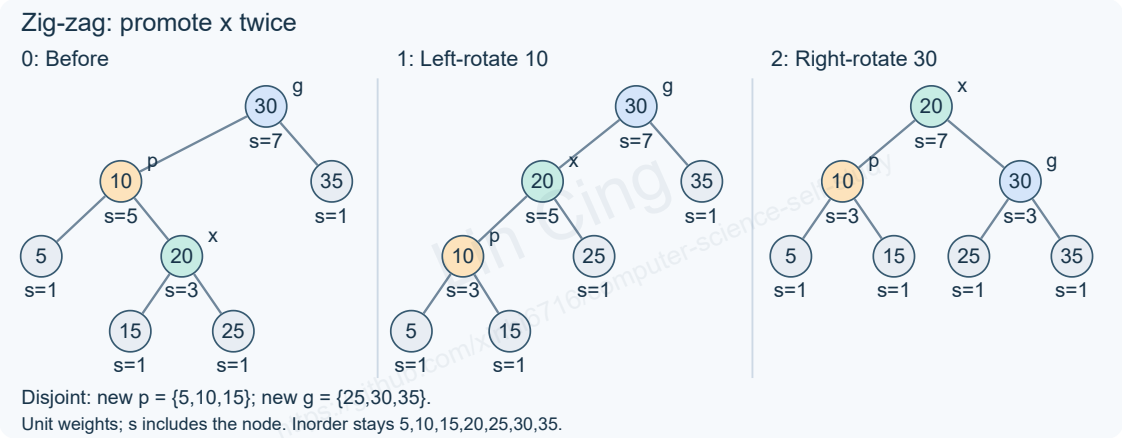
Use unit weights: s(v) counts the subtree including v, and $\Phi = \sum_v \log_2 s(v)$. The four leaf ranks stay zero; only three nodes need accounting.

Node	Initial s	After right-rotate 30	After right-rotate 20
x=10	3	3	7
p=20	5	7	5
g=30	7	3	3

Both endpoint potentials equal $\log_2(3 \cdot 5 \cdot 7) = \log_2 105$. Hence ΔΦ=0 and amortized rotation cost is 2. The lemma gives $3\log_2(7/3) \approx 3.667$, which exceeds 2. **Potential need not decrease on every operation; its changes account for the whole sequence.**

Zig-zag: promote the target twice

Fix x=20, p=10, g=30. Left-rotate 10, then right-rotate 30. Restore the four leaf attachments at each step.



In the final tree the subtrees rooted at p and g are disjoint. These are the two sets needed for the concavity inequality on the next page.

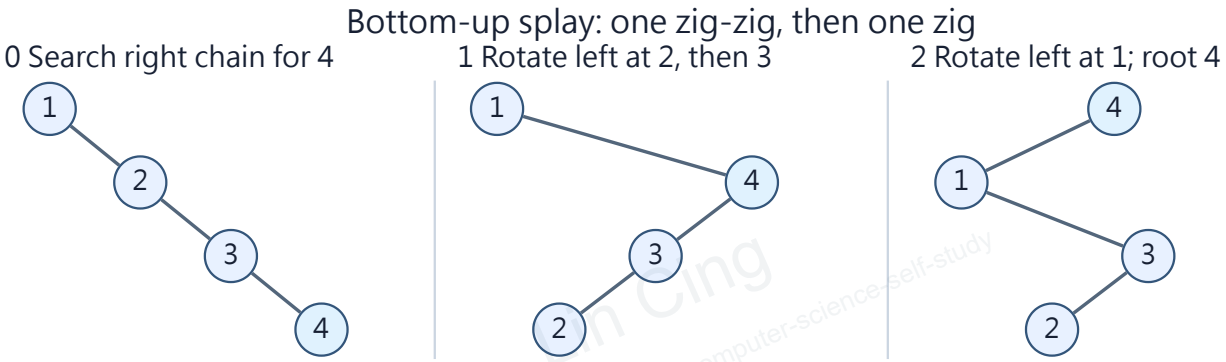
Connect the zig-zag numbers to concavity

Node	Initial s	After left-rotate 10	After right-rotate 30
x=20	3	5	7
p=10	5	3	3
g=30	7	7	3

$\Delta\Phi = \log_2(7 \cdot 3 \cdot 3) - \log_2(3 \cdot 5 \cdot 7) = \log_2(3/5) \approx -0.737$. Thus amortized cost is about 1.263, below $2\log_2(7/3) \approx 2.445$.

Here a=b=3 and a+b=6≤S=7. Since $(a-b)^2 \geq 0$, $ab \leq ((a+b)/2)^2 \leq (S/2)^2$. Taking base-2 logs yields $\log_2 a + \log_2 b \leq 2\log_2 S - 2$. Positive weights make the logs defined. **The example checks values; the general argument holds for arbitrary positive a and b.**

From a right-chain search to a complete splay



Search visits 1,2,3,4. Splay left-rotates 2, then 3, then 1. Finally 4.left=1, 1.right=3, 3.left=2; inorder remains [1,2,3,4]. A repeated search for 4 visits only the root and performs no rotations. Endpoint potentials both equal $\log_2 24$; the amortized guarantee does not require a decrease at every access.

Local accounting for the access lemma

- Use fixed positive weights, base-2 ranks, and one cost unit per rotation.
- Let p and g be x's parent and grandparent, and primes denote values after a double rotation. Since r′(x)=r(g) and all other node subtree weights are unchanged, amortized double-rotation cost is $2 + r'(p) + r'(g) - r(x) - r(p)$.
- For disjoint subtree weights a, b with a+b≤S, concavity gives $\log_2 a + \log_2 b \leq 2 \log_2 S - 2$. The next two slides use this inequality to pay for both double-rotation cases.
- This proof counts rotations plus potential change. Constant pointer-work factors can be paid by scaling potential. Search depth equals rotation count up to endpoint work.

Zig-zig amortized inequality

- In zig-zig, the old x subtree and new g subtree are disjoint and both lie in the new x subtree. Thus $r(x) + r'(g) \leq 2r'(x) - 2$.
- Since r(p)≥r(x) and r′(p)≤r′(x), the double-rotation cost is at most $2 + r'(x) + r'(g) - 2r(x)$.
- Substituting the first inequality gives $\hat{c} \leq 3(r'(x) - r(x))$. These rank differences can be summed without assuming every rotation reduces global height.
- Identify the disjoint subtrees before applying concavity. Two immediate rotations alone do not prove constant amortized work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zag, zig, and telescoping

- After zig-zag, p and g subtrees are disjoint, so $r'(p) + r'(g) \leq 2r'(x) - 2$. With r(p)≥r(x), cost is at most $2(r'(x) - r(x))$, and therefore at most three times the difference.
- A final zig costs $1 + r'(p) - r(x) \leq 1 + r'(x) - r(x) \leq 1 + 3(r'(x) - r(x))$.
- The x-rank differences telescope. Amortized rotations for the whole splay are at most $3(\log_2 W - \log_2 s_{\text{before}}(x)) + 1$, where W is total tree weight.
- With unit weights, W=n and $s_{\text{before}}(x) \geq 1$, giving amortized O(log n). One Θ(n) operation is still allowed because actual totals include initial minus final potential.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay insertion, deletion, and constraints

- Insert a new BST leaf and splay it to the root.
- To delete, splay the target to the root and remove it, leaving L and R. If L is nonempty, splay its maximum to the root. Its right child is empty, so attach R there.
- An unsuccessful search may splay the last visited node, adapting to the query sequence.
- Reads modify structure. Concurrent read-only access and snapshots require additional design. Strict per-operation latency requirements favor worst-case balanced trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

Complete splay rotations and split/join updates

```
splay(x):
    while x.parent!=null:
        p=x.parent; g=p.parent
        if g==null: rotate x over p
        else if (x is p.left)==(p is g.left):
            rotate p over g; rotate x over p
        else: rotate x over p; rotate x over g
    root=x
insert(k):
    if root==null: root=Node(k,parent=null); return
    search k; splay(found or last visited node)
    if root.key==k: update/reject duplicate; return
    if k<root.key:
        z=Node(k,left=root.left,right=root,parent=null); root.left=null
        if z.left!=null: z.left.parent=z
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete splay rotations and split/join updates (continued)

```
    else:
        z=Node(k,left=root,right=root.right,parent=null); root.right=null
        if z.right!=null: z.right.parent=z
    root.parent=z; root=z
delete(k):
    search and splay target or last visited node; if absent: return
    (L,R)=(root.left,root.right)
    if L!=null: L.parent=null
    if R!=null: R.parent=null
    release root
    if L==null: root=R
    else:
        root=L; splay(maximum(L)); root.right=R
        if R!=null: R.parent=root
```

Empty insertion creates a leaf directly. Split/join restores both child and parent fields. Before releasing the old root, clear the parents of L and R so splay cannot follow a released ancestor. For chain 1→2→3→4, searching 4 first performs two left rotations over 2 and 3, then a zig over 1. The final tree is 4 with left child 1, 1.right=3, and 3.left=2, not a reversed chain.

Complete splay rotations and split/join updates (continued)

The access lemma proves searches. Updates additionally change weights and potential. With unit weights, split insertion adds a root rank at most log(n+1); severed subtrees only reduce existing ranks. Extra potential increase is O(log(n+1)). Deletion's split does not increase ranks, splaying max(L) uses the lemma, and attaching R increases only the new root rank by at most log n. Thus updates also have logarithmic amortized cost, without worst-case single-operation height or nonmutating-read guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Comparison and extensions

Structure	Objective	Search guarantee	Update characteristic
Optimal BST	Known static weighted cost	Optimal for the specified distribution	Rebuilding charged separately
AVL	Local height difference	O(log n) worst	Height maintenance, cascading deletion repairs
Red-black	Black height and red constraints	O(log n) worst	Color and rotation repair
Splay	Entire operation sequence	O(log n) amortized	Searches themselves rotate

Graduate extensions can use fixed nonuniform weights in the access lemma to study access-frequency costs, or investigate optimal BST root monotonicity. The first requires stating how weights relate to query counts. The complete Knuth-optimization proof is outside this edition.

References and further reading

- Sleator and Tarjan, Self-Adjusting Binary Search Trees, JACM 32(3), 1985, § 2: bottom-up splay and the access lemma. Numeric diagrams use this chapter’ s own keys and unit weights.
- The MIT 6.006 2020 index, L7, identifies AVL material. Princeton balanced search trees uses LLRB and its insertion/deletion code must not be mixed with general red-black conventions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prerequisites and objectives

- Prerequisites are BSTs, binary search, ordered sequences, tree height, and disk-page concepts.
- Derive B-tree occupancy and height constraints from multiway searching. Trace splits, borrowing, and merging.
- Distinguish B-tree and B+ tree record placement, separator semantics, and range-scan costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiway search intervals

- An internal node stores increasing keys $k_1 < \dots < k_r$, at most $m-1$ keys, and $r+1$ child subtrees.
- Child intervals are $(-\infty, k_1)$, (k_1, k_2), …, $(k_r, +\infty)$, further restricted by ancestor intervals.
- Binary search within one node uses O(log m) CPU comparisons. An arbitrary m-way tree guarantees neither balance nor minimum occupancy.
- If nodes may retain only one child or leaves have unequal depths, maximum fanout m alone does not imply height $O(\log_m n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

External memory and fanout

- Data is divided into blocks. One I/O transfers one block between main and external memory. B denotes the number of items fitting a block.
- Putting multiple keys and child pointers on one page allows one page read to choose the next-level path. Larger fanout reduces levels.
- With internal and leaf capacities Θ(B), $O(1 + \log_B n)$ counts page I/Os. It does not mean O(1) CPU comparisons within every page.
- Page size, key length, pointers, headers, and fill ratio determine actual fanout. SSD pages and caches also matter, but theoretical bounds alone do not determine latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree minimum degree t

- Use integer t≥2. Every nonroot node has t−1 through 2t−1 keys. An internal node has one more child than keys.
- A nonleaf root has at least one key and two children. A leaf root may be empty. All leaves have the same depth.
- Keys and subtree intervals obey search order. Records may be stored in internal nodes as well as leaves.
- Another text's order m may mean maximum child count. Convert definitions before comparing formulas. It is not automatically our t.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving the B-tree height bound

- For a nonempty tree of edge height h≥1, the root has at least two children and subsequent nonroot internal nodes at least t children.
- Depth d≥1 contains at least $2t^{d-1}$ nodes, each with at least t−1 keys.
- Therefore $n \geq 1 + 2(t-1)\sum_{d=1}^{h} t^{d-1} = 2t^h - 1$, giving $h \leq \log_t((n+1)/2)$.
- If t=Θ(B) and one node fits one block, search follows one root-to-leaf path in $O(\log_B n)$ I/Os for nonsmall trees. Caching the root can save one read. A root-only tree needs O(1) I/Os.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search checks both keys and child intervals

1. Find the first node key not less than x, at position i.
2. On equality, return the record. If not equal and at a leaf, report absence.
3. Otherwise descend into child i and repeat.

Root keys `[20,50]` direct query 37 to the middle child. A B-tree search can succeed at an internal key. Our B+ search must continue to a leaf.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Split a full child before descending

- Use top-down insertion. If the root is full, create a new root and split the old root before descent.
- A full node contains $2t-1$ keys. Promote its tth key to the parent, leaving $t-1$ keys on each side with corresponding children.
- Split full children before entering them, so the eventual leaf has space. The parent also has space by the descent invariant.
- Splitting preserves the key set and equal leaf depth. Only a root split increases overall height.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

Worked t=2 insertion trace

- Insert `10,20,5`, giving full leaf root `[5,10,20]`.
- Before inserting 6, split the root and promote 10. Leaves become `[5]` and `[20]`. Insert 6 into the left leaf to get `[5,6]`.
- Insert 12 and 30, filling the right leaf `[12,20,30]`. Before inserting 25, split it and promote 20.
- The final root is `[10,20]`, with leaves `[5,6]`, `[12]`, `[25,30]`. Internal keys 10 and 20 are not also retained in leaves.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete B-tree split and insertion

```
split_child(parent,i):
    y=parent.children[i]; require len(y.keys)==2*t-1
    z=Node(leaf=y.leaf,keys=[],children=[])
    median=y.keys[t-1]
    z.keys=y.keys[t:]; y.keys=y.keys[:t-1]
    if not y.leaf:
        z.children=y.children[t:]; y.children=y.children[:t]
    insert median at parent.keys[i]
    insert z at parent.children[i+1]
insert_nonfull(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k: return already_present
    if x.leaf: insert k at i; return
    if x.children[i] is full:
        split_child(x,i)
        if k>x.keys[i]: i+=1
        else if k==x.keys[i]: return already_present
    insert_nonfull(x.children[i],k)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete B-tree split and insertion (continued)

If the root is full, first create a new root with the old root as its sole child, split_child(newroot,0), then call insert_nonfull. Internal splitting divides child pointers too: t children per side correspond to t−1 keys. Only a root split raises all leaf depths.

After the chapter sequence at t=2, root=[10,20], leaves=[5,6],[12],[25,30]. Insert 7 into [5,6,7]. Before inserting 8, split that leaf and promote 6: root=[6,10,20], leaves=[5],[7,8],[12],[25,30]. The parent is now full but this insertion is done; the next insertion splits the root before descending. Top-down full-node splitting is distinct from overflow-first splitting.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Strengthen a child before deletion descent

- If the destination child has only t−1 keys, borrow or merge first so deletion cannot violate minimum occupancy.
- If an adjacent sibling has at least t keys, rotate one key through the parent separator, moving a child pointer too when necessary.
- If adjacent children both have t−1 keys, merge them with their parent separator to obtain 2t−1 keys.
- Borrowing preserves height. Merging reduces the parent's key count. The top-down invariant keeps nonroot parents legal, with a separate root case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Internal-key deletion and root contraction

- For an internal target key, if the left child has at least t keys, replace the target with its predecessor and delete that predecessor in the left subtree.
- Otherwise use the successor if the right child has at least t keys. If both have t−1 keys, merge them with the target and delete within the merged node.
- A root with zero keys and one child is replaced by that child, reducing height by one. An empty leaf root represents an empty tree.
- Only constantly many node reorganizations occur per level. I/O is $O(1 + \log_t n)$. With array movement of t keys per node, CPU work can be $O(t(1 + \log_t n))$. The additive constant covers a root-only tree.

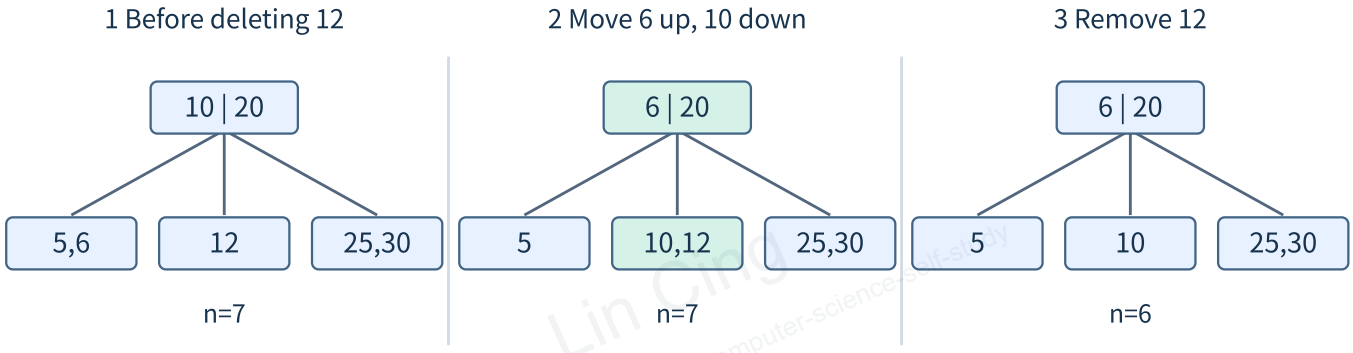
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked borrowing trace

- Delete 12 with t=2 from root `[10,20]` and leaves `[5,6]`, `[12]`, `[25,30]`.
- The middle leaf has only one key. Borrow from the left before descending. Parent key 10 moves down, and left-sibling maximum 6 moves up.
- The state becomes root `[6,20]`, leaves `[5]`, `[10,12]`, `[25,30]`. Delete 12, leaving middle leaf `[10]`.
- Every nonroot node has at least one key, all leaves have equal depth, and exactly 12 has left the key set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree borrowing moves two data keys



With t=2, every nonroot leaf needs 1 to 3 keys. Before descending into [12], move the parent's data key 10 into the middle leaf and the left sibling's maximum data key 6 into the vacated parent slot. Two distinct records move; 6 is not copied while also remaining in the left leaf. Only after state 2 has [10,12] may deletion remove 12, producing state 3.

B-tree borrowing moves two data keys (continued)

Inorder is [5,6,10,12,20,25,30] in both first states, then [5,6,10,20,25,30]. Borrowing neither loses nor duplicates a record; deletion removes exactly one. All leaves remain at depth 1. An internal sibling also transfers its boundary child pointer, which the leaf picture does not show. A B+ parent separator is only a routing copy, so its leaf-borrowing rule must not be replaced by this B-tree data movement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree deletion borrowing, merging, and root contraction

```
prepare_child(x,i):
    child=x.children[i]
    if len(child.keys)>=t: return i
    if i>0 and len(x.children[i-1].keys)>=t:
        left=x.children[i-1]
        child.keys.insert(0,x.keys[i-1]); x.keys[i-1]=left.keys.pop()
        if not child.leaf: child.children.insert(0,left.children.pop())
    else if i+1<len(x.children) and len(x.children[i+1].keys)>=t:
        right=x.children[i+1]
        child.keys.append(x.keys[i]); x.keys[i]=right.keys.pop(0)
        if not child.leaf: child.children.append(right.children.pop(0))
    else:
        if i>0: merge_children(x,i-1); return i-1
        merge_children(x,i); return i
    return i
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree deletion borrowing, merging, and root contraction (continued)

Delete directly at a leaf. At an internal target, use a predecessor if the left child has at least t keys, otherwise a successor if the right has at least t; if both have t−1, merge them with the target before descent. For an absent local target, prepare the destination and use its returned index, since merging left shifts i. Contract an empty root with one child.

Deleting 12 first borrows 6 upward from [5,6], moving 10 downward. The result has root [6,20] and leaves [5],[10],[25,30]. To delete 5 next, merge [5]+6+[10] into [5,6,10], leaving root [20], then remove 5 to obtain [6,10]. Delete 25 from the right leaf to obtain [30], and 6 from the left to obtain [10]. Before deleting 10, merge [10]+20+[30] into [10,20,30]. The root empties and contracts to that leaf. Removal leaves [20,30].

Borrowing preserves order and leaf depth. Merging produces exactly2t−1 keys. One-page nodes give constant I/O per level and $O(1 + \log_t n)$ total, while intra-page shifts may cost O(t) per level.

B-tree root entry and complete deletion recursion

```
insert(k):
    if root is full:
        old=root; root=Node(leaf=false,keys=[],children=[old])
        split_child(root,0)
    insert_nonfull(root,k)
erase_from(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if x.leaf: remove x.keys[i]; return
        if len(x.children[i].keys)>=t:
            p=maximum_key(x.children[i]); x.keys[i]=p
            erase_from(x.children[i],p)
        else if len(x.children[i+1].keys)>=t:
            s=minimum_key(x.children[i+1]); x.keys[i]=s
            erase_from(x.children[i+1],s)
```

B-tree root entry and complete deletion recursion (continued)

```
        else:
            merge_children(x,i); erase_from(x.children[i],k)
    else:
        if x.leaf: return absent
        i=prepare_child(x,i); erase_from(x.children[i],k)
erase(k):
    erase_from(root,k)
    if len(root.keys)==0 and not root.leaf:
        old=root; root=old.children[0]; release(old)
merge_children(x,i):
    left=x.children[i]; right=x.children[i+1]
    left.keys+= [x.keys[i]] + right.keys
    if not left.leaf: left.children+=right.children
    remove x.keys[i] and x.children[i+1]; release(right)
```

An empty B-tree has a zero-key leaf root, so erasing from it reports absence. Keys abbreviate complete key-value records; move the complete pair in an implementation. Merging two children with t−1 keys yields (t−1)+1+(t−1)=2t−1 keys, exactly within capacity. The intervening parent key is a real record, unlike a B+ routing separator.

B-tree root entry and complete deletion recursion (continued)

The descent invariant gives each nonroot recursive entry at least t keys. Prepare_child borrows or merges before entering it. An internal-key predecessor/successor branch chooses a child with at least t keys, leaving at least t−1 after removal. If neither can lend, merge both children with the target key first. A left merge must return i−1 because the old child[i] was released. An absent-key deletion may still restructure the search path; the abstract key set remains unchanged, while physical shape need not.

For root [20] and leaves [10],[30] with t=2, deleting 10 first merges both children and parent record 20 into [10,20,30]. Deletion leaves [20,30], and the empty internal root contracts to that leaf. Dropping separator 20 during merging would lose an undeleted record. Same-level sibling restructuring preserves equal leaf depth; root contraction changes the common depth for the entire tree.

B+ records stay in leaves

- All key-value records are in leaves. Internal nodes contain only separators and child pointers.
- A separator equals the minimum key of its right child subtree. On equality, descend right. Do not mix this with a left-maximum convention.
- Leaves are linked in key order, enabling sequential range scanning after locating the first leaf.
- Separators are routing information and can duplicate leaf keys. Counting internal keys plus leaf keys does not count data records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

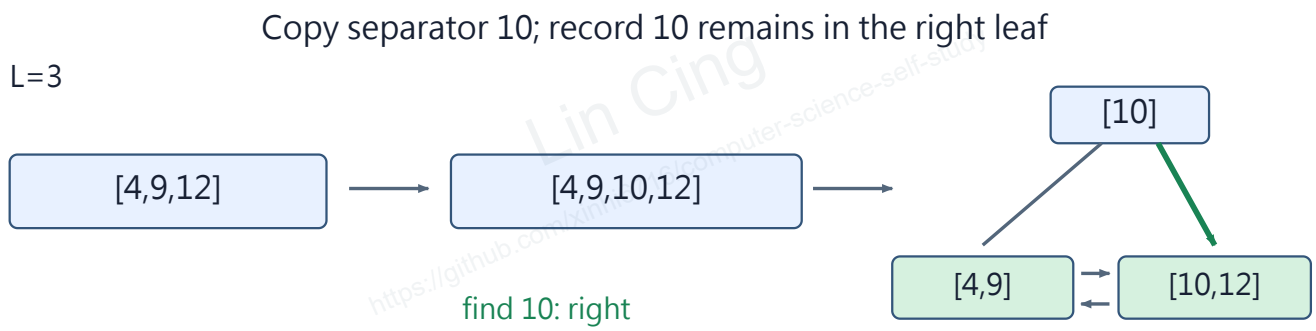
Specify internal and leaf capacities separately

- Use integers $M \geq 3$, $L \geq 2$. Nonroot internal nodes have a through M children, where $a = \lceil M/2 \rceil \geq 2$. An internal root has 2 through M children. Internal key count is child count minus one. Minimum fanout at least two supports logarithmic height.
- Nonroot leaves contain $\lceil L/2 \rceil$ through L records. A leaf root may have 0 through L.
- M and L may differ. Internal entries store child pointers, while leaves may store record pointers or full records.
- After a leaf split, copy the right leaf's minimum into the parent. Internal splits promote a partition boundary and redistribute children. They do not use the same key-copy rule as leaf splits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked B+ leaf split

- With L=3, inserting 10 into `[4,9,12]` gives temporary `[4,9,10,12]`.
- Split into `[4,9]` and `[10,12]`, copy separator 10 to the parent, and repair the leaf chain.
- If these are the only two children, root `[10]` routes a query for 10 rightward to the actual record.
- Unlike B-tree promotion, copying 10 into the parent does not remove it from its leaf.

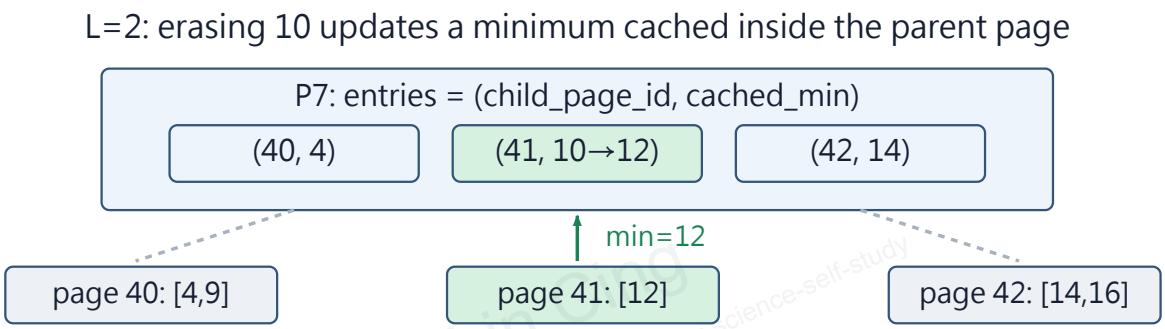


Disk-page layout and local minimum updates

```
// Internal page: entries[j]=(child_page_id,cached_min), all in this page.
// Separators are entries[1:].cached_min; no persistent parent pointers.
refresh_local(p):
    if p.leaf: p.minimum=p.records[0].key if nonempty else absent
    else: p.minimum=p.entries[0].cached_min
entry(p): return (p.id,p.minimum)
find_leaf(k):
    path=[]; p=read(root_id)
    while not p.leaf:
        i=upper_bound(p.entries[1:].cached_min,k)  // equality goes right
        path.push((p.id,i)); p=read(p.entries[i].child_page_id)
    return path,p
```

Each node occupies one disk page. An internal page stores every child page ID and an exact copy of that child's minimum inside the parent page. Separators are the cached minima after entry 0, so no child read is needed to obtain them. A leaf stores ordered records and next/prev page IDs. Changed children return new minima up an ancestor stack; unchanged entries retain their values.

Disk-page layout and local minimum updates (continued)



Gray dashes are page-id links; green returns the already-read leaf minimum.
Read P7 and leaf 41 only; rebuild [12,14] from minima stored inside P7.

With M=3,L=2, page P7 initially stores (40,4),(41,10),(42,14), and leaf 41 holds [10,12]. Erasing 10 leaves [12], meeting minimum occupancy 1. Update entry 1 to (41,12), changing separators [10,14] to [12,14]. Pages 40,42 and their cached minima are unchanged and need not be read.

Disk-page layout and local minimum updates (continued)

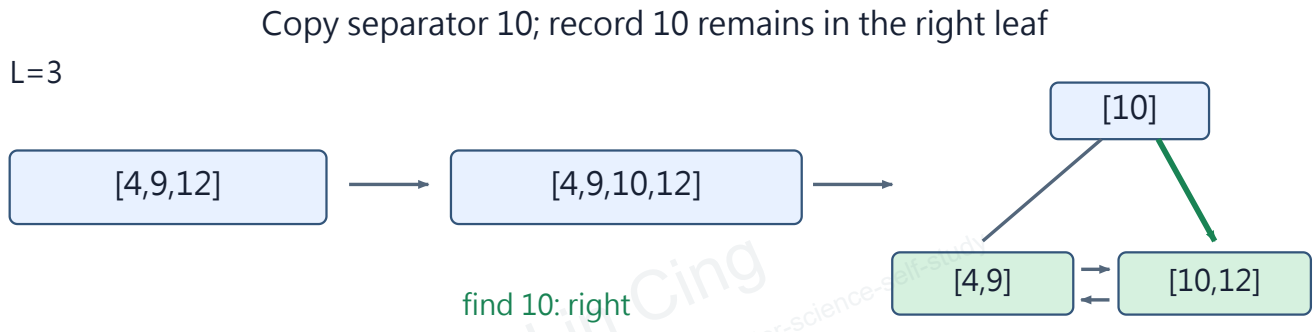
Reading M separate child.minimum fields can add M cold page reads to one refresh. Rewriting parents of Θ(M) children moved by an internal split likewise adds Θ(M) I/Os. Calling a field cached does not place it in the parent page. This layout explicitly places the copies locally and removes persistent parent fields. Store only (page_id,index) in the ancestor stack, rereading one parent on ascent.

The reason for avoiding child-page parent rewrites also appears in Open Data Structures § 14.2.2. The B+ cached-entry and leaf-chain layout here is an explicit teaching model.

B+ insertion, exact separators, and upward splits

```
insert(k,v):
    path,node=find_leaf(k)            // stack holds (parent_page_id,index)
    insert/update (k,v) in sorted node.records
    while true:
        if node exceeds L records or M entries:
            parts=split_into_floor_ceil_halves(node)
            if node.leaf: splice both leaf links and repair adjacent leaf
        else: parts=[node]
        for p in parts: refresh_local(p); write(p)
        if path empty:
            if len(parts)==1: root_id=parts[0].id
            else:
                r=new_internal(entries=[entry(p) for p in parts])
                refresh_local(r); write(r); root_id=r.id
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        parent.entries[i:i+1]=[entry(p) for p in parts]
        node=parent
// entry(p)=(p.id,p.minimum); internal splits move complete cached entries.
```


B+ insertion, exact separators, and upward splits (continued)



With L=3, insert 10 into [4,9,12], overflow to [4,9,10,12], then split into leaves [4,9],[10,12] under root [10]. Inserting 14 gives right leaf [10,12,14]. Inserting 16 splits it into [10,12],[14,16], and the root becomes [10,14]. The parent keys 10 and 14 are routing copies. Both records remain in leaves. Routing intervals are (−∞,10), [10,14), [14,+∞).

B+ insertion, exact separators, and upward splits (continued)

Use the preceding disk layout: each internal page stores (child_page_id,cached_min) entries locally; child pages have no parent field. The left half keeps the original page ID and the right gets a new page. Splitting moves complete entries with their cached minima. Refresh each new page from its own data and return only changed entries to the parent. Never fetch other child pages for their minima or rewrite moved children's on-disk parents.

Find_leaf saves ancestor page IDs and descent indices, rather than pinning all ancestor pages. On ascent reread one parent, replace the old entry with one or two returned entries, and split upward if needed. Only root overflow creates a new root. Propagate exact minima even without overflow. Nonroot internal fanout is at least a=ceil(M/2), leaf occupancy at least ell=ceil(L/2); $M \geq 3, L \geq 2$ make both split halves legal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ insertion, exact separators, and upward splits (continued)

Each level touches constantly many pages. Doubly linked leaf splicing adds at most one adjacent-leaf page. Insertion therefore takes O(h+1) page I/Os in this layout, while intra-page array work is O(L+Mh) worst case. The ancestor stack stores O(h+1) words plus constantly many page buffers of O(M+L) words. Fix record, entry, and page sizes before translating a RAM representation into disk costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deletion and boundary updates

- After leaf underflow, first try borrowing from an adjacent leaf. Otherwise merge and remove the parent's separator.
- Even without underflow, changing a right-subtree minimum requires updating the corresponding ancestor boundary under our exact-minimum convention.
- In a separate L=2 example, deleting 10 from right leaf `[10,12]` leaves `[12]`, which meets minimum occupancy 1. Still change the parent separator from 10 to 12.
- Some systems allow an older but still valid fence key. That convention differs. Exact right minima are incompatible with a claim that parent keys never need updating.

B+ deletion borrowing, merging, and range retrieval

```
erase(k):
    path,node=find_leaf(k)
    if k absent: return
    remove record k from node
    while true:
        refresh_local(node)
        if path empty:
            if node internal and len(node.entries)==1:
                root_id=node.entries[0].child_page_id; free(node)
            else: write(node)
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        if node meets its nonroot minimum occupancy:
            write(node); parent.entries[i]=entry(node)
        else:
```

B+ deletion borrowing, merging, and range retrieval (continued)

```
            read adjacent sibling(s) of the same parent
            if a sibling can lend:
                move a boundary record or complete cached entry
                refresh_local both; write both; update their parent entries
            else:
                merge adjacent records or cached-entry arrays
                repair both leaf links if leaves
                refresh_local/write merged page; free the other page
                replace their two parent entries by entry(merged)
        node=parent
range(lo,hi):
    locate first record>=lo
    scan leaf links until key>hi or leaf chain ends
```

Let L=3, with root [10,14], leaves [4,9],[10,12],[14,16], and minimum occupancy 2. Delete 10 to leave [12]. Neither sibling can lend, so merge with the left leaf into [4,9,12]. The root becomes [14], and leaf links join [4,9,12]↔[14,16]. Delete 14, leaving [16]. The left leaf can now lend 12, producing [4,9]↔[12,16] and root separator 12. Exact separators also update when a minimum changes without underflow.

B+ deletion borrowing, merging, and range retrieval (continued)

Borrow or merge complete local entries without reading moved child pages or changing persistent parent fields. After borrowing, continue minimum propagation through the saved stack even though underflow is repaired. Internal merging uses at most 2a−1≤M entries, leaf merging at most $2\ell - 1 \leq L$ records; same-level merging preserves leaf depth. Contract a one-child root by adopting its child page ID. Constantly many parent/sibling/neighbor-leaf page accesses per level give O(h+1) I/Os, with O(L+Mh) worst-case RAM shifts.

For range [9,14], locate 9, follow next to 12, and stop at 16>14, returning [9,12]. With general height h and minimum leaf occupancy ell, index I/Os are $O(h + 1 + k/\ell)$. Only M,L both proportional to block capacity B justify $O(1 + \log_B n + k/B)$. Full records stored elsewhere add D data-page reads, worst-case D=k. Output work remains Θ(k).

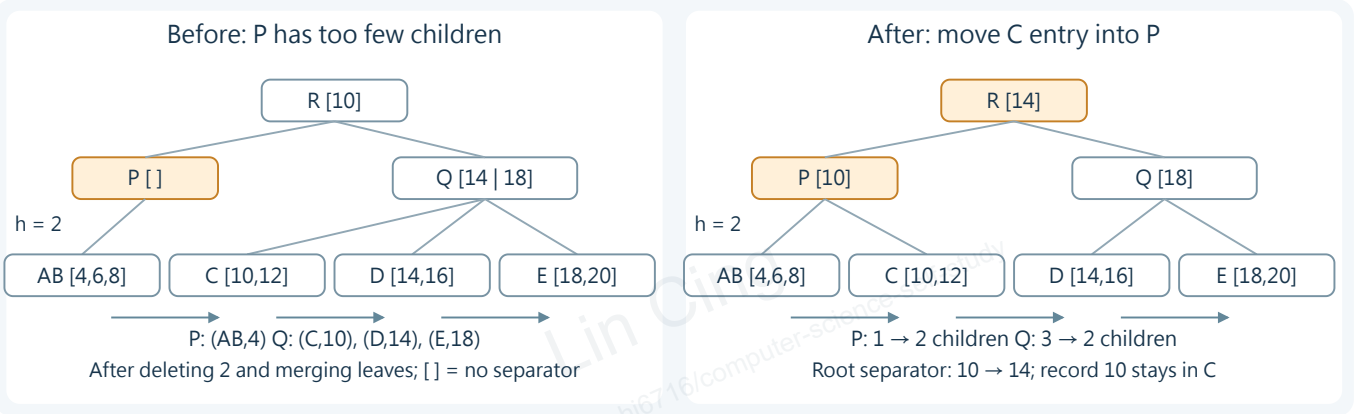
Internal underflow: count children, not just separators

Set M=4 and L=3. Each nonroot internal page needs at least two children; each nonroot leaf needs at least two records. Root R has children P,Q. P owns leaves A=[2,4], B=[6,8]; Q owns C=[10,12], D=[14,16], E=[18,20]. Every leaf is at depth two, and R has separator 10.

Step	Actual page changes	Why repair continues
Delete 2	A becomes [4]	One record is below the leaf minimum two
Inspect B	B=[6,8] cannot lend	Lending would leave B below minimum too
Merge A and B	Keep A's page ID as AB=[4,6,8]; free B	Three records fit L exactly
Update P	P.entries=[(AB,4)]; P.minimum=4	**One child is below the internal minimum two**

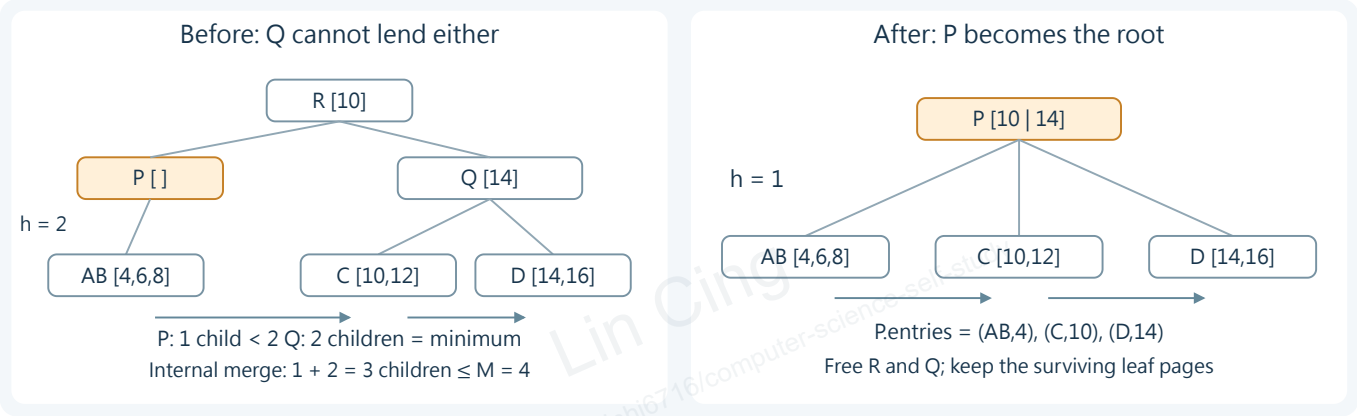
P has no separator but still points to leaf AB; it is not an empty leaf. Replace chain A↔B↔C with AB↔C and set C.prev to AB. The root temporarily holds (P,4),(Q,10). This is an intermediate update state; repair P before exposing a valid index.

Internal borrowing moves an entry, not a leaf record



Q has three children and can lend its first entry (C,10). Append it to P, giving P=[(AB,4),(C,10)] and Q=[(D,14),(E,18)]. Both now have two children. P's separator is 10; Q's is 18. Replace root entries (P,4),(Q,10) with (P,4),(Q,14).

When neither sibling can lend: merge and contract the root



Independently omit E, keeping A,B,C,D and the same capacities. Delete 2 and merge A/B: P=[(AB,4)], Q=[(C,10),(D,14)]. Q cannot lend. Concatenate into P=[(AB,4),(C,10),(D,14)], with separators [10,14], and free Q. R now has only (P,4): set root_id=P and free R. Height drops from two to one.

Check the rebuilt index through searches and the leaf chain

Each branch deletes only record 2. Internal repair moves page IDs and cached minima; C still physically stores [10,12]. The old parent separator 10 is not an extra record to insert into a leaf.

Check	After internal borrowing	After merging and root contraction
Internal pages and height	R=[14]; P=[10], Q=[18]; h=2	New root P=[10,14]; h=1
Search for 10	Left from R to P; equality goes right to C	Equality at P's 10 goes right to C
Leaf chain	AB↔C↔D↔E; nine records	AB↔C↔D; seven records
Record 10	Appears once, in C	Appears once, in C

In the contracted tree, range [8,15] first finds 8 in AB=[4,6,8]. Follow next to C=[10,12], then D=[14,16]. Emit 8,10,12,14 and stop upon inspecting 16>15. Diagram arrows show only next; prev points to the preceding page in the same chain. Internal repair does not change these leaf links.

Why internal repair preserves records, order, and leaf depth

Entries are ordered by subtree minima. Borrowing the right sibling's first entry shifts the boundary one subtree right; borrowing the left sibling's last entry does the reverse. Subtrees retain their contents and equal depth. Rebuild exact separators from the resulting entry arrays, and continue propagating changed minima into ancestor entries.

Write a=ceil(M/2). An underfull internal page has a−1 children; a sibling unable to lend has a. Their merge has 2a−1≤M children: M−1 for even M, M for odd M. Here 1+2=3≤4. **B+ merging concatenates child entries; it does not insert the parent separator as another data record.** The merged page automatically has one fewer separator than children.

At each level inspect the current page, its parent, and at most two adjacent siblings. Leaf merging additionally repairs constantly many chain fields. With no persistent child.parent field, moving an entry does not rewrite that child's page. The total is O(h+1) page I/Os; exact read/write counts still depend on buffer residency. Root contraction alone reduces every leaf's depth together by one.

B+ height needs fanout and leaf-occupancy assumptions

```
a=ceil(M/2) >= 2; ell=ceil(L/2) >= 1
h=number of edges on a root-to-leaf path
if h>=1:
    leaf_count >= 2*a^(h-1)
    n >= 2*ell*a^(h-1)
    h <= 1+log_a(n/(2*ell))
// Root-only trees h=0 cost one page, including the empty tree.
```

An internal root has at least two children; each later internal page has at least a. Depth h therefore has at least $2a^{h-1}$ leaves, each with at least ell records. Treat a leaf root, h=0, separately. A cold lookup reads h+1 pages, giving the general bound $O(1+\log_a(1+n/\ell))$. M≥3,L≥2 alone do not permit changing the logarithm base to B.

For any integer q≥3, choose M=3,$L=B=2^q$, exactly two children per internal page, and 2^q full B-record leaves. This legal tree has h=q and n=B^2. A cold lookup reads q+1 pages, while $\log_B n$ stays 2. The claimed uniform $O(\log_B n)$ bound fails because minimum internal fanout remains 2 instead of growing with B.

B+ height needs fanout and leaf-occupancy assumptions (continued)

To simplify, additionally require fixed-size keys/entries/records, one node per page, M=Θ(B), and L=Θ(B). Then $a, \ell = \Theta(B)$, so height is $O(1 + \log_B n)$. A range with k outputs touches at most two partially used end leaves; every fully scanned interior leaf contributes at least ell outputs. Thus general index cost is $O(h + 1 + k/\ell)$, simplifying to $O(1 + \log_B n + k/B)$ only under both capacity assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Point-query and range-query I/O

Locating a leaf costs O(h+1) pages under the general height bound just proved. Index range cost is $O(h + 1 + k/\ell)$. Only M=Θ(B),L=Θ(B), fixed-size entries/records, and one node per page permit $O(1 + \log_B n + k/B)$.

Record pointers can add D data-page reads, worst-case D=k. Clustered fixed-size records at Θ(B) per page can instead bound those data pages by O(1+k/B). Producing k outputs still needs Ω(k) basic output work; CPU and page-I/O counts have different units.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Simplified B+ trees and database implementations

- PostgreSQL 18 documentation describes multilevel B-tree indexes whose pages form a doubly linked list at each level. Leaf tuples point to table rows and internal tuples to lower-level pages.
- This supports the practical motivation for multiway page indexes. It is not a guarantee that our simplified deletion steps reproduce PostgreSQL internals.
- Databases also need concurrency control, consistent reads during splits, WAL, and crash recovery. A single-threaded pointer diagram does not handle them.
- This chapter does not provide a deployable disk-index engine. Exercises use explicit capacities and update sequences to check invariants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources and definition mapping

- External-memory motivation and minimum-occupancy height reasoning were checked against Pat Morin's Open Data Structures § 14.2. Its B-to-2B child convention maps to our t-to-2t convention. Our B remains block capacity.
- The database example follows PostgreSQL 18 B-tree implementation, using only page and tuple-placement facts.
- The B+ M, L capacities, separator rule, and example are independently specified.
- Exercises and worked solutions are separate. Every problem uses the capacity conventions stated in this chapter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study